Oracle 19c Real Application Clusters

RAC 기술 아키텍처

아키텍처 개관

개요

- 단일 인스턴스와 RAC 의 차이를 인스턴스·데이터베이스 관계로 말한다
- 노드·인터커넥트·공유 스토리지, Clusterware·ASM·DB 세 층을 그린다
- 공유되는 파일과 인스턴스별 구성 요소(리두 스레드, UNDO)를 가른다
- Cache Fusion 이 블록을 옮기는 경로와 gc 대기의 뜻을 안다
- 공용 IP·VIP·SCAN·사설 IP 가 각각 무엇을 막는지 안다
- OCR·OLR·Voting File 의 위치와 읽는 주체를 안다
- Clusterware 의 두 스택과 기동 순서로 장애 계층을 판별한다

Agenda

- 1 RAC 데이터베이스 구성
- 2 RAC 데이터베이스 파일
- 3 클러스터와 네트워크
- 4 OCR 과 Voting File
- 5 Clusterware 프로세스와 도구
- 6 ASM
- 7 Fleet Patching and Provisioning

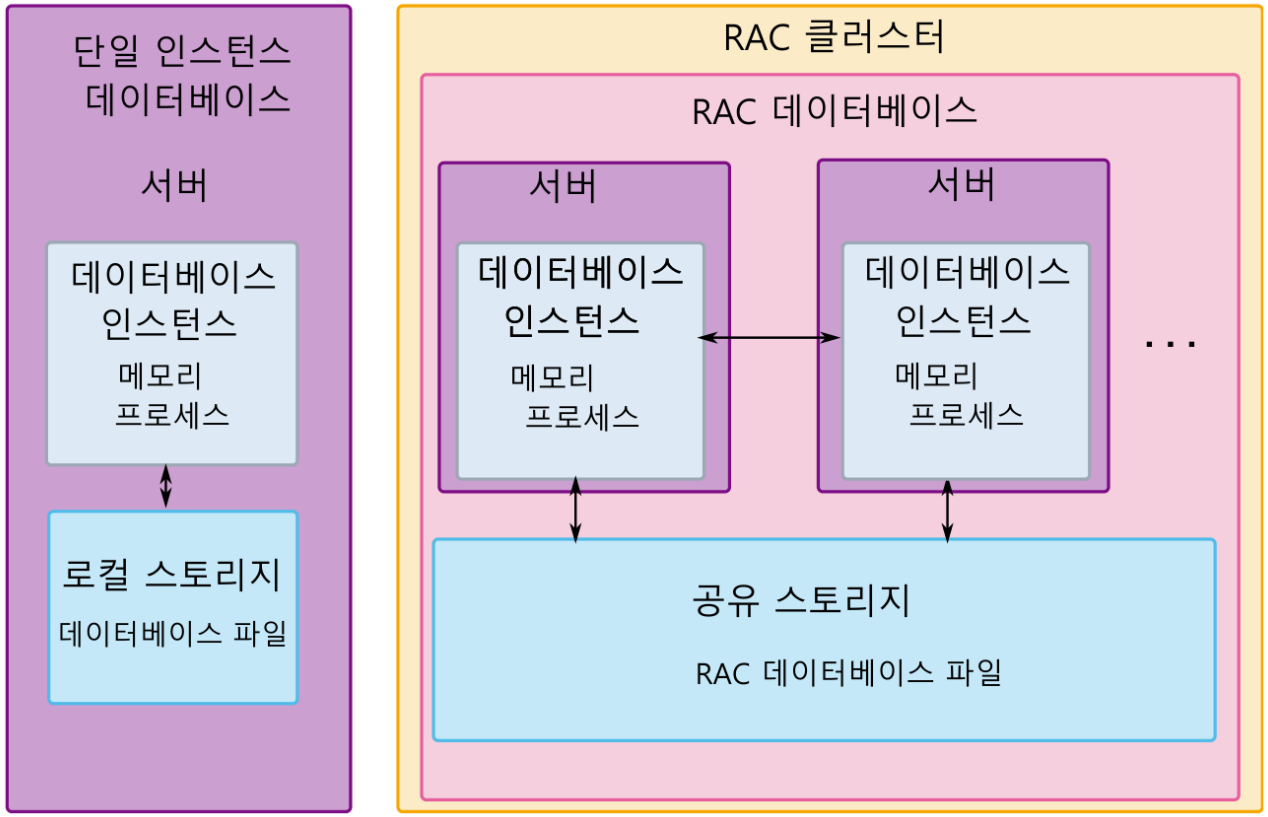
1

RAC 데이터베이스 구성

- 단일 인스턴스와 RAC
- 확장성과 가용성
- 인스턴스 안의 글로벌 프로세스와 Cache Fusion
- RAC One Node 와 관리 도구

데이터베이스 하나, 인스턴스는 여럿

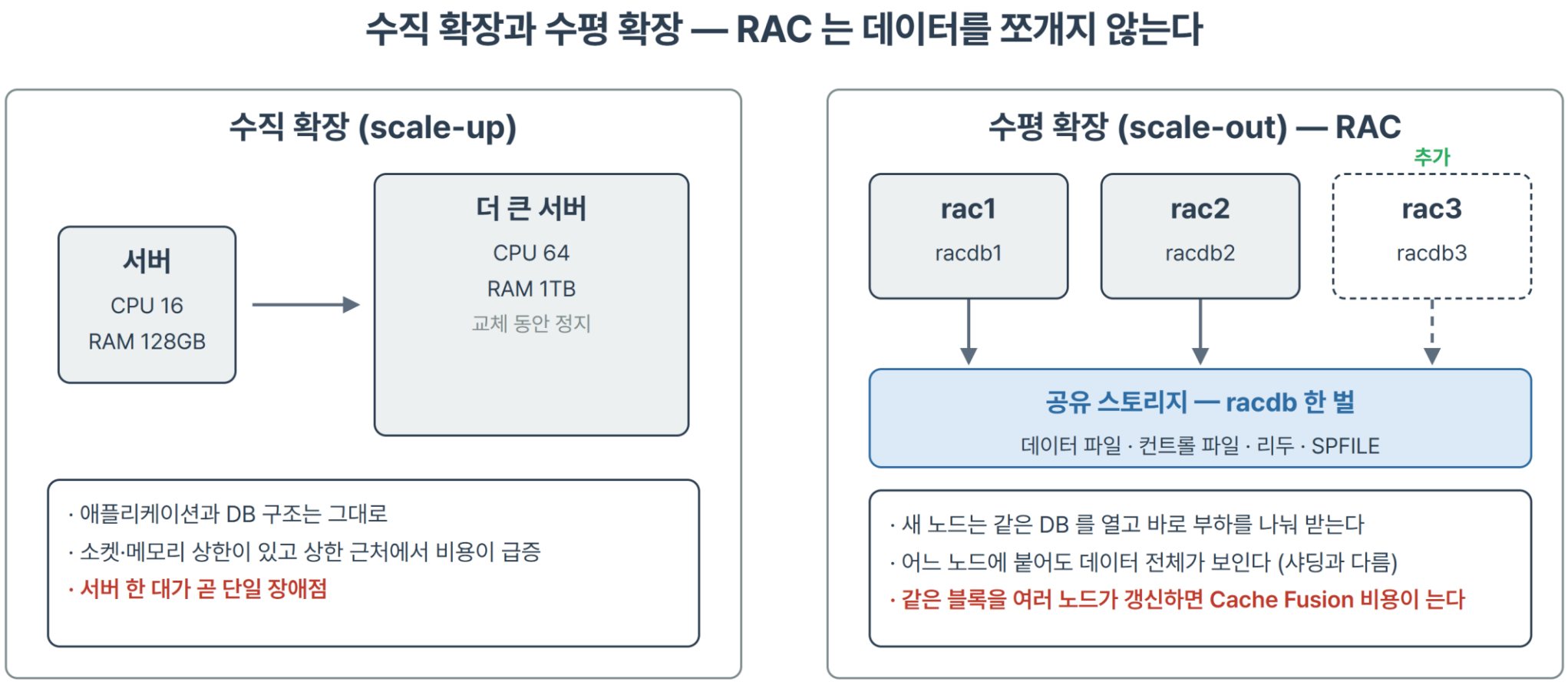
단일 인스턴스는 1:1, RAC 는 1:N



- 단일 인스턴스 — 소프트웨어·파일·메모리·프로세스가 서버 한 대에
- RAC — 파일은 공유 스토리지에 한 벌, 인스턴스는 서버마다
- 인스턴스는 최대 100개, 모두 같은 RAC 클러스터 소속

수직 확장과 수평 확장

RAC 는 데이터를 쪼개지 않는다



확장은 선형이 아니다 — 노드가 늘수록 노드 사이에 오가는 블록·잠금 메시지도 는다
그래서 서비스로 업무를 노드별로 나눠 같은 블록을 두 노드가 번갈아 갱신하는 일을 줄인다

무엇을 막고 무엇을 못 막는가

RAC · Data Guard · 백업은 서로를 대신하지 않는다

RAC · Data Guard · 백업 — 막는 위험이 다르다

RAC

같은 데이터센터, 같은 스토리지

막는 것

- 노드 하드웨어 고장, OS 패닉
- 인스턴스 이상 종료
- 롤링 패치 중 노드 하나 정지

못 막는 것

- 공유 스토리지 손상
- 데이터센터 전체 정지

Data Guard

다른 사이트, 다른 스토리지

막는 것

- 스토리지·사이트 장애
- 주 DB 전체 손상

못 막는 것

- 잘못된 DELETE 도 그대로 전파
- 순간적인 세션 단절

백업 · 플래시백

시간을 되돌린다

막는 것

- 잘못된 DML · DROP
- 논리적 손상
- 특정 시점 복구

못 막는 것

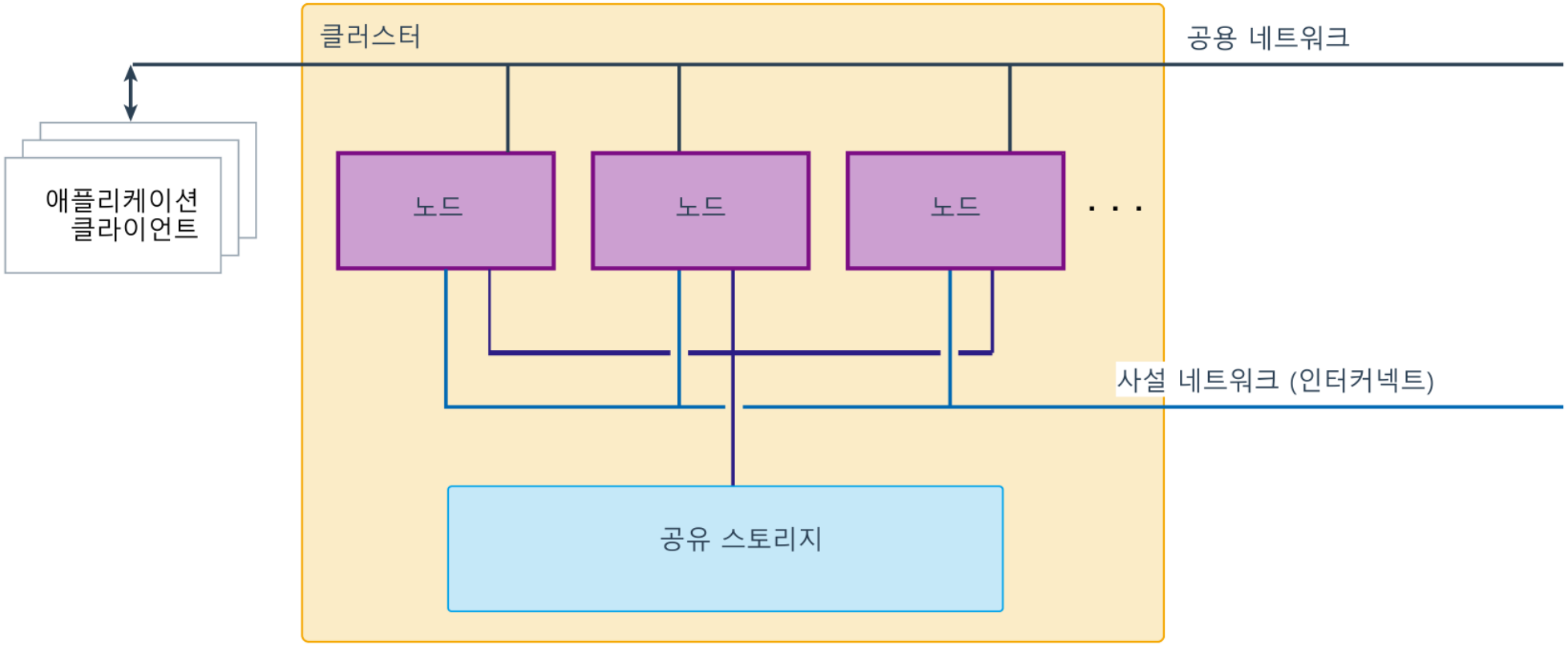
- 복구 시간만큼의 정지
- 서비스 연속성

RAC 는 하나의 스토리지를 여러 노드가 함께 쓰는 구조라 노드는 지켜도 데이터 사본은 늘리지 않는다

세 가지는 서로를 대신하지 않는다 — 운영 DB 는 보통 셋을 함께 둔다

RAC 클러스터 개요

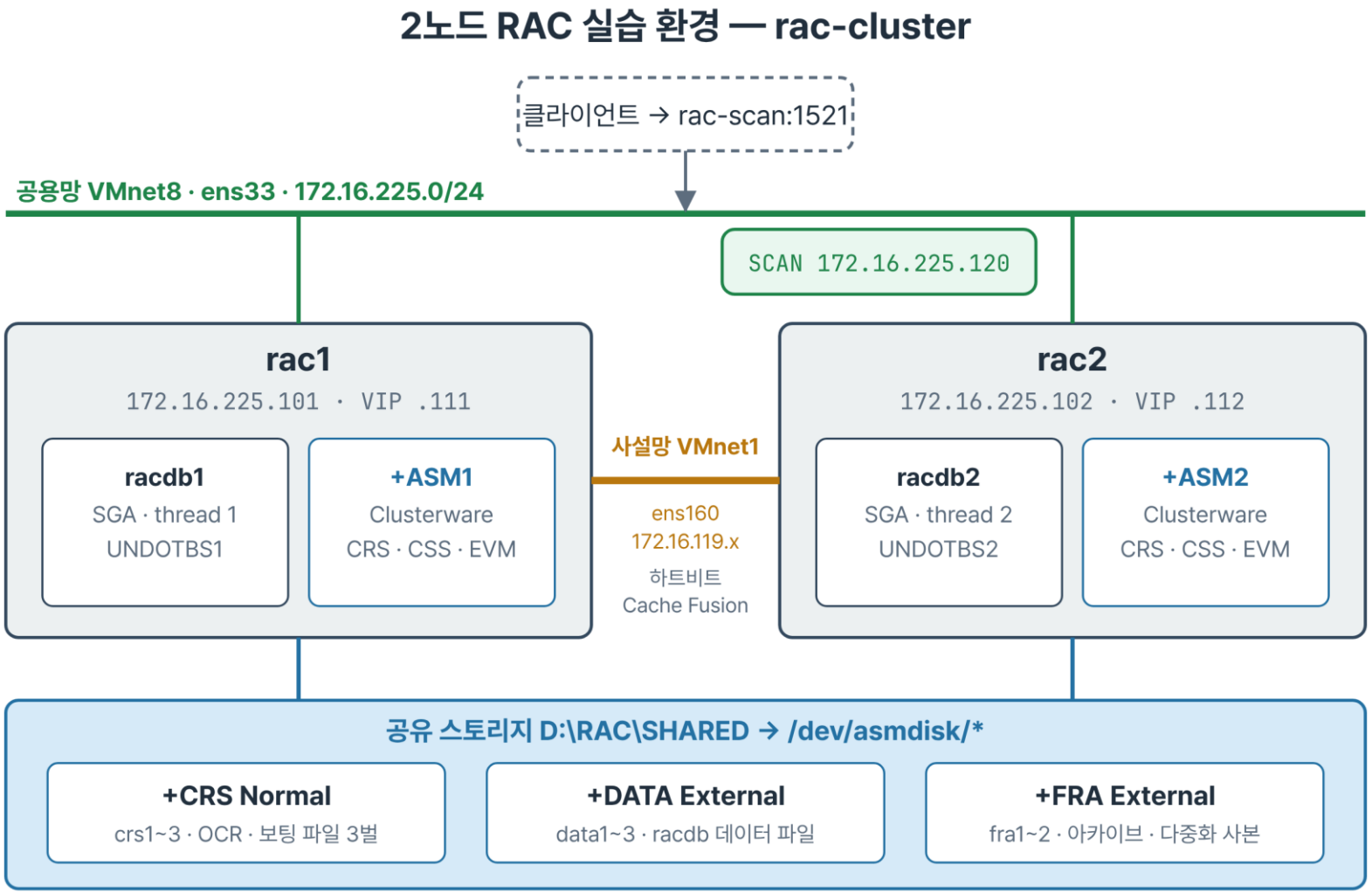
노드마다 GI 를 로컬에 설치하고, 파일은 공유 스토리지에



- 공용 네트워크 — 애플리케이션 접속
- 사설 네트워크(인터커넥트) — 노드 사이 통신
- 네트워크마다 NIC·스위치 이중화 권장

실습 클러스터 rac-cluster

이 과정의 모든 예시 값



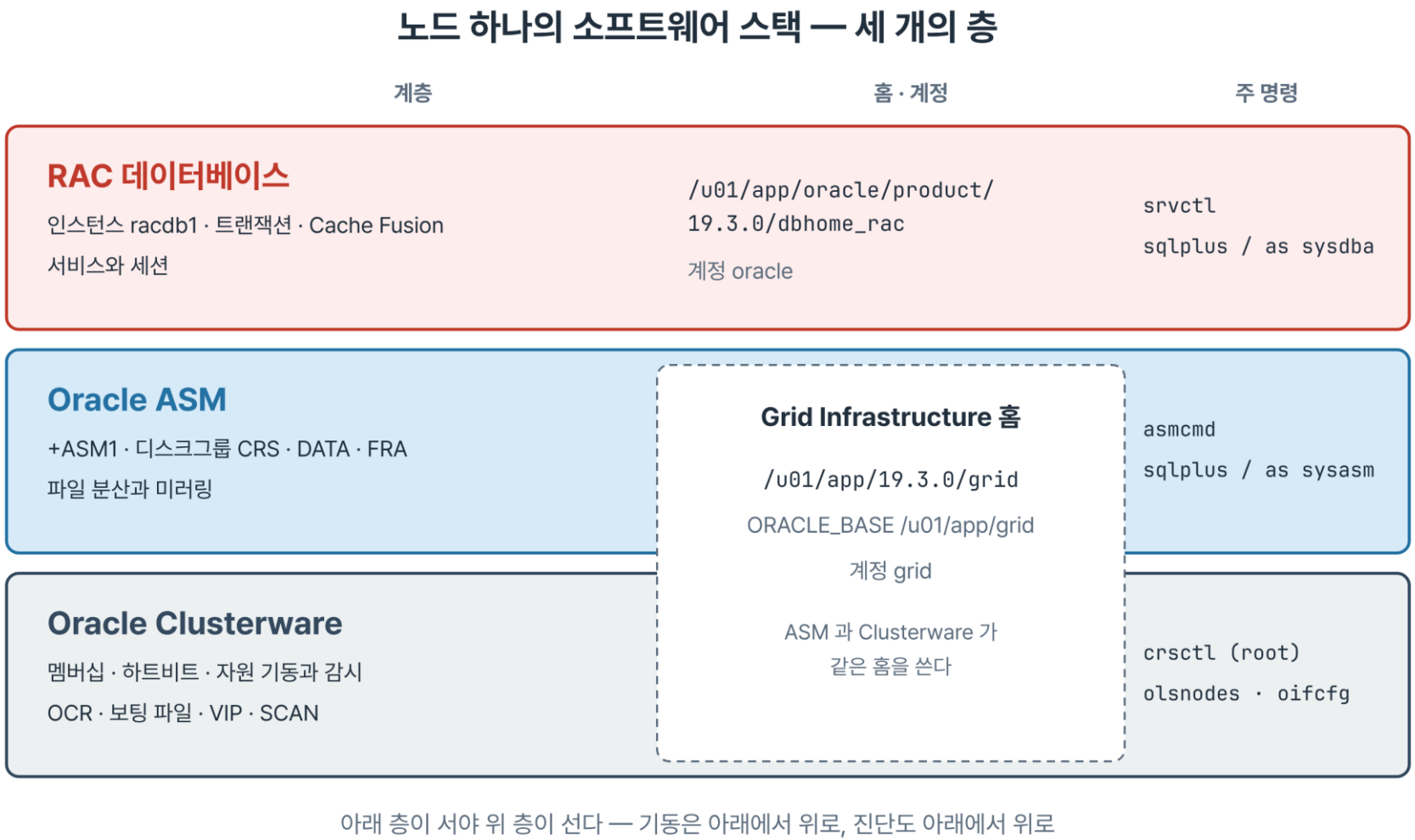
노드 · 인터커넥트 · 공유 스토리지

물리 요소 셋

- 노드 — 같은 OS·패치·Oracle 버전, 노드 목록은 olsnodes -n -s -t
- 인터커넥트 — 하트비트와 Cache Fusion 블록·잠금 메시지
- 인터커넥트에 백업·파일 복사 금지 — 순간 지연이 노드 재부팅으로
- 공유 스토리지 — 모든 노드가 같은 디스크를 동시에 읽고 쓴다
- 장치 이름 고정 — udev 로 /dev/asmdisk/crs1 같은 이름

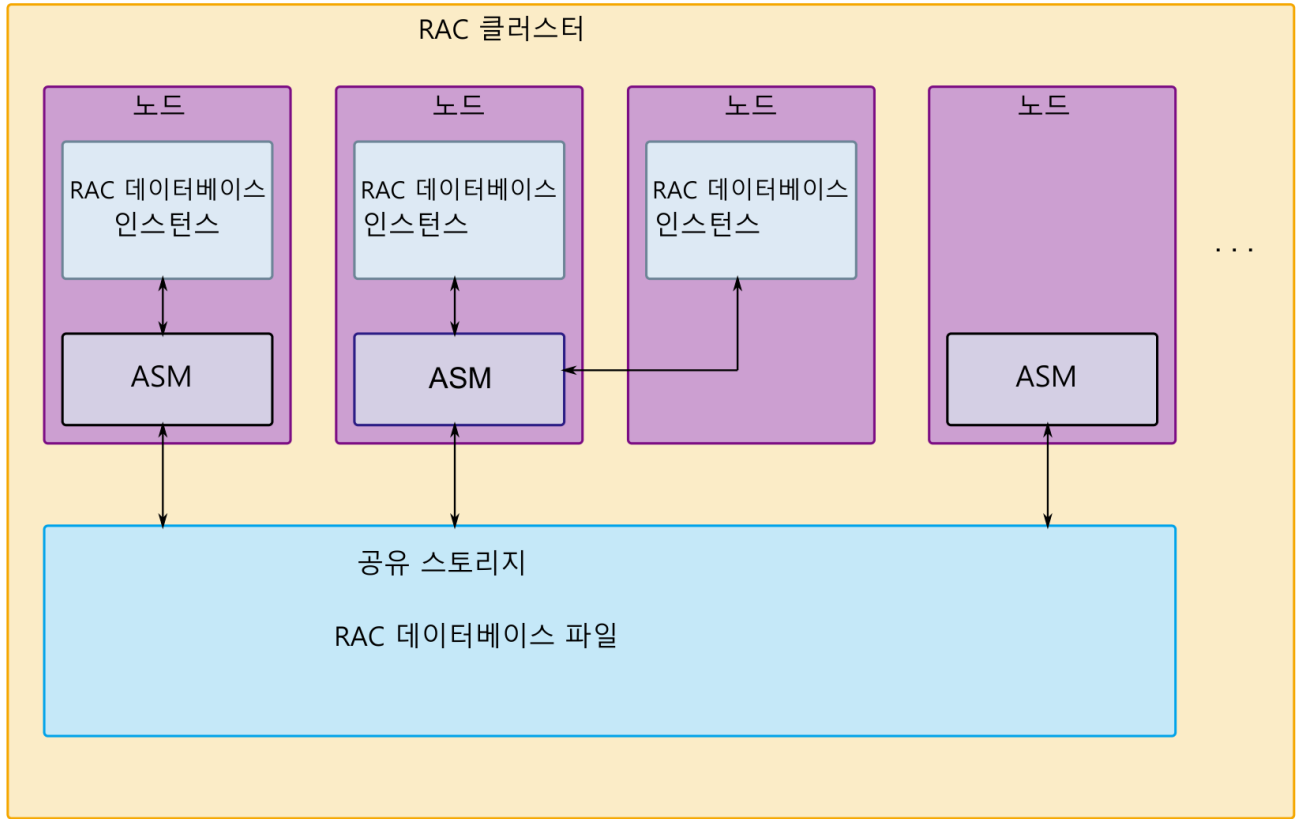
노드 하나의 소프트웨어 스택

아래 층이 서야 위 층이 선다



RAC 데이터베이스와 Flex ASM

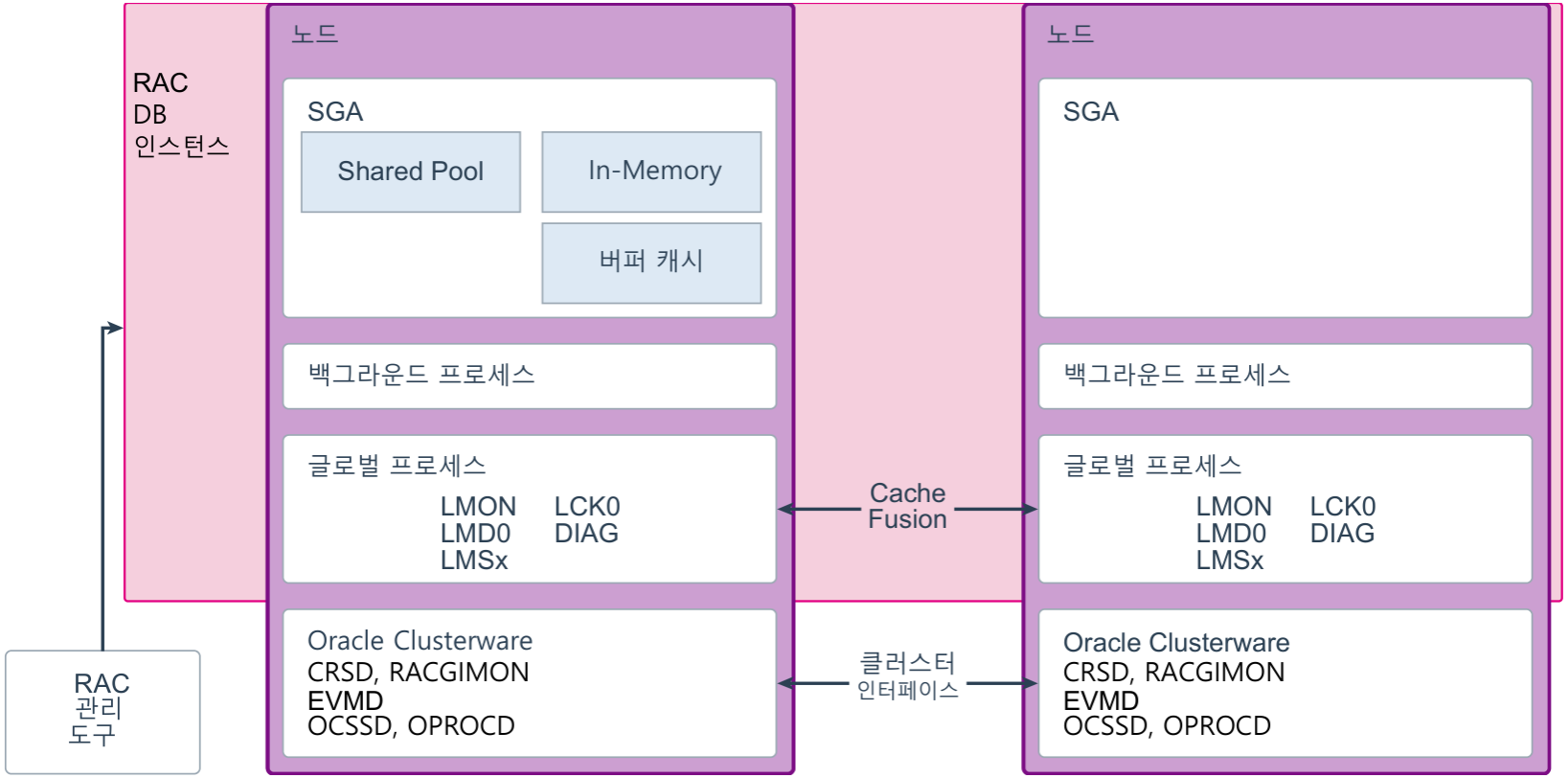
19c 의 ASM 구성은 Flex ASM 하나



- 인스턴스 여럿이 공유 스토리지의 파일 한 벌을 연다
- ASM 인스턴스 배치는 카디널리티가 정한다
- 실습 환경 — ora.asmgroup 3번째 OFFLINE 은 정상

RAC 인스턴스 안에 더해진 것

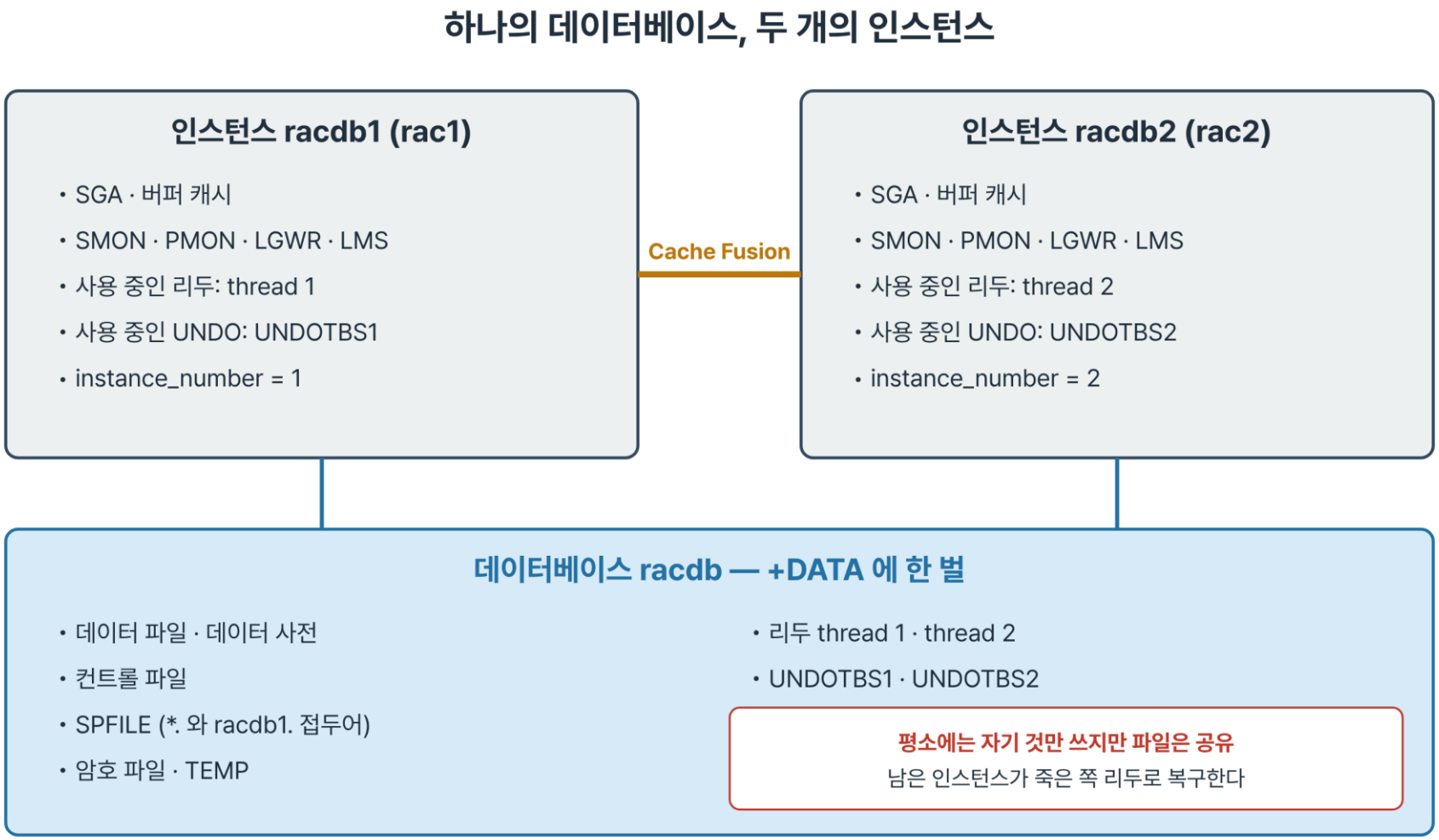
Cache Fusion 과 글로벌 프로세스



- SGA·백그라운드 프로세스는 단일 인스턴스와 같다
- 글로벌 프로세스 — LMON, LMD, LMS, LCK0, DIAG
- Cache Fusion 과 클러스터 인터페이스는 인터커넥트로

공유되는 것과 인스턴스마다 따로인 것

리두 스레드와 UNDO 도 파일은 공유 스토리지에



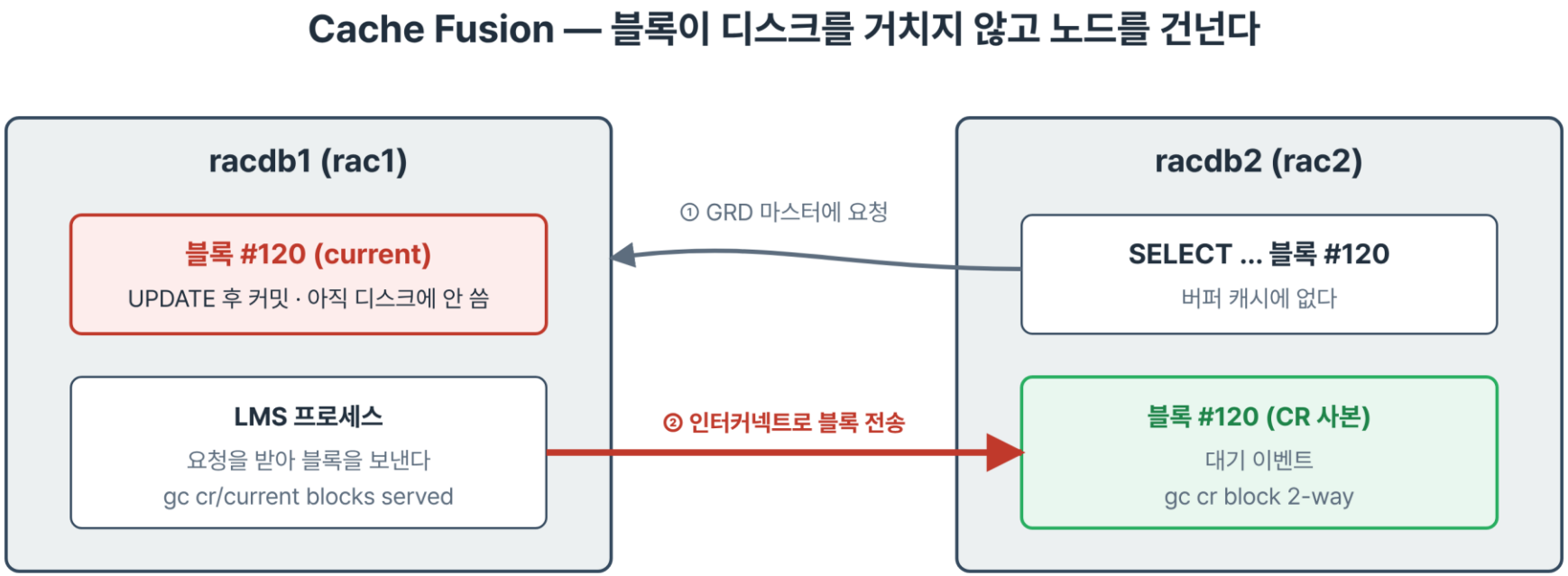
V$ 와 GV$

다른 노드의 문제는 V$ 에 나타나지 않는다

- V$ — 접속한 인스턴스만
- GV$ — 모든 인스턴스, INST_ID 열로 출처 구분
- GV$ 는 병렬 슬레이브로 모은다 — 응답 없는 인스턴스가 있으면 멈출 수 있다
- INST_ID 는 출처, INSTANCE_NUMBER 는 파라미터가 정한 번호
- 확인 — SELECT inst_id, instance_name, thread# FROM gv$instance

Cache Fusion

블록이 디스크를 거치지 않고 노드를 건넌다

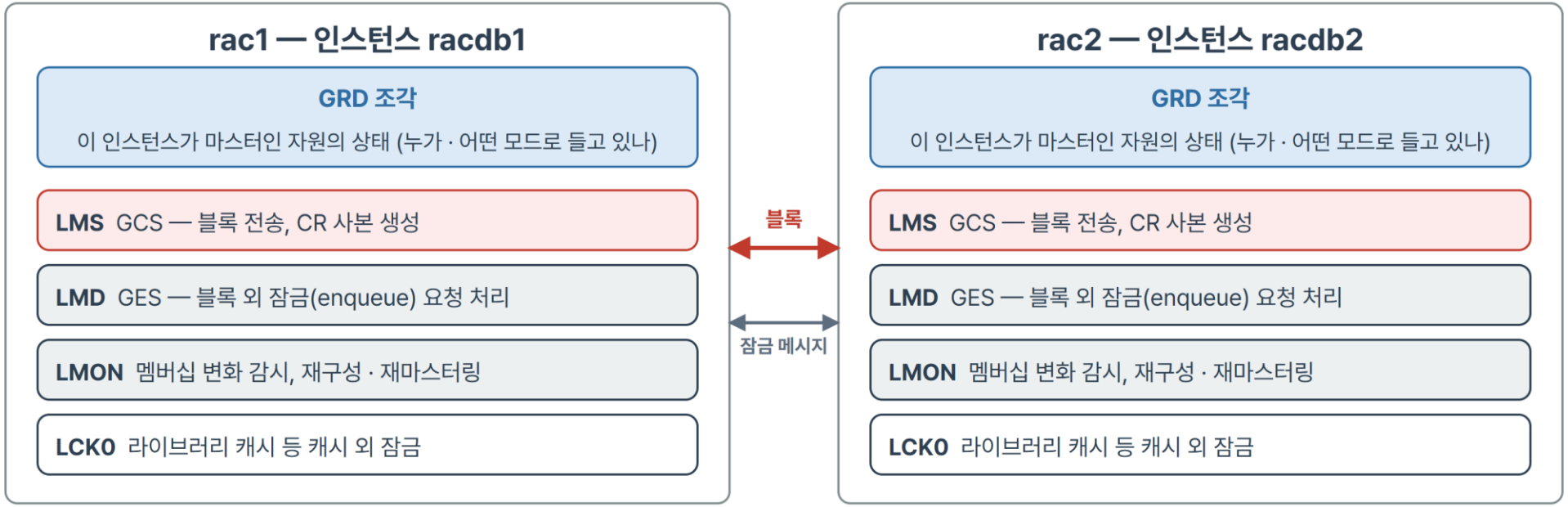


- 2-way: 요청 노드 → 마스터 = 보유 노드 → 요청 노드 (메시지 2번)
- 3-way: 요청 노드 → 마스터 → 보유 노드 → 요청 노드 (2노드에서는 드물다)
- current 블록은 갱신용, CR 블록은 읽기 일관성용 사본
- 디스크 I/O 대신 네트워크 왕복 — 인터커넥트 지연이 곧 DB 지연

글로벌 프로세스

GRD 는 한곳에 없다 — 인스턴스들이 나눠 맡는다

글로벌 프로세스 — GRD 를 나눠 맡는 일꾼들



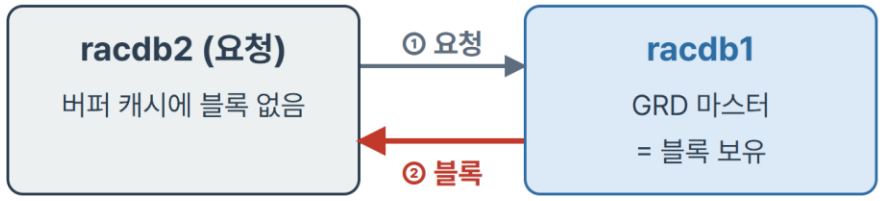
· GRD(Global Resource Directory)는 한 곳에 모여 있지 않다. 자원마다 마스터 인스턴스가 정해지고 각자 자기 몫만 관리한다
· GCS(Global Cache Service)는 데이터 블록을, GES(Global Enqueue Service)는 그 밖의 잠금을 맡는다
· 인스턴스가 빠지면 LMON 이 재구성을 이끌고, 빠진 인스턴스가 마스터였던 자원을 남은 인스턴스로 다시 나눈다
ps -ef | grep -E "lms|lmd|lmon|lck" 로 각 노드에서 확인할 수 있다

2-way 와 3-way

2노드에서는 거의 모두 2-way

Cache Fusion — 2-way 와 3-way

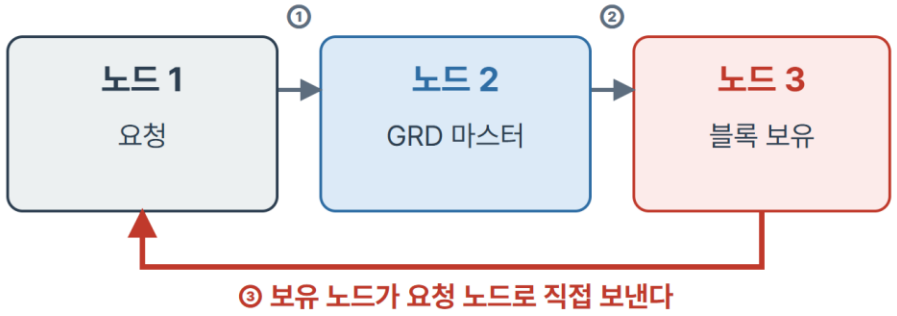
2-way — 마스터가 블록도 들고 있을 때



- 메시지 2번 — 2노드 클러스터는 거의 모두 이 경우
- 대기 이벤트: gc cr block 2-way, gc current block 2-way
- 블록을 보내는 것은 보유 노드의 LMS
- 디스크 I/O 대신 인터커넥트 왕복 한 번

마스터는 블록마다 정해지고 인스턴스들이 나눠 맡는다

3-way — 마스터와 보유 노드가 다를 때



- 메시지 3번 — 3노드 이상에서 생긴다
- 대기 이벤트: gc cr block 3-way, gc current block 3-way
- 블록은 마스터를 거치지 않는다

노드가 늘수록 3-way 비율이 오른다

current 블록은 갱신용 최신본(한 시점에 한 인스턴스만 배타 모드), CR 블록은 읽기 일관성용으로 보유 노드가 만들어 보낸 사본

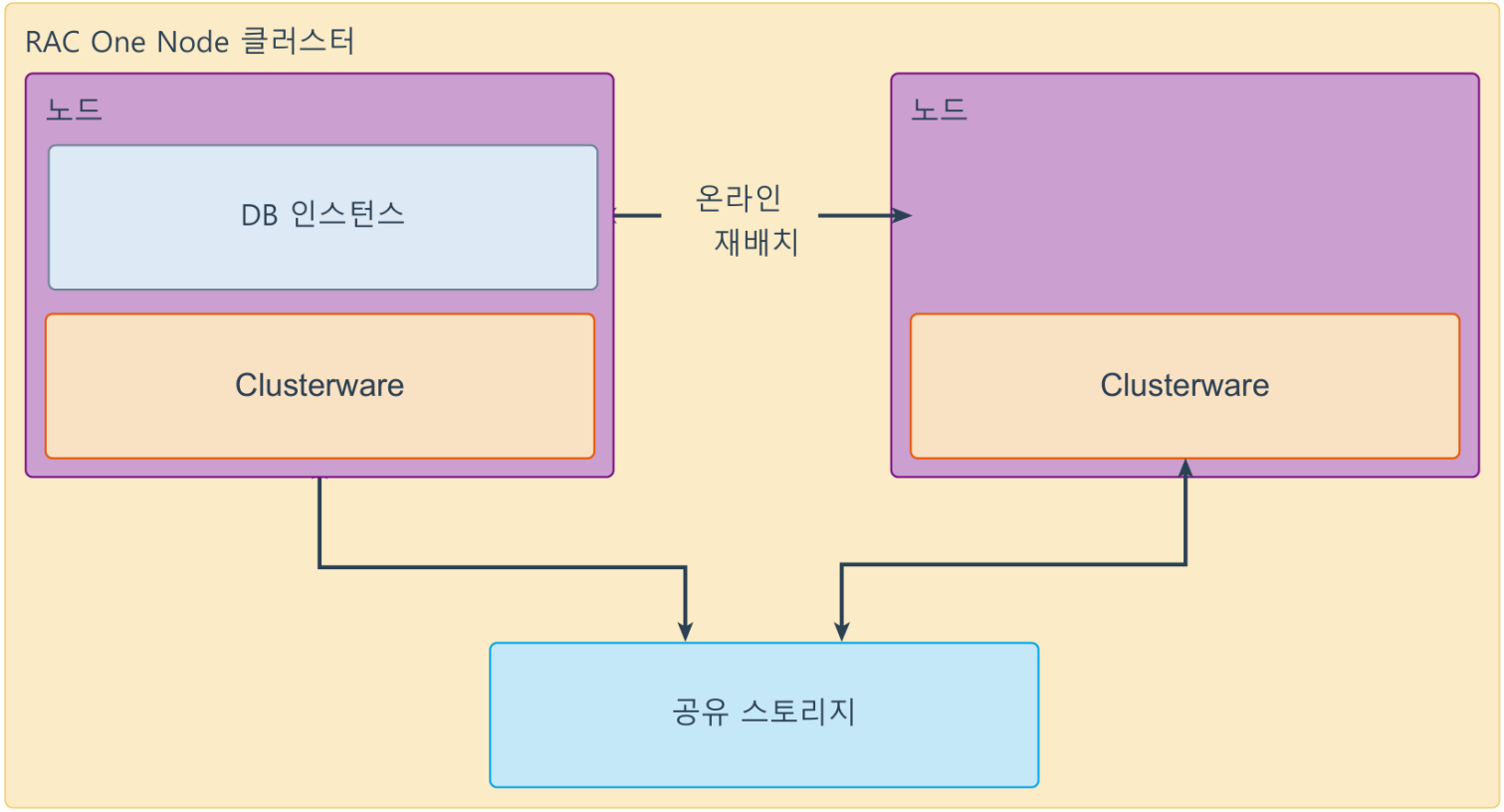
current 블록과 CR 블록

통계로 블록의 이동 방향을 읽는다

- current — 갱신용 최신본, 한 시점에 한 인스턴스만 배타 모드
- CR — 읽기 일관성용으로 보유 노드가 만들어 보낸 사본
- 대기 — gc cr block 2-way, gc current block 2-way
- 통계 — gc cr/current blocks received · served (누적값)
- 한쪽 received 증가 = 다른 쪽 served 증가 → 그 방향으로 이동
- gc 대기가 길면 — 인터커넥트 지연, hot block, 보내는 쪽 LMS CPU

RAC One Node

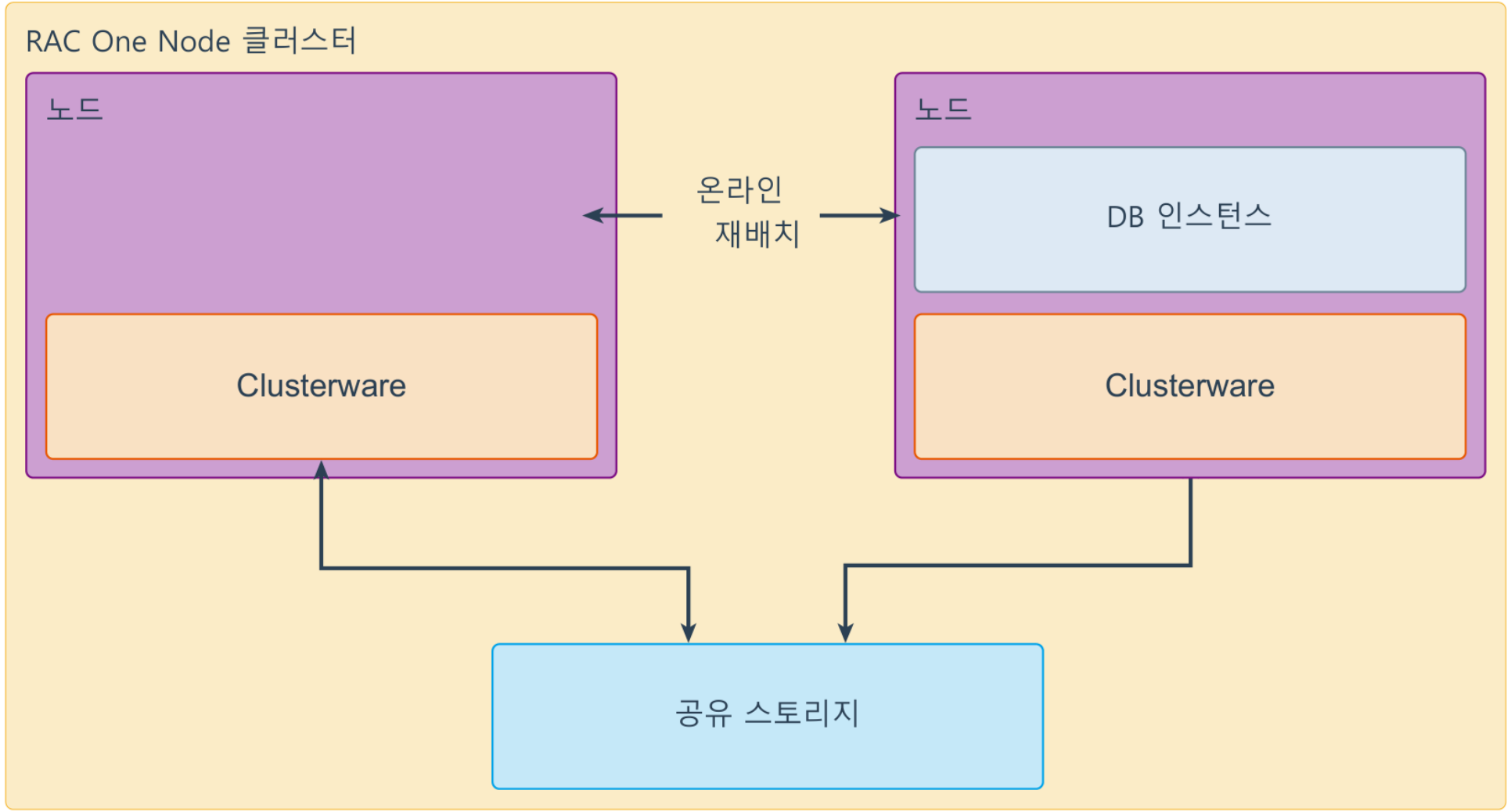
평소엔 인스턴스 하나, 필요하면 옮긴다



- 온라인 재배치 — 애플리케이션 연속성을 유지한 채 다른 노드로
- 이점 — 페일오버, 온라인 롤링 패치, OS·Clusterware 롤링 업그레이드
- 계획되지 않은 정지 때는 잠깐 인스턴스가 하나도 없는 순간

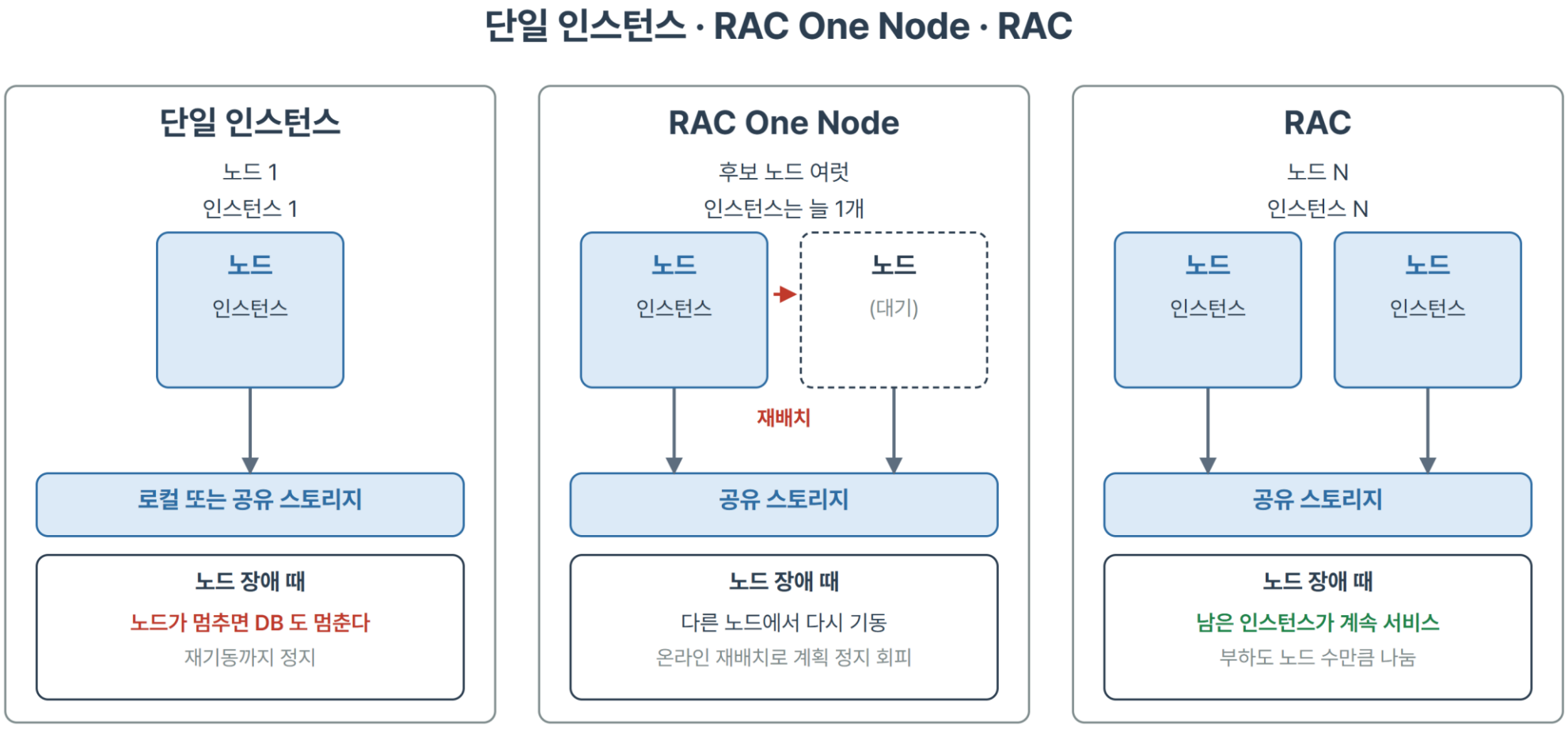
재배치가 끝난 뒤

인스턴스는 오른쪽 노드에서 돈다



단일 인스턴스 · RAC One Node · RAC

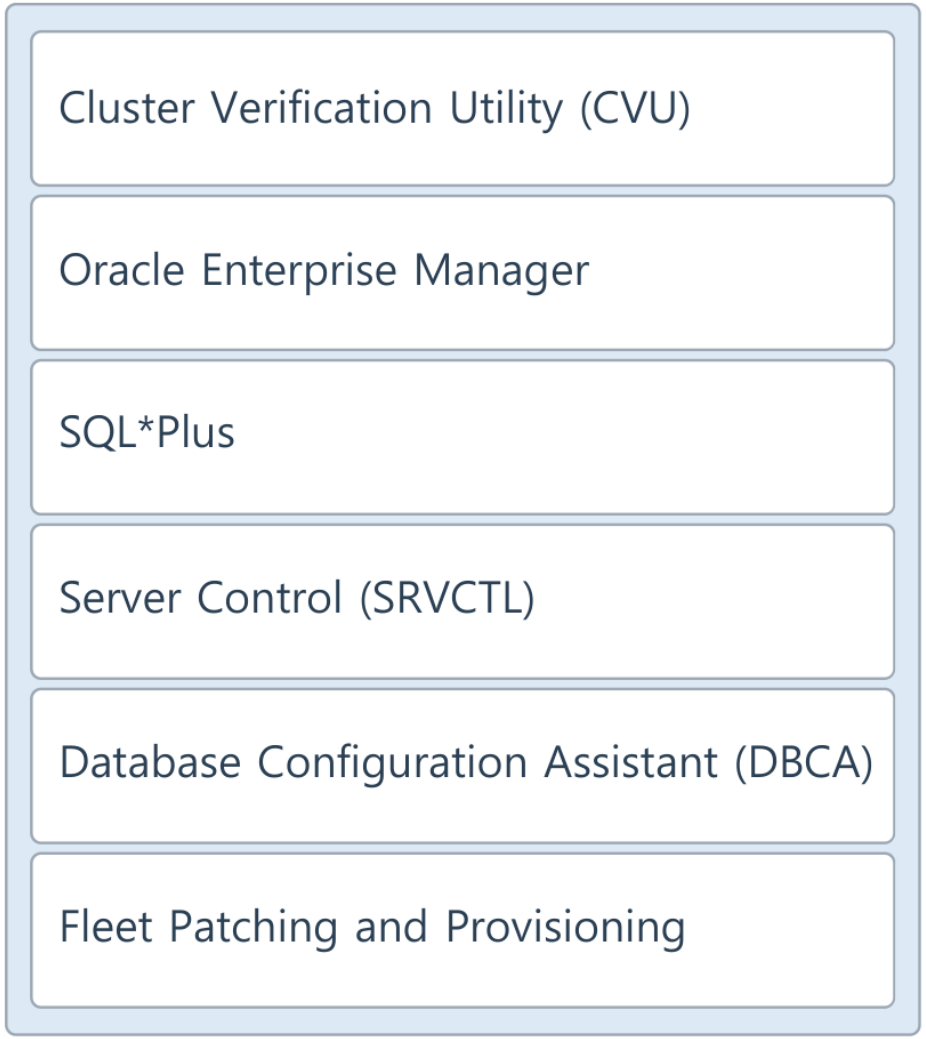
동시에 여는 인스턴스 수만 다르다



RAC One Node 도 Clusterware · ASM · 공유 스토리지 · 스레드와 UNDO 분리를 RAC 와 똑같이 쓴다
다른 것은 동시에 여는 인스턴스 수 하나다

RAC 관리 도구

클러스터 DB 를 시스템 하나처럼 관리한다



- CVU — 설치 전후, 노드 추가·삭제 뒤 검증
- Enterprise Manager — GUI, RAC One Node 포함
- SQL*Plus — 현재 인스턴스에 작용
- SRVCTL — DB·인스턴스·서비스 기동·정지·재배치
- DBCA — RAC·RAC One Node·단일 인스턴스 생성
- FPP — 패치·업그레이드·프로비저닝

라이선스와 옵션 경계

기능이 있어도 쓸 수 있는지는 계약이 정한다

- RAC — Enterprise Edition 의 별도 유료 옵션
- 2노드면 두 노드 프로세서 모두에 EE + RAC
- RAC One Node — 별도 옵션
- Clusterware · ASM — DB 라이선스가 있는 서버에서 추가 비용 없음
- 확인 — V$OPTION, DBA_FEATURE_USAGE_STATISTICS

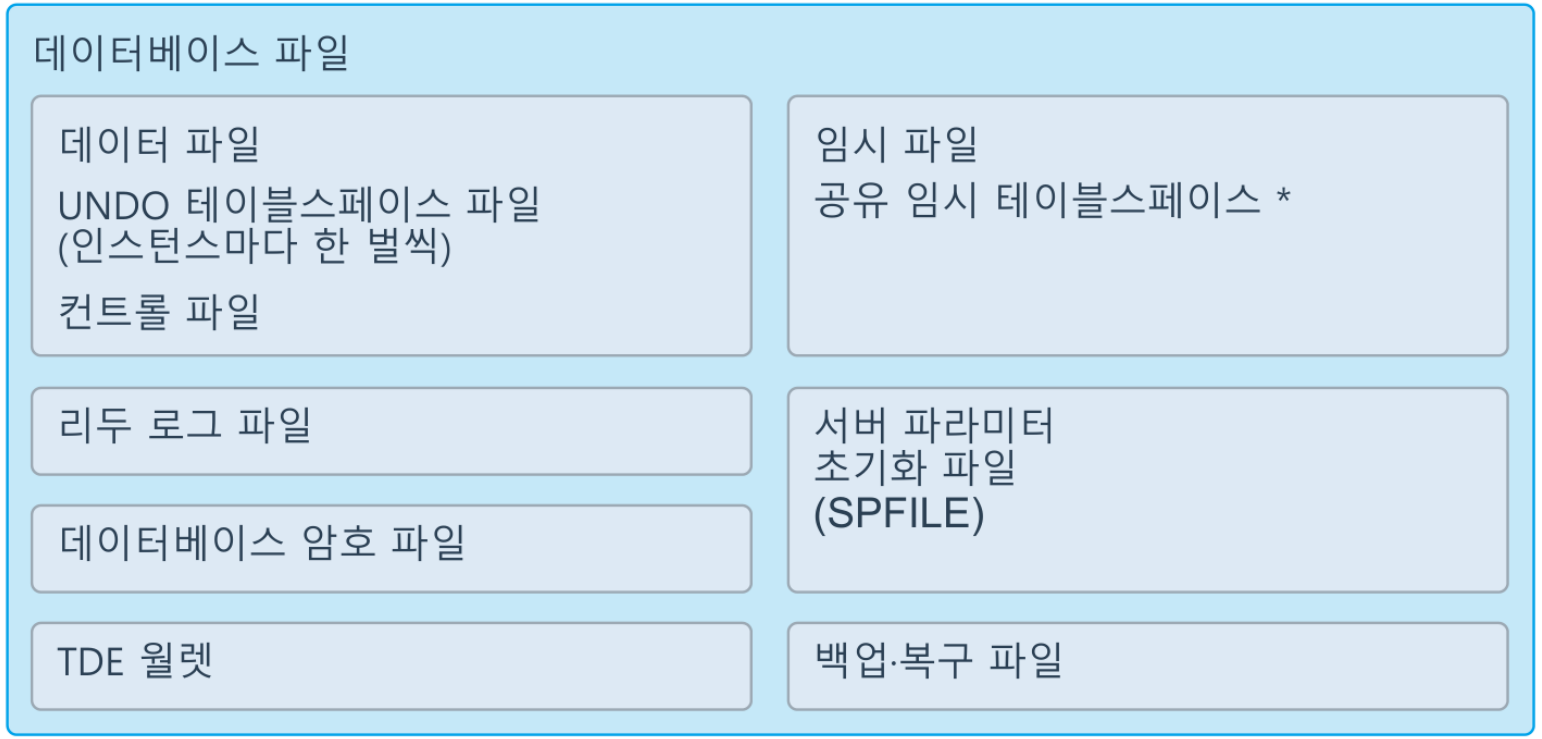
2

RAC 데이터베이스 파일

- 어떤 파일을 공유하고 무엇을 인스턴스별로 두는가
- SPFILE 과 SID
- TDE, UNDO, 리두 스레드, 백업
- PDB 와 In-Memory

RAC 데이터베이스를 이루는 파일

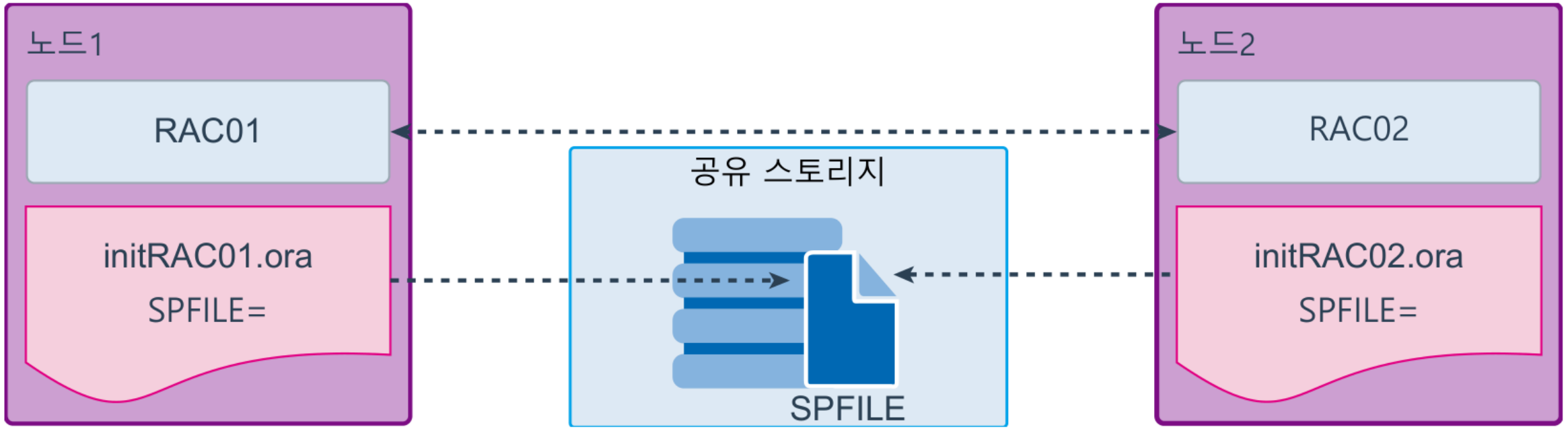
모든 인스턴스가 모든 파일에 접근한다



- UNDO — 인스턴스마다 하나, 모두 공유 스토리지에
- 리두 — 인스턴스마다 그룹 둘 이상 = 스레드
- 컨트롤 파일 둘 이상, 사본은 다른 디스크그룹
- 공통 SPFILE 하나를 공유 스토리지에

서버 파라미터 파일

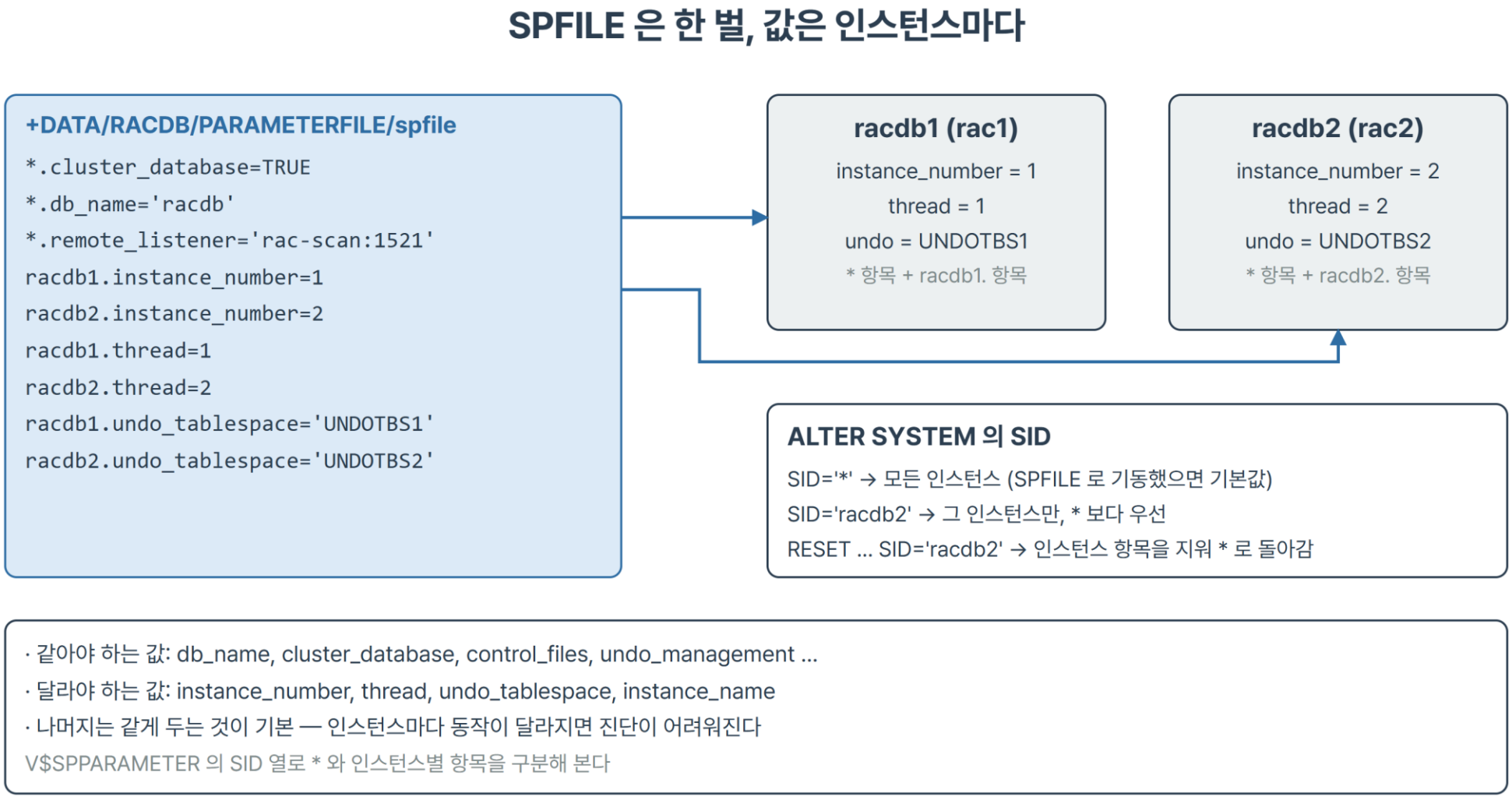
PFILE 은 공유 SPFILE 을 가리키기만 한다



- 모든 인스턴스가 같은 SPFILE
- 바이너리 — 편집하지 않고 ALTER SYSTEM 으로
- SPFILE 이 없거나 STARTUP PFILE= 일 때만 PFILE

SPFILE 한 벌, 값은 인스턴스마다

* 항목과 sid. 항목



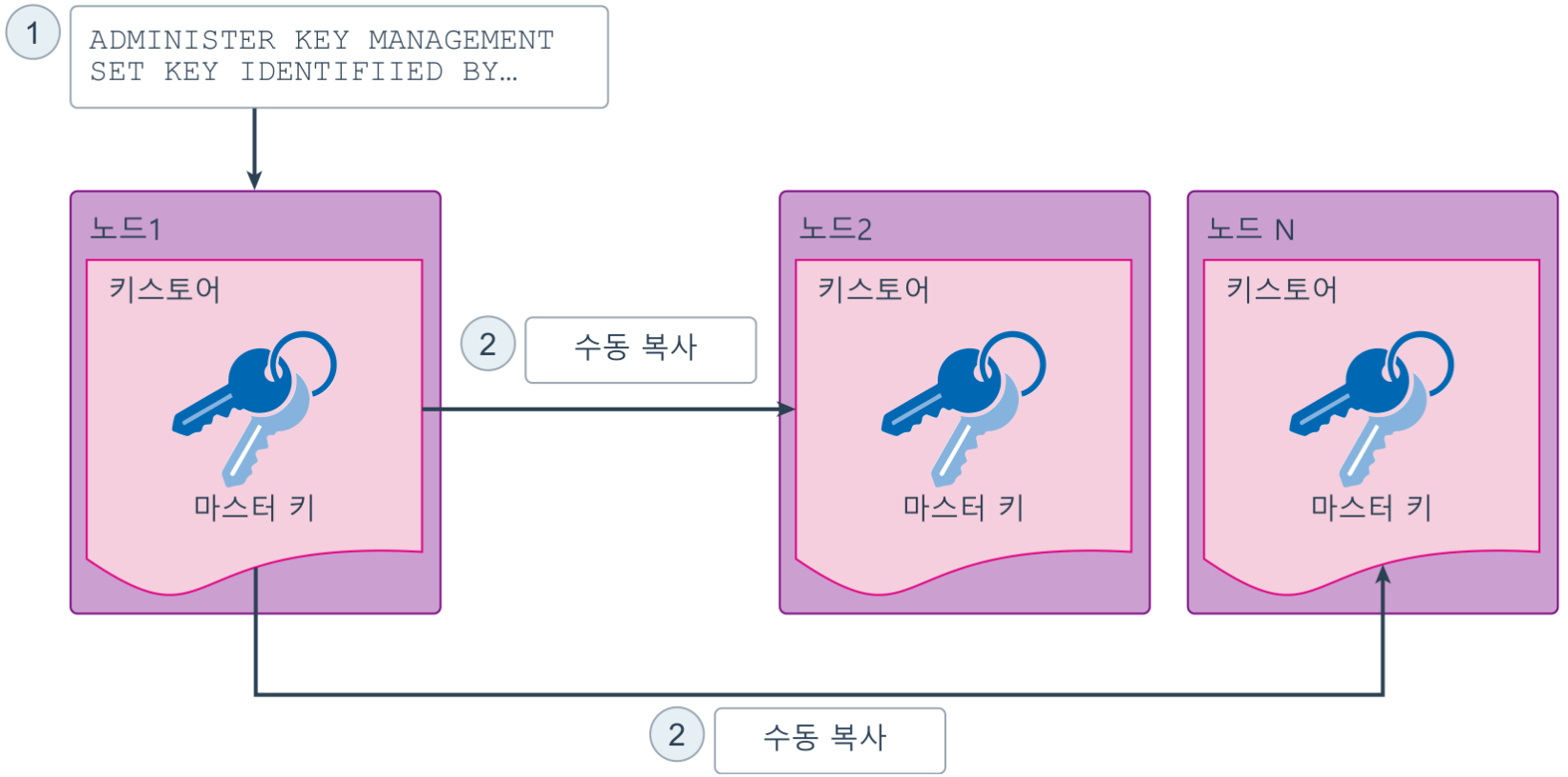
ALTER SYSTEM 의 SID 와 RAC 파라미터

SID 지정 값이 * 보다 우선

- SID='*' — 모든 인스턴스 (SPFILE 로 기동했으면 기본값)
- SID='racdb2' — 그 인스턴스만
- RESET … SID='racdb2' — 인스턴스 항목을 지워 * 로
- CLUSTER_DATABASE=TRUE — 클러스터 모드
- CLUSTER_INTERCONNECTS — 보통 설정하지 않는다 (oifcfg 가 우선)

TDE 키스토어

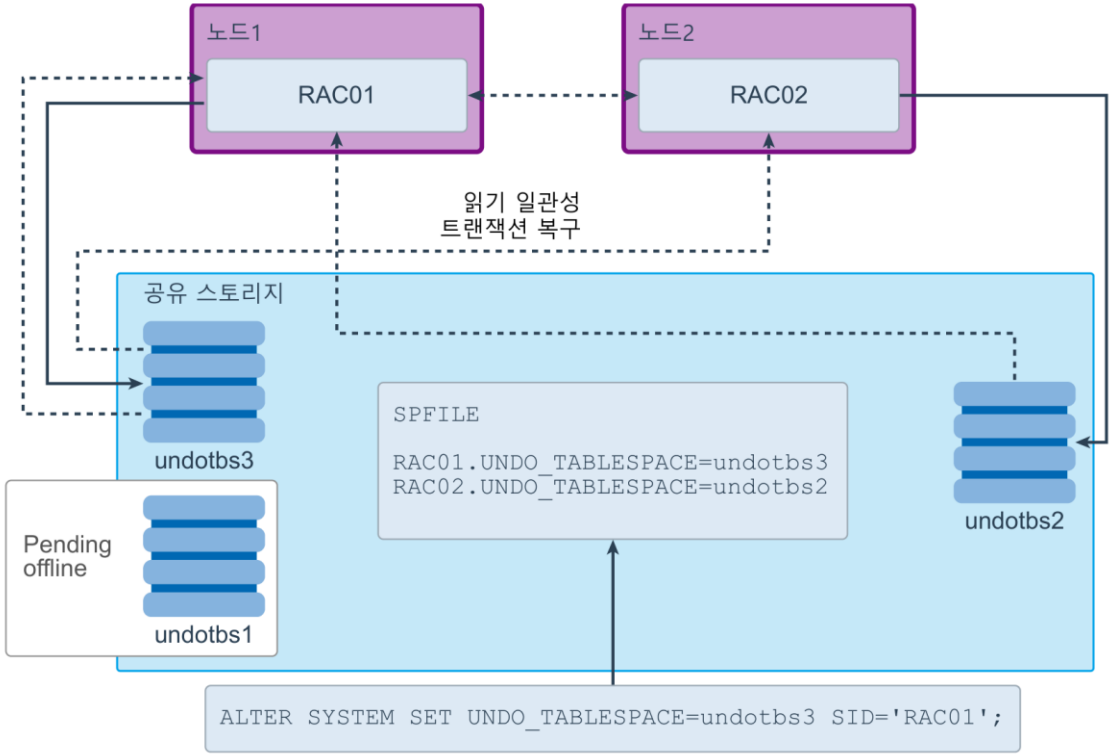
로컬 사본이면 키를 바꿀 때마다 다른 노드로 복사



- 공유 스토리지에 월렛 하나 — 추가 구성 없음
- 로컬 사본 — 초기 구성 뒤, 키 교체 때마다 수동 복사
- 19c — WALLET_ROOT 가 sqlnet.ora 설정보다 우선

UNDO 테이블스페이스

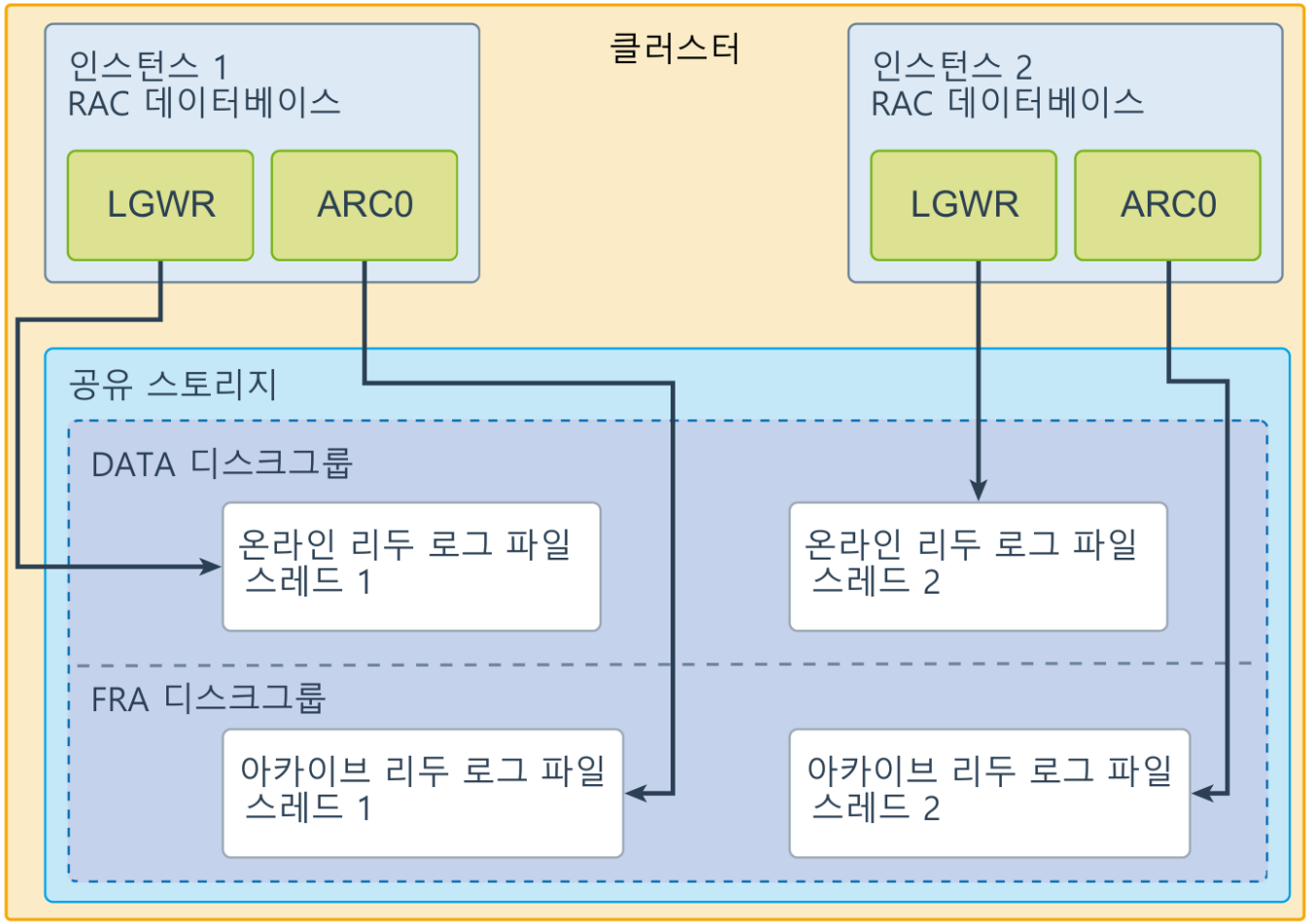
평소엔 자기 것만, 복구 때는 남의 것도



- 쓰기 — 배정받은 인스턴스만
- 읽기 — 읽기 일관성을 위해 모든 인스턴스가
- 트랜잭션 복구 — 아무도 쓰지 않는 UNDO 면 어느 인스턴스든
- 전환 — 이전 UNDO 는 Pending offline 으로 남는다

리두와 리두 스레드

인스턴스마다 자기 스레드에 쓰고 자기 스레드를 아카이브

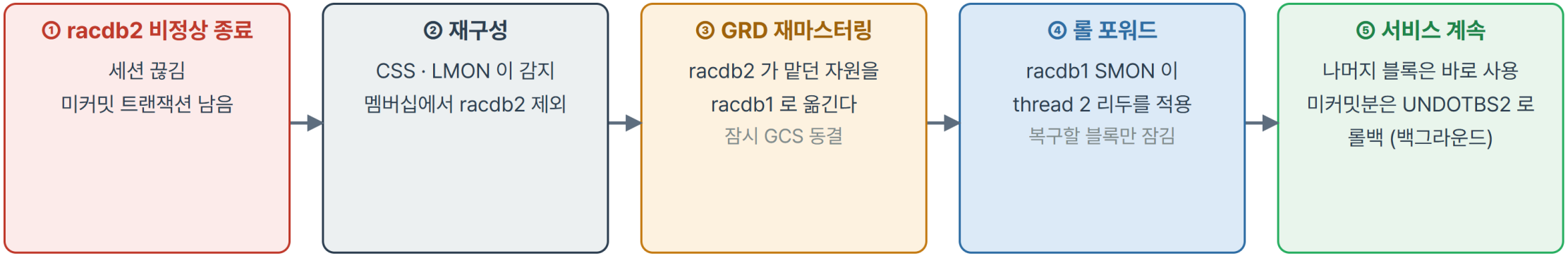


- 스레드마다 그룹 둘 이상, 그룹마다 멤버 둘 이상
- 모든 스레드는 공유 스토리지 — 복구 때 남의 스레드도 읽는다
- 실습 환경 — racdb1 thread 1, racdb2 thread 2, 아카이브는 +FRA

인스턴스 복구

racdb2 가 죽은 뒤 racdb1 이 하는 일

인스턴스 복구 — racdb2 가 죽은 뒤 racdb1 이 하는 일



복구가 가능한 이유

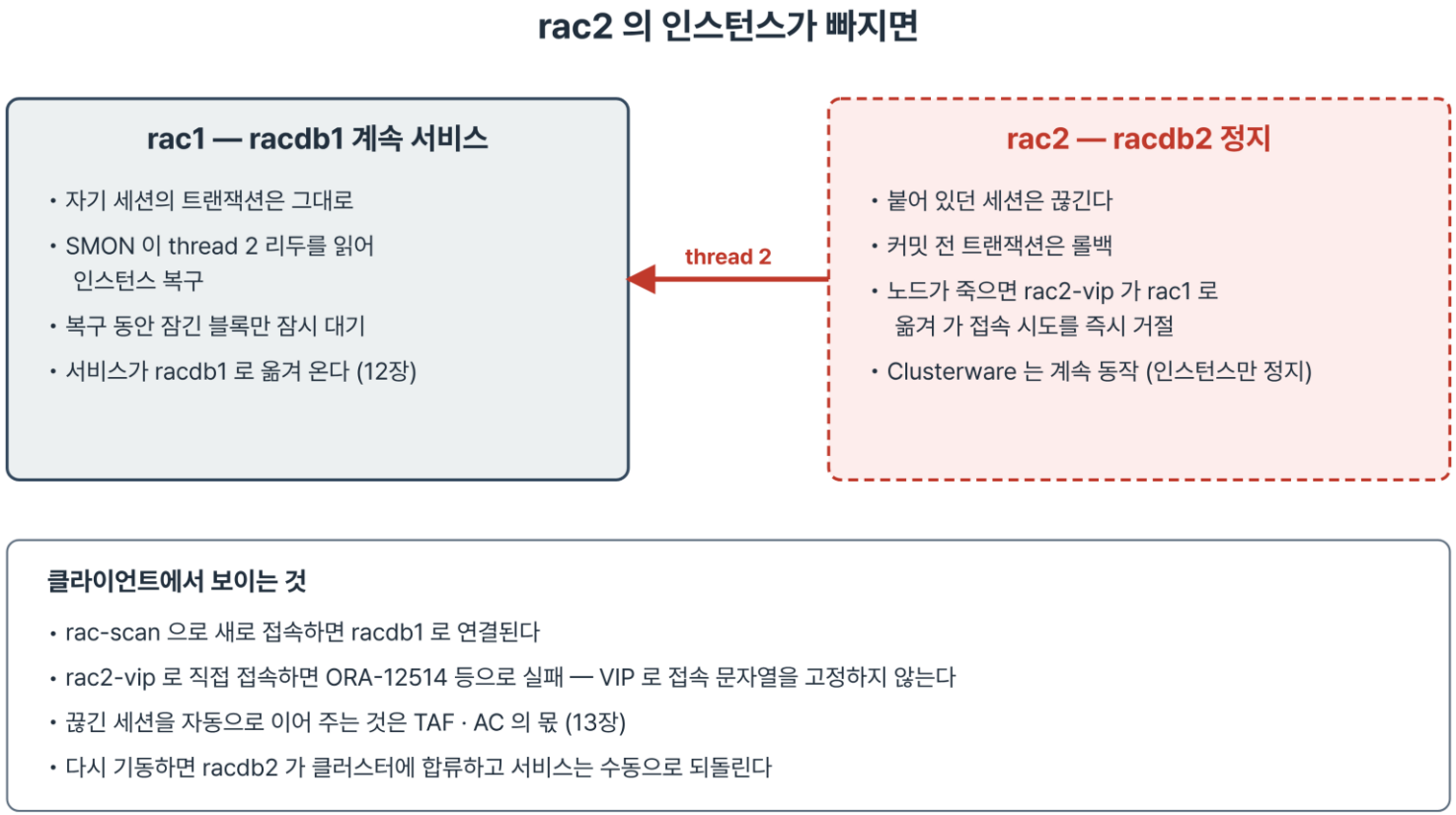
· 리두 스레드와 UNDO 는 인스턴스별로 쓰지만
파일은 +DATA 에 있어 모든 인스턴스가 읽는다
· 살아 있는 인스턴스가 죽은 쪽 스레드를 읽는다
· 노드 재기동을 기다리지 않는다

관찰 지점

· racdb1 alert.log: Reconfiguration started,
Instance recovery: looking for dead threads,
Completed instance recovery
· GV$INSTANCE 에서 racdb2 행이 사라진다
· srvctl start instance 로 올리면 다시 합류

인스턴스 하나가 빠지면

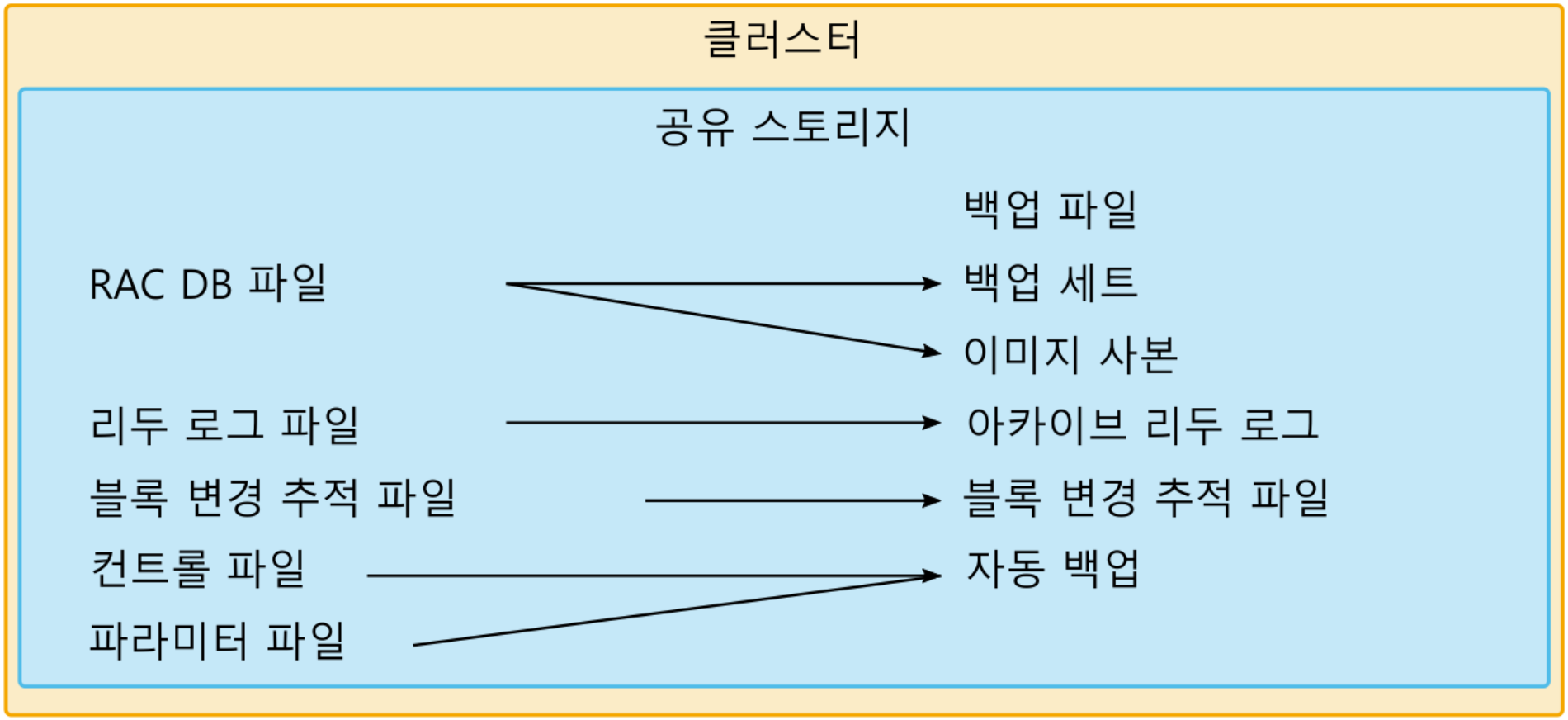
계속되는 것과 끊기는 것



- rac2-vip 로 직접 접속 → ORA-12514 (리스너는 있고 서비스가 없다)
- 그 주소에 리스너가 없으면 ORA-12541, 노드가 꺼졌으면 ORA-12170

RAC 의 백업 파일

어느 인스턴스든 복원·복구를 할 수 있어야 한다



- 성격은 단일 인스턴스와 같다
- 백업과 아카이브는 공유 스토리지에 — 언제든 모든 인스턴스가 읽게

PDB 와 Database In-Memory

단일 인스턴스 기능의 RAC 구현

- CDB 가 곧 RAC 데이터베이스 — CDB 인스턴스가 노드마다
- PDB 는 모든 인스턴스나 일부에서 연다
- 꽂은 PDB 는 기본적으로 어디서도 자동으로 열리지 않는다
- 첫 동적 서비스를 배정하면 그 서비스가 도는 인스턴스에서 열린다
- In-Memory — 노드마다 IM 열 저장소, 객체는 모든 노드에 나뉘어 적재

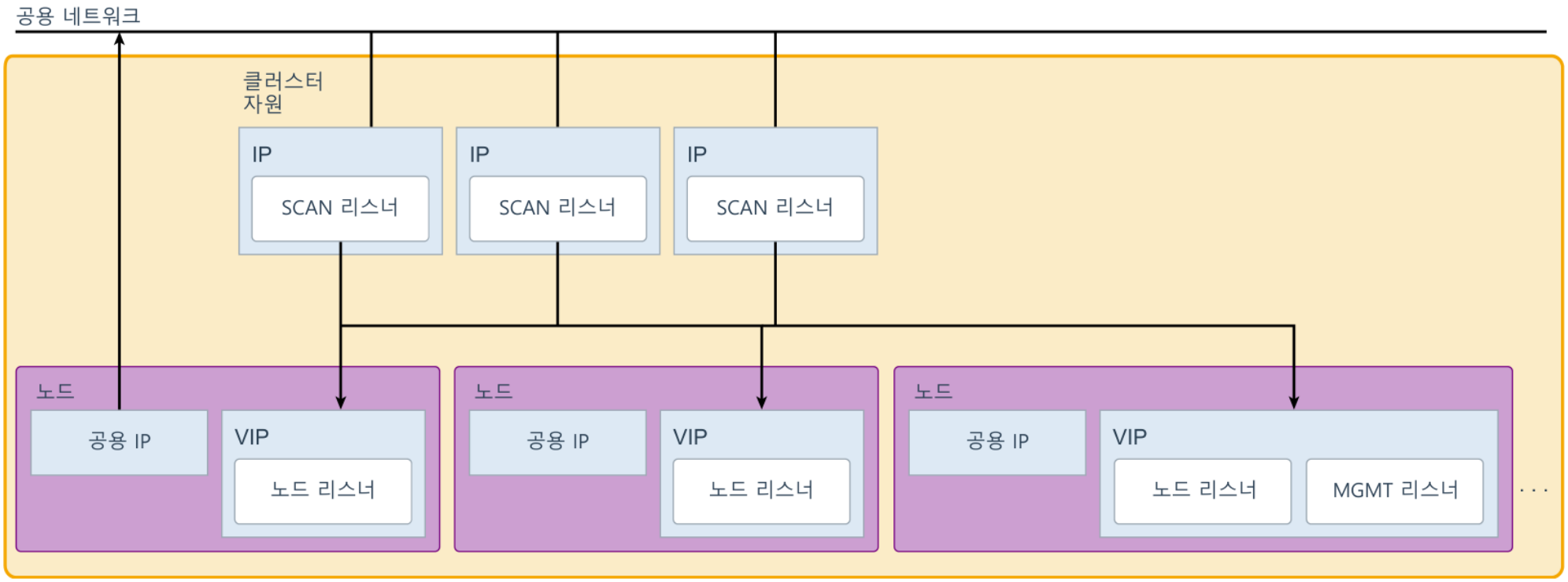
3

클러스터와 네트워크

- 리스너와 SCAN
- 네 종류의 주소, VIP
- 클러스터 노드와 서버 네트워크
- 클러스터 구성 방식과 스토리지

RAC 의 리스너

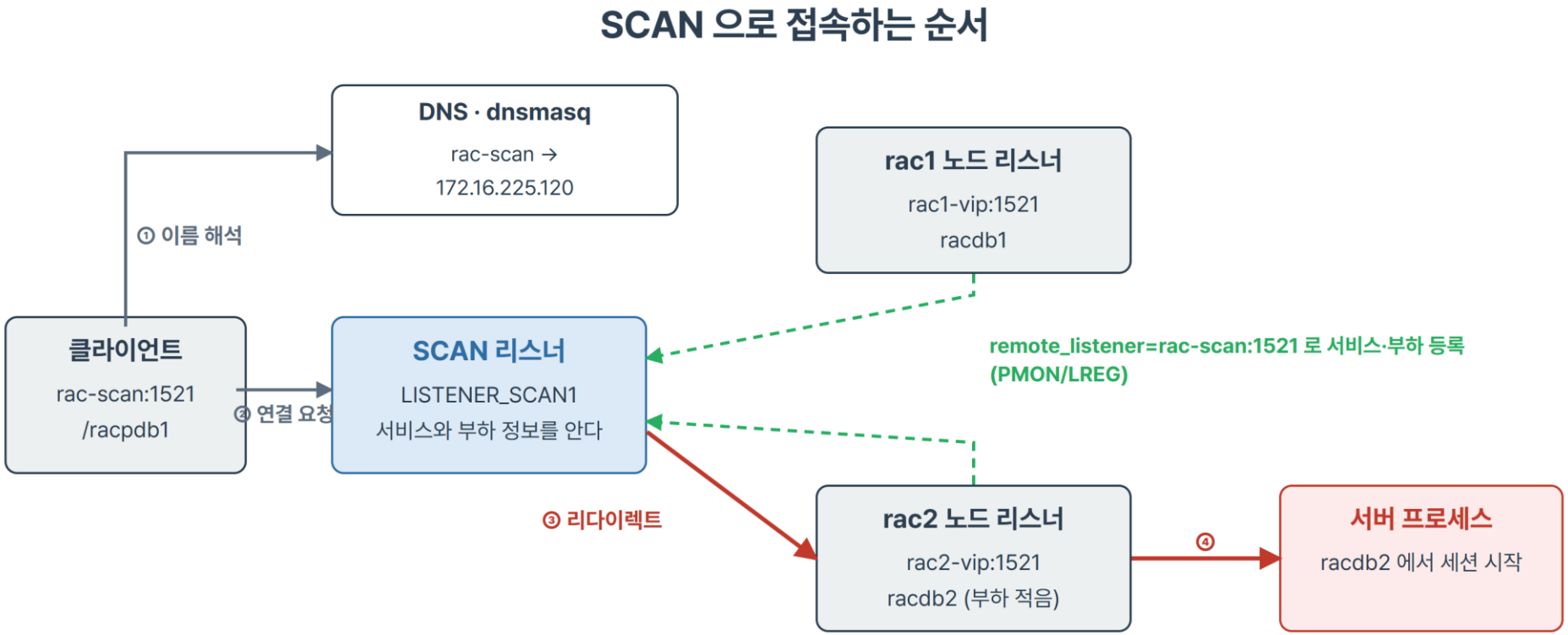
SCAN 리스너 · 노드 리스너 · MGMT 리스너



- SCAN — 클러스터 이름 하나, DNS 가 IP 3개 이상으로 라운드 로빈
- SCAN IP 마다 SCAN 리스너, 노드가 죽으면 다른 노드로
- 노드 리스너 — VIP 에서 듣는다
- MGMT 리스너 — GIMR 용, 19c 에서 GIMR 은 선택

SCAN 으로 접속하는 순서

SCAN 리스너는 세션을 받지 않고 넘긴다



· 클라이언트는 노드 이름을 몰라도 된다 — 노드를 넣고 빼도 접속 문자열은 그대로
· SCAN 리스너는 세션을 받지 않고 가장 한가한 인스턴스의 노드 리스너로 넘긴다
· 운영 환경은 DNS 에 SCAN IP 3개 → SCAN 리스너 3개. 실습 환경은 1개라 LISTENER_SCAN1 뿐

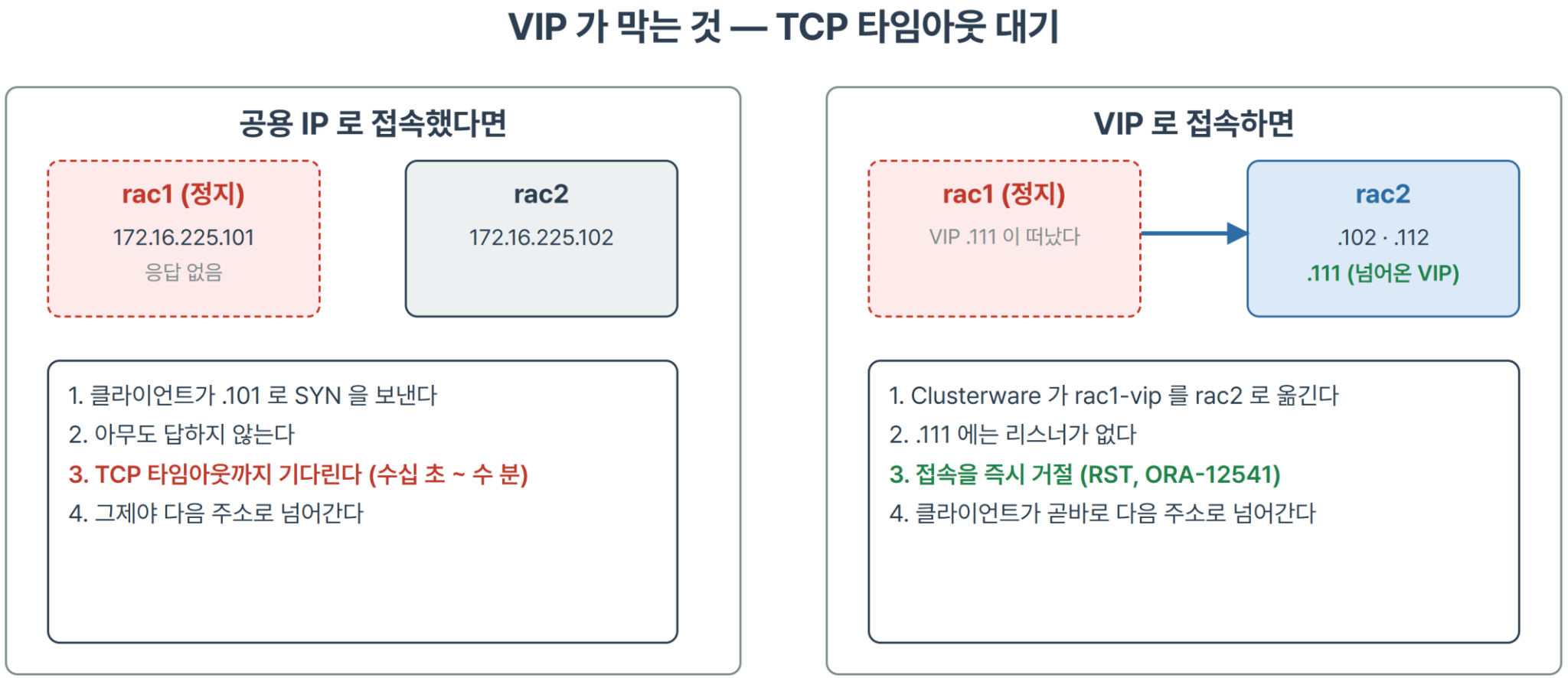
네 종류의 주소

어떤 주소로 접속하느냐가 장애 때 동작을 가른다

- 공용 IP — rac1 .101, rac2 .102 (ens33) · 관리 접속, SSH
- VIP — rac1-vip .111, rac2-vip .112 · 노드 리스너, 노드 장애 때 이동
- SCAN — rac-scan .120:1521 · 클라이언트 접속 이름
- 사설 IP — rac1-priv 172.16.119.101, rac2-priv .102 (ens160) · 하트비트, Cache Fusion, ASM
- 공용·사설은 OS 가, VIP·SCAN·HAIP 는 Clusterware 가 올린다

VIP 가 막는 것

TCP 타임아웃 대기 대신 즉시 거절

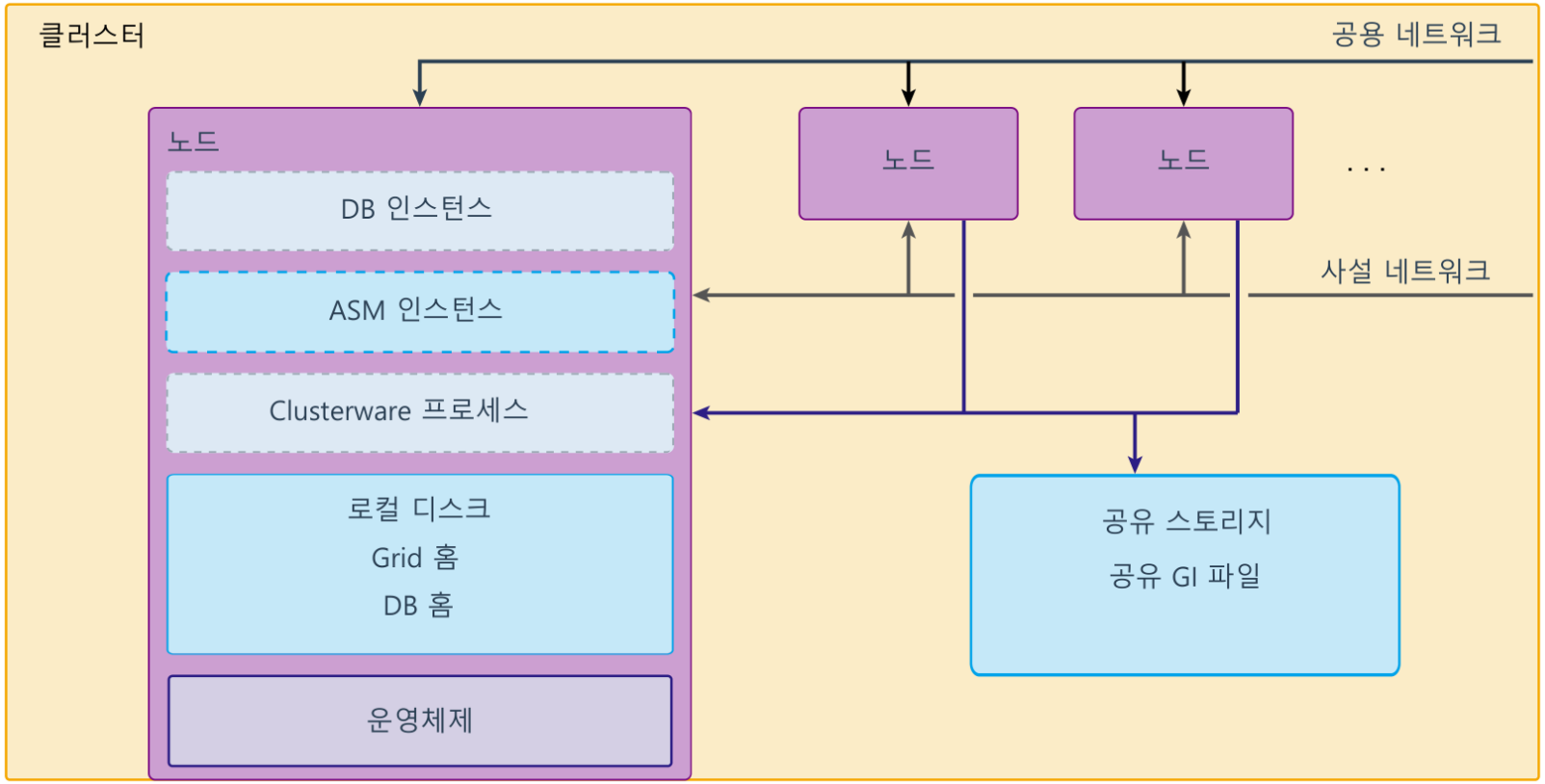


VIP 는 노드가 죽을 때만 옮겨 간다 — 노드가 살아 있고 인스턴스만 내려갔다면 VIP 는 그대로이고 접속은 ORA-12514 로 끝난다

노드가 클러스터에 돌아오면 노드 VIP 는 제 노드로 돌아간다 — SCAN VIP 는 옮겨 간 자리에 남는다

클러스터 노드

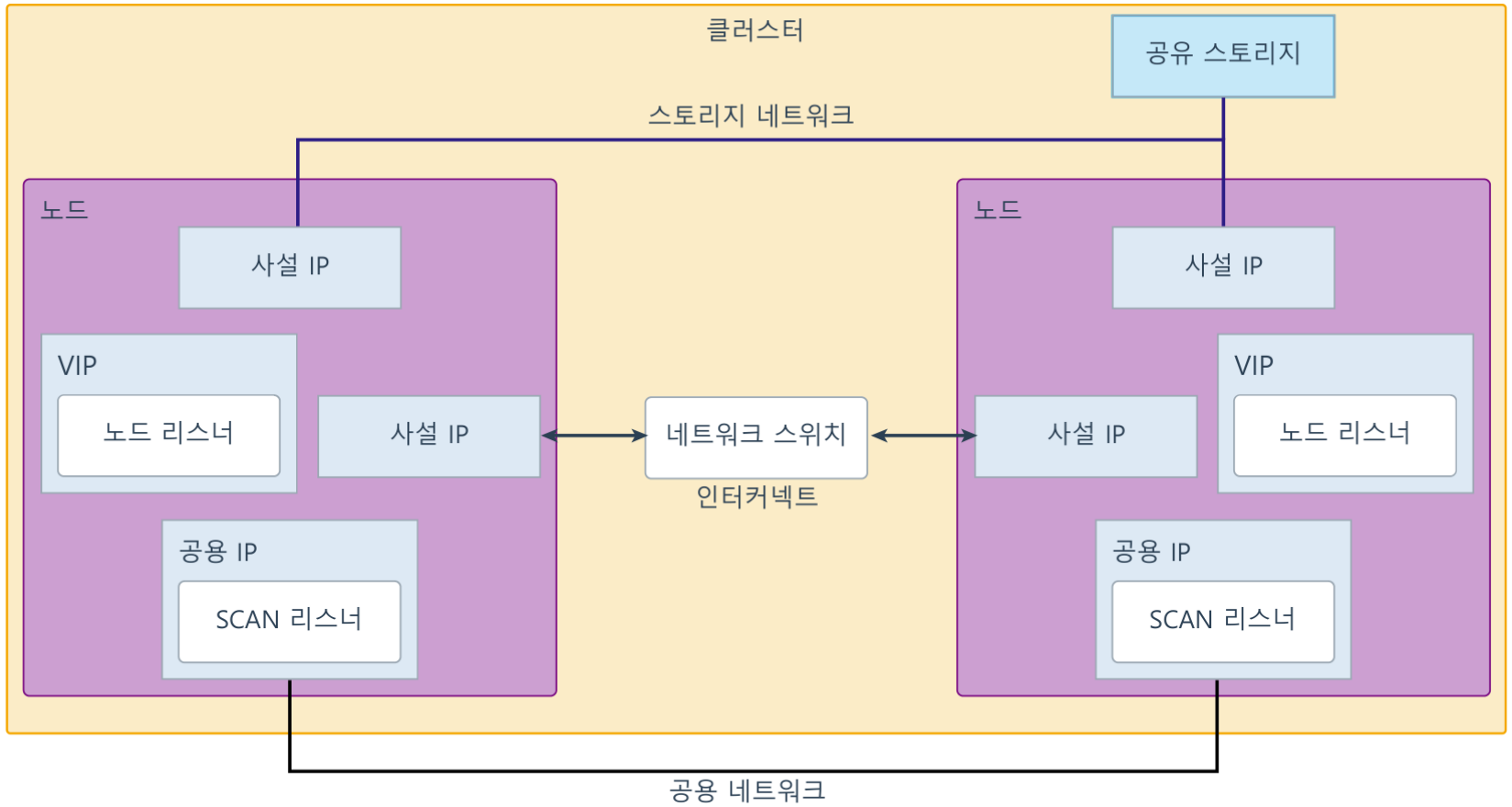
로컬 디스크에는 홈과 OLR, 공유 스토리지에는 GI 파일



- 노드마다 Clusterware 인스턴스 하나
- ADR 과 OLR 은 노드 로컬
- 홈은 로컬 설치 권장 — 롤링 업그레이드·재배치

서버 네트워크 구성

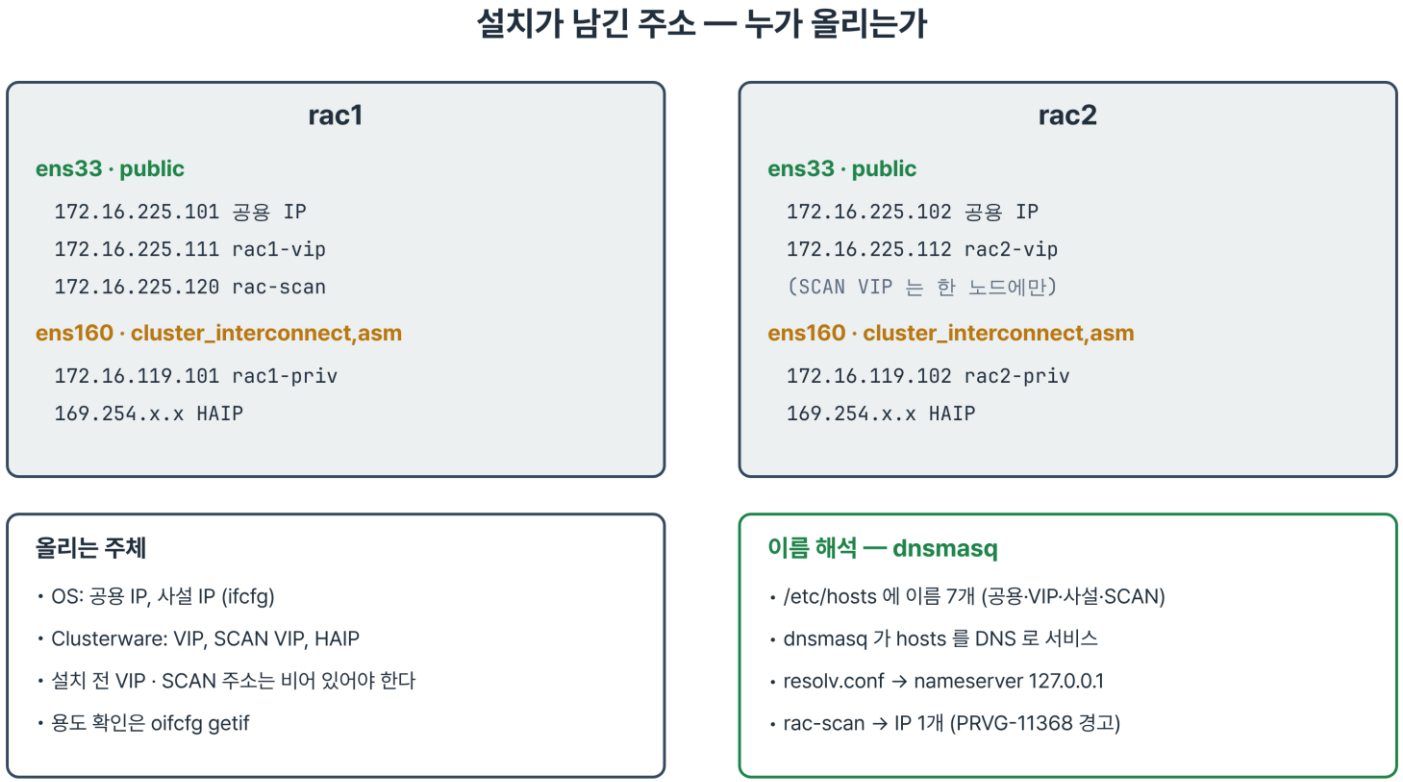
최소 NIC 2장 — 공용과 사설



- 스토리지 네트워크는 NAS 를 쓸 때만
- 본딩·트렁킹·티밍 지원

설치가 남긴 주소

누가 어느 주소를 올리는가



- 등록된 용도는 oifcfg getif — ens33 public, ens160 cluster_interconnect,asm
- iflist -p 의 PRIVATE 은 RFC 1918 대역이라는 뜻일 뿐
- HAIP 169.254.x.x — ASM·DB 의 인터커넥트 통신 주소

클러스터 구성 방식

독립형 · 확장 · 클러스터 도메인

독립형 클러스터

로컬 노드에 Grid Infrastructure 와 ASM 을 둔 단일 클러스터

확장 클러스터

노드가 지리적으로 떨어진 여러 곳에 퍼진 클러스터

클러스터 도메인

도메인 서비스 클러스터

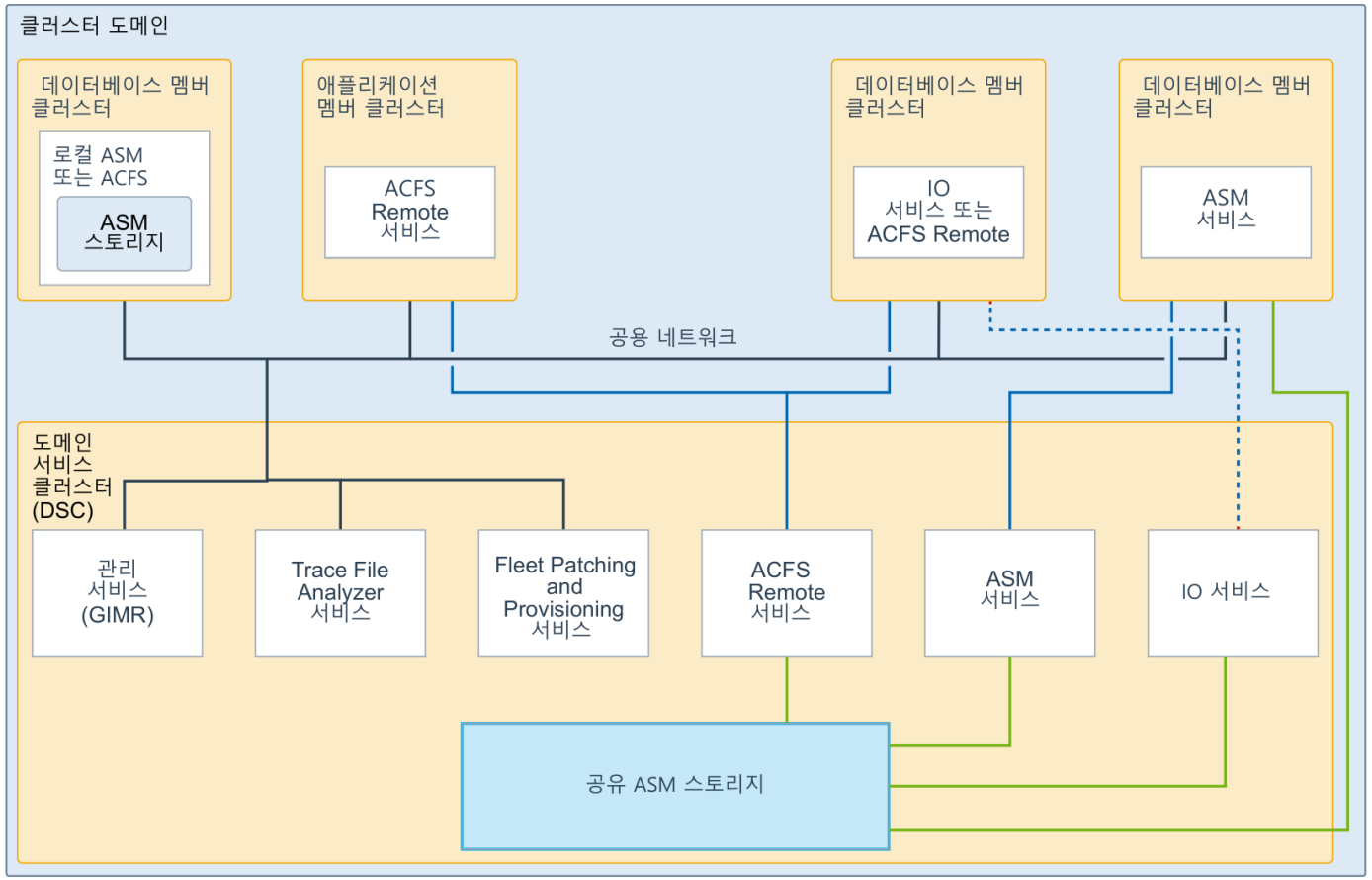
클러스터 도메인에 중앙 집중 서비스를 제공하는 클러스터

멤버 클러스터

클러스터 도메인에 속해 도메인 서비스 클러스터의
서비스를 받는 클러스터

클러스터 도메인

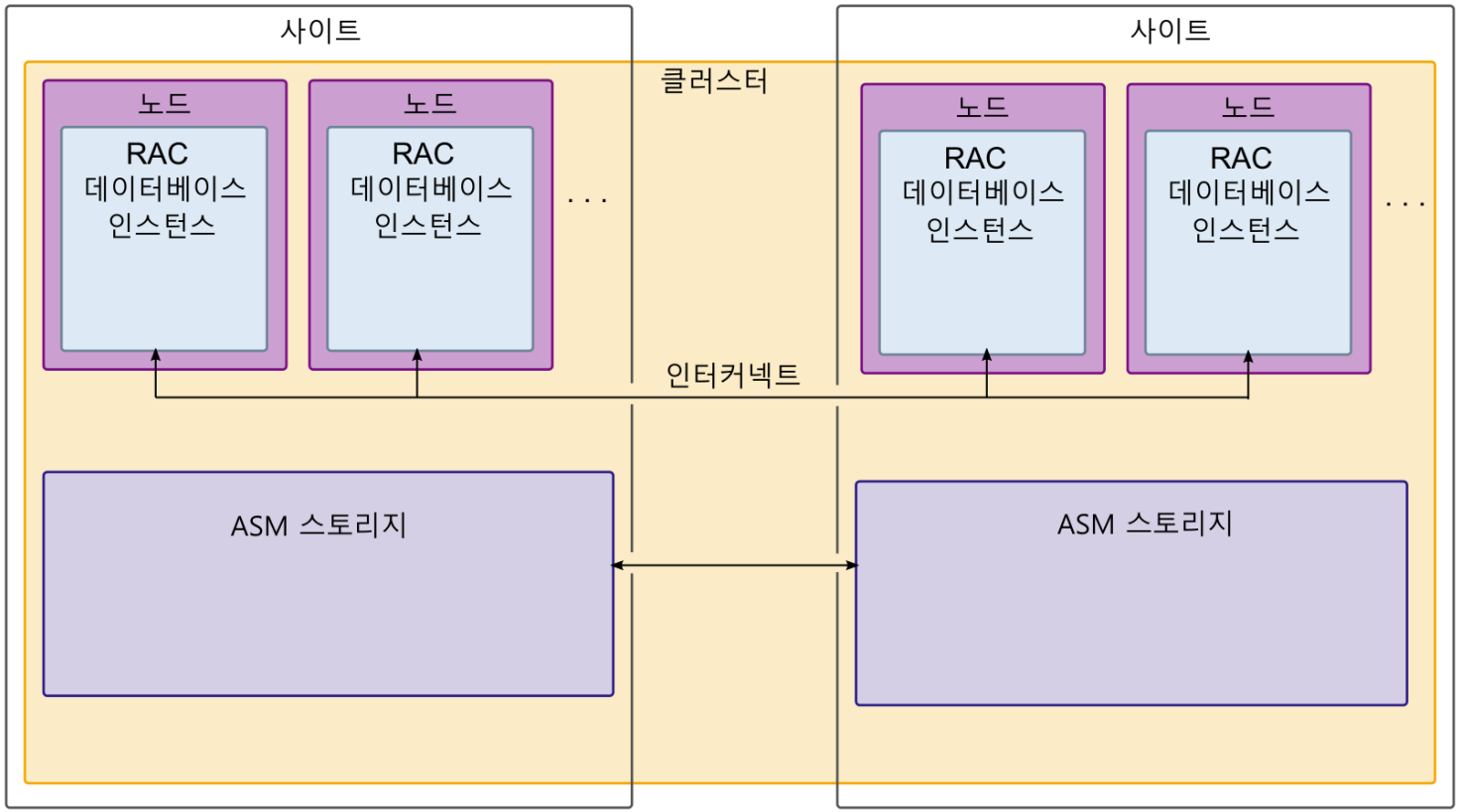
DSC 가 서비스를 주고 멤버 클러스터가 받는다



- DSC 서비스 — GIMR, TFA, FPP, ACFS Remote, ASM, IO
- 멤버 클러스터 — 로컬 ASM, ASM 서비스, IO 서비스·ACFS Remote

확장 클러스터

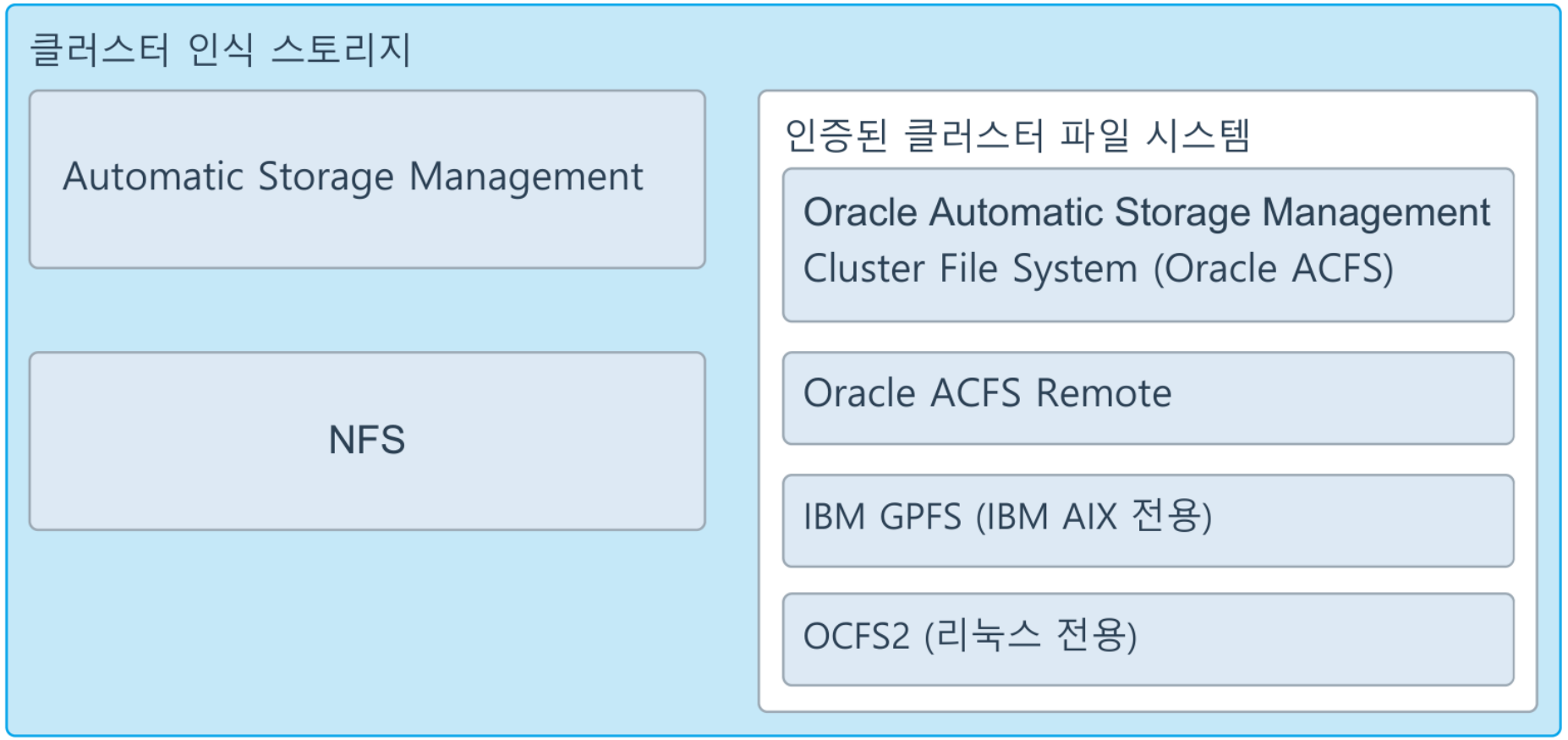
사이트 둘 이상, 사이트마다 스토리지



- Normal 이중화 디스크그룹 권장 — 사이트 하나 실패를 견딘다
- 사이트 → 장애 그룹 → 디스크
- 사이트 사이 100km 미만 권장

클러스터 스토리지

ASM 을 강하게 권장



- NFS, OCFS2(리눅스), GPFS(AIX) 도 지원
- GI 바이너리는 블록·원시 장치나 ACFS 에 둘 수 없다

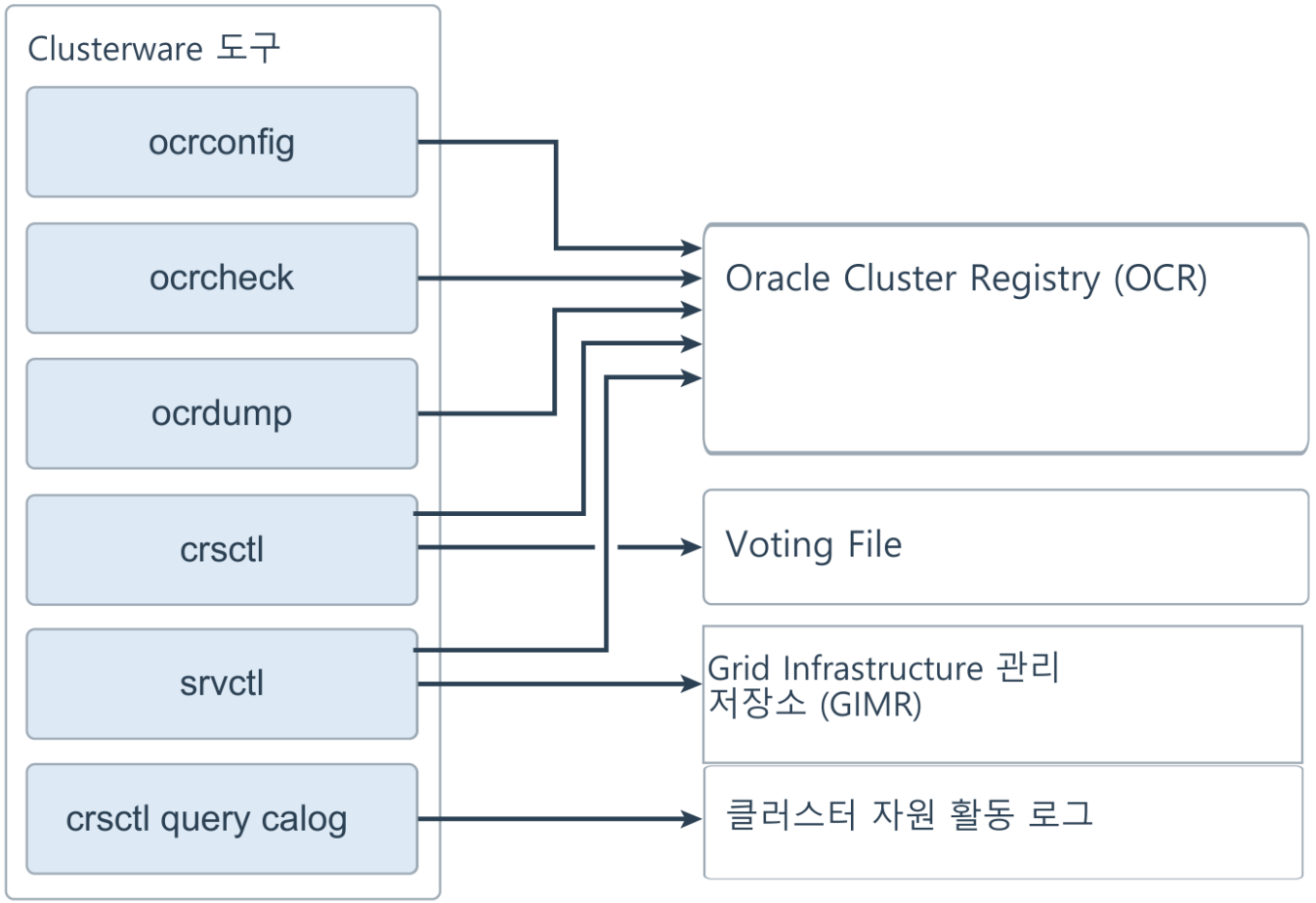
4

OCR 과 Voting File

- GI 파일과 관리 저장소
- OCR · OLR · 백업 · 다중화
- Voting File · 하트비트 · 과반수
- 복구 절차

GI 파일과 도구

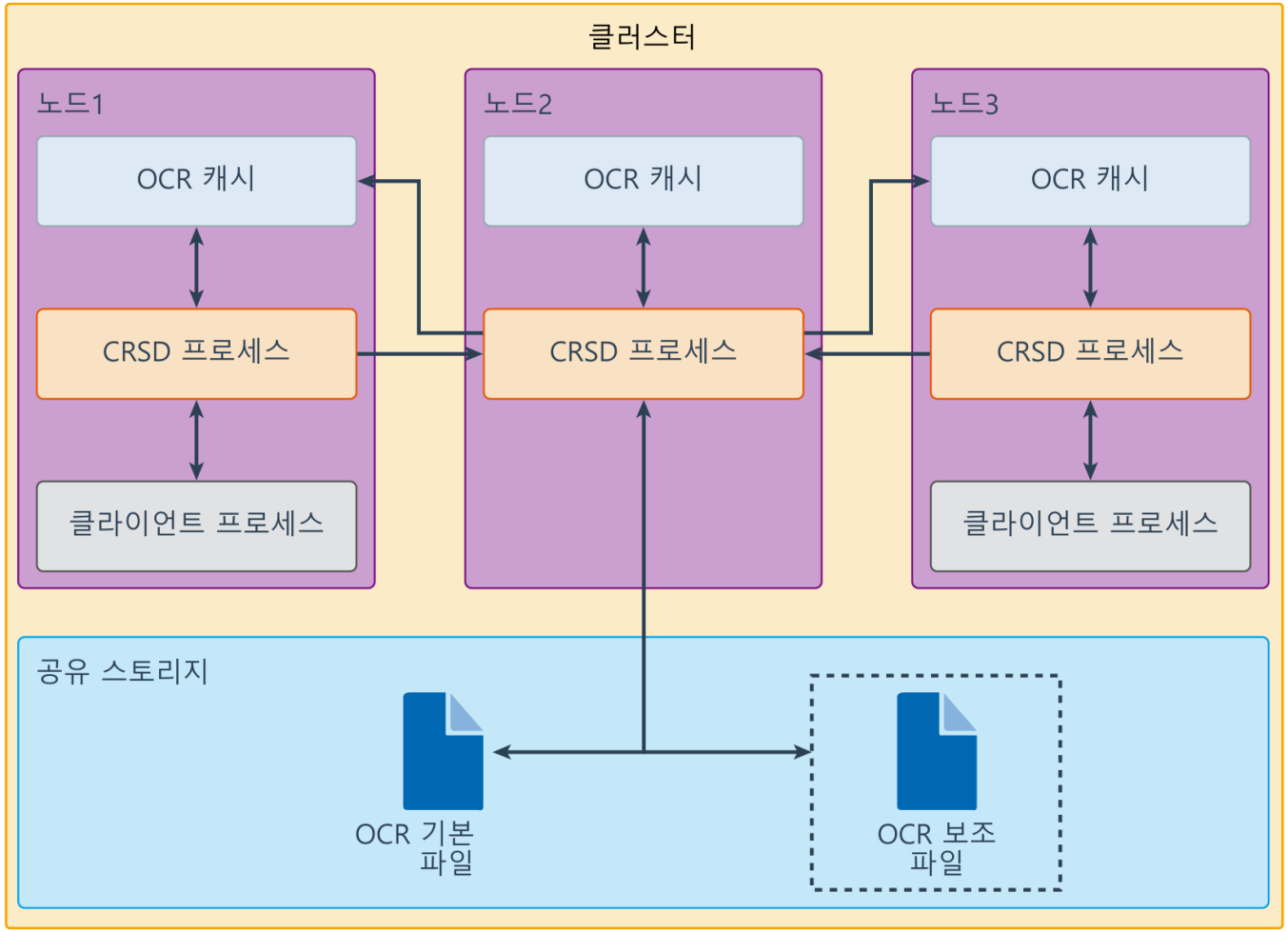
OCR 은 ocrconfig·ocrcheck·ocrdump, Voting File 은 crsctl



- Voting File — 디스크 하트비트와 멤버십으로 노드 펜싱
- OCR — 클러스터 구성과 자원 정보
- 둘 다 ASM 에 둔다, RAID 가 없으면 여러 곳에
- GIMR — 19c 독립형 클러스터에서는 권장

OCR 과 CRSD

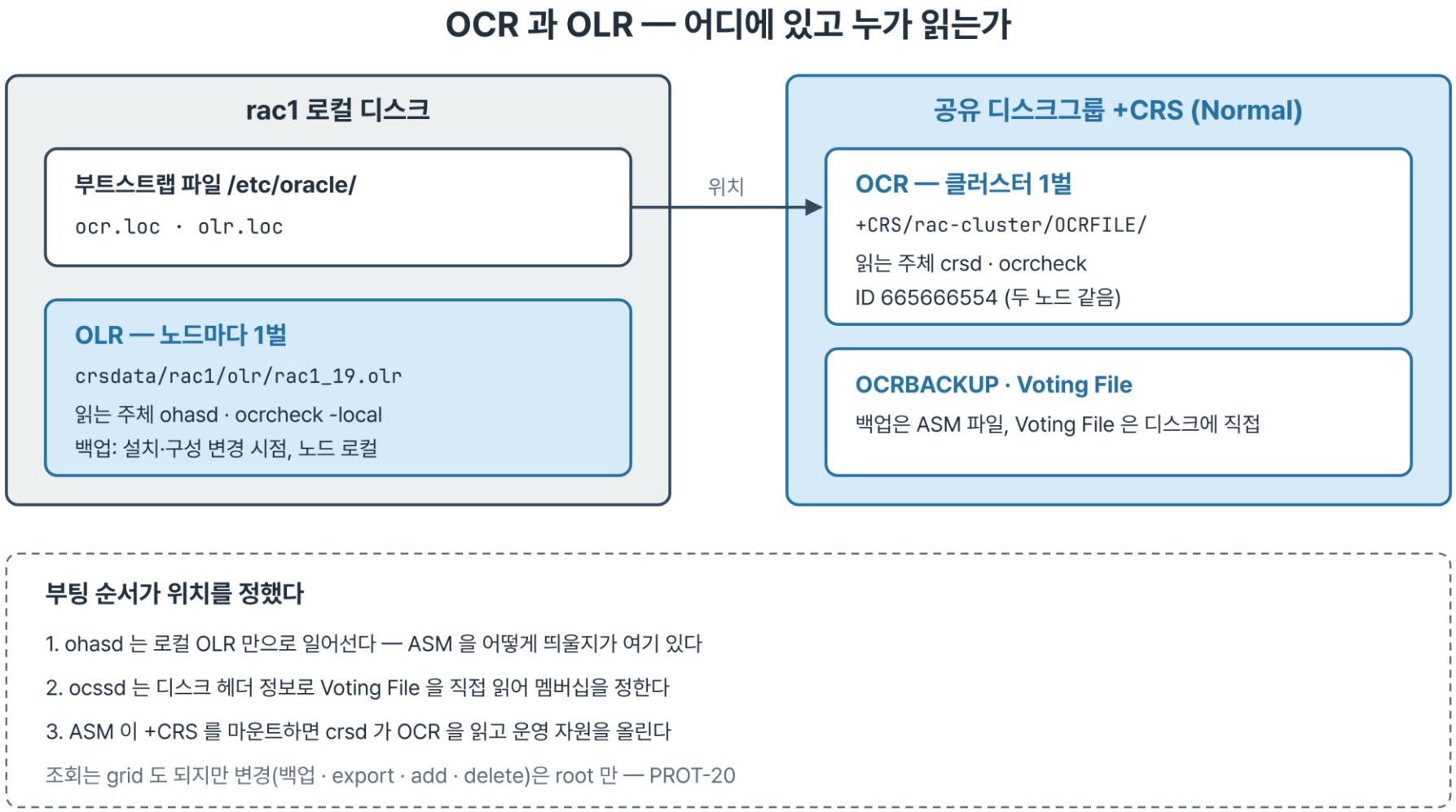
파일을 읽고 쓰는 CRSD 는 하나뿐



- 노드마다 OCR 캐시와 CRSD
- 조회 — 로컬 CRSD 에 직접
- 갱신 — 로컬 CRSD → I/O 담당 CRSD → 디스크
- OCR 위치는 최대 5곳

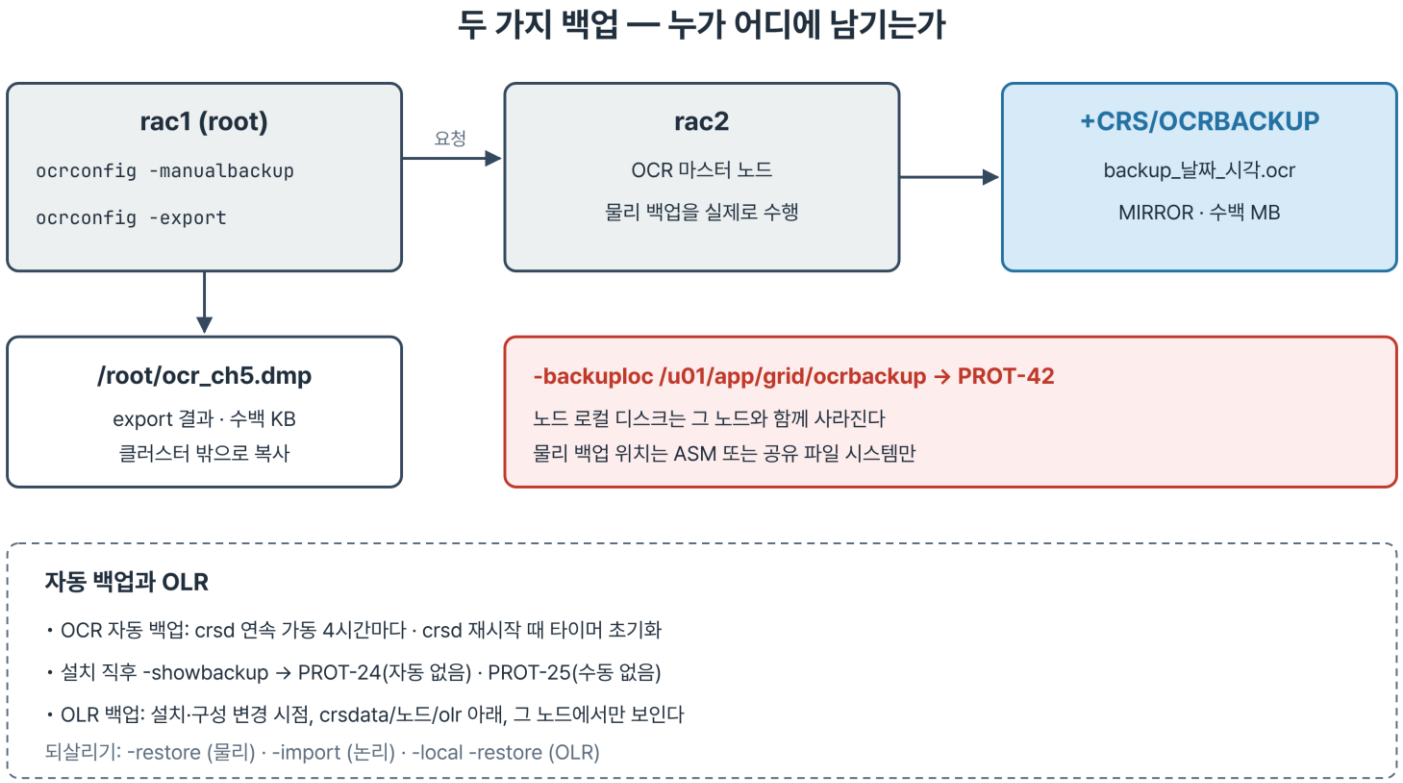
OCR 과 OLR

어디에 있고 누가 읽는가



OCR 백업 두 가지

물리 백업과 논리 내보내기



- 설치 직후 — PROT-24·25, 백업이 한 벌도 없다
- 자동 백업 — crsd 연속 가동 4시간마다, 재시작하면 타이머 초기화
- 로컬 경로 백업 위치는 PROT-42 로 거절

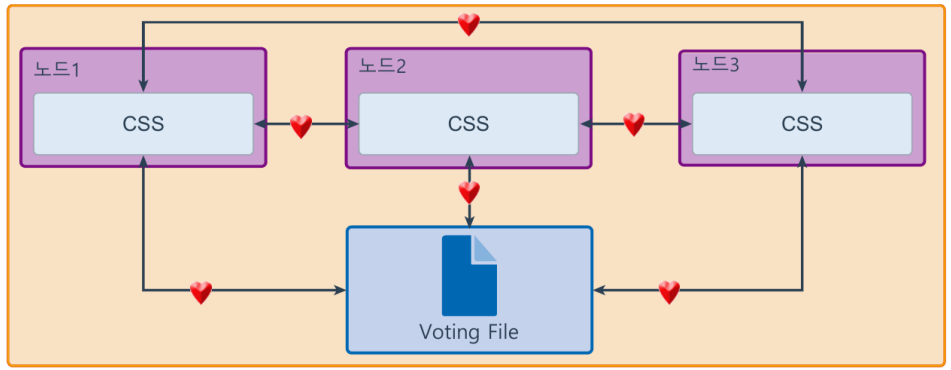
OCR 다중화와 권한

PROT 번호로 원인을 읽는다

- 조회는 grid, 변경은 root — grid 로 바꾸면 PROT-20
- 없는 디스크그룹 추가 → PROT-30 + PROC-50(접근 못 한 노드)
- 이미 있는 위치 → PROT-29, 마지막 한 벌 삭제 → PROT-28
- 클러스터가 떠 있으면 복원 불가 → PROT-19
- 위치 이동 — 새 위치 -add, 검증, 옛 위치 -delete

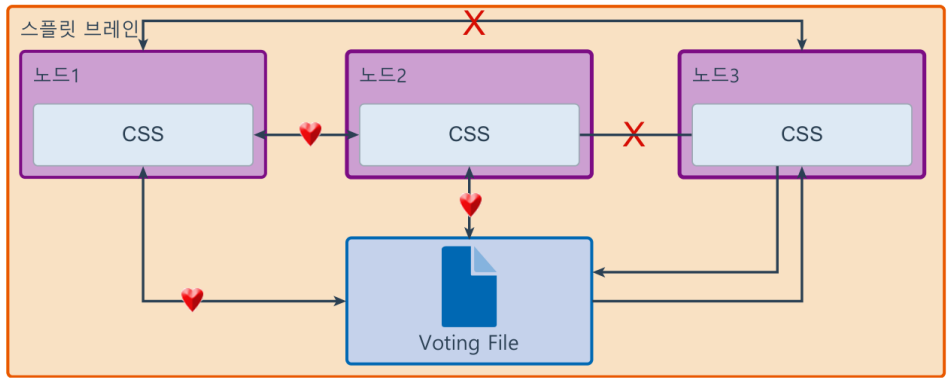
Voting File

정상 하트비트와 스플릿 브레인



노드들은 사설 인터커넥트로 보내는 하트비트와 Voting File 에 쓰는 하트비트로 서로의 생존을 확인한다.

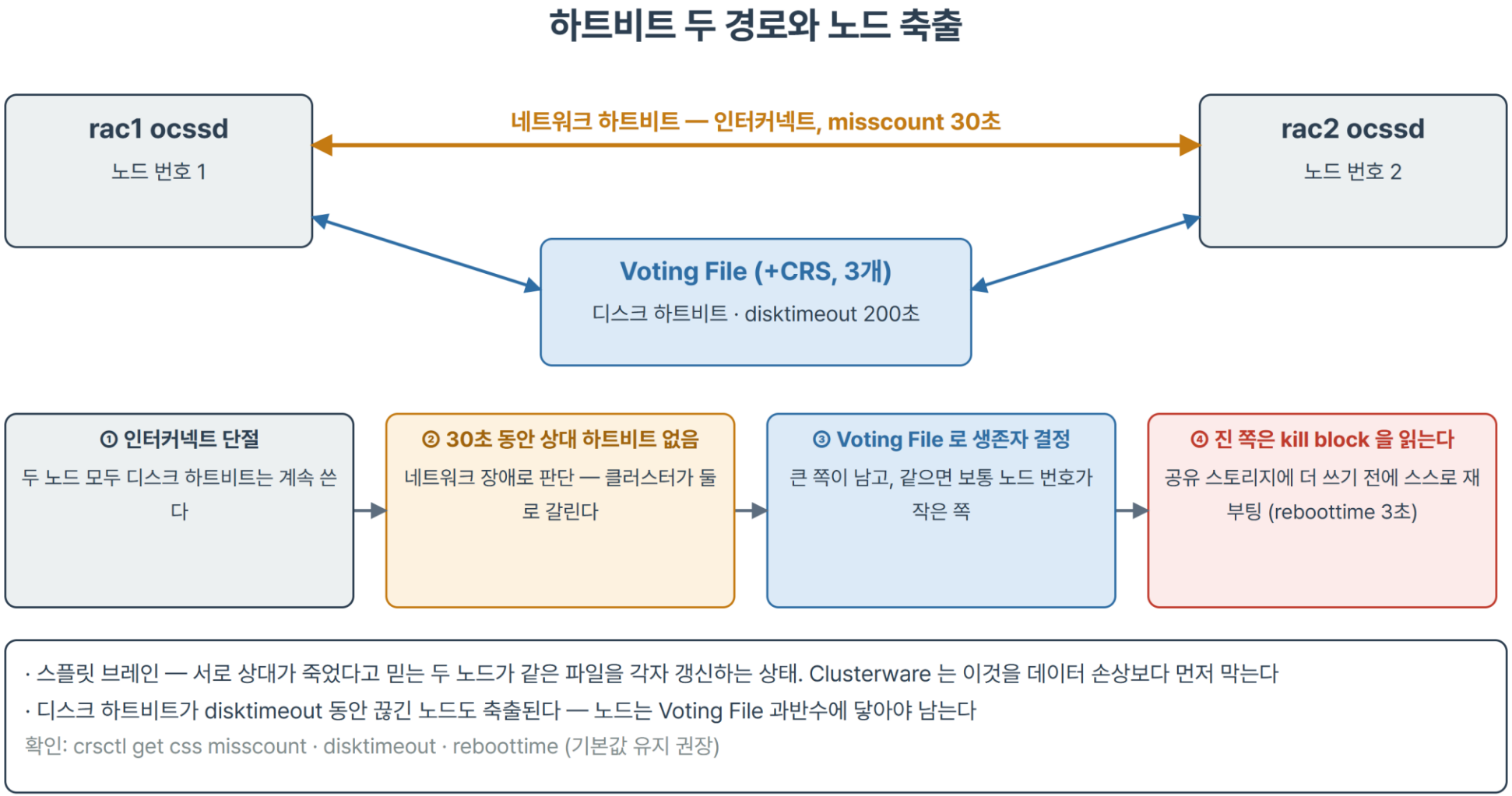
노드3 이 사설 인터커넥트로 통신하지 못한다. 다른 노드는 노드3 의 하트비트를 더 이상 보지 못하고 노드3 을 축출한다.



- 네트워크 하트비트 — 인터커넥트
- 디스크 하트비트 — Voting File
- 노드3 의 하트비트가 끊기면 나머지가 Voting File 로 노드3 축출
- 노드3 은 kill block 을 읽고 스스로 재부팅

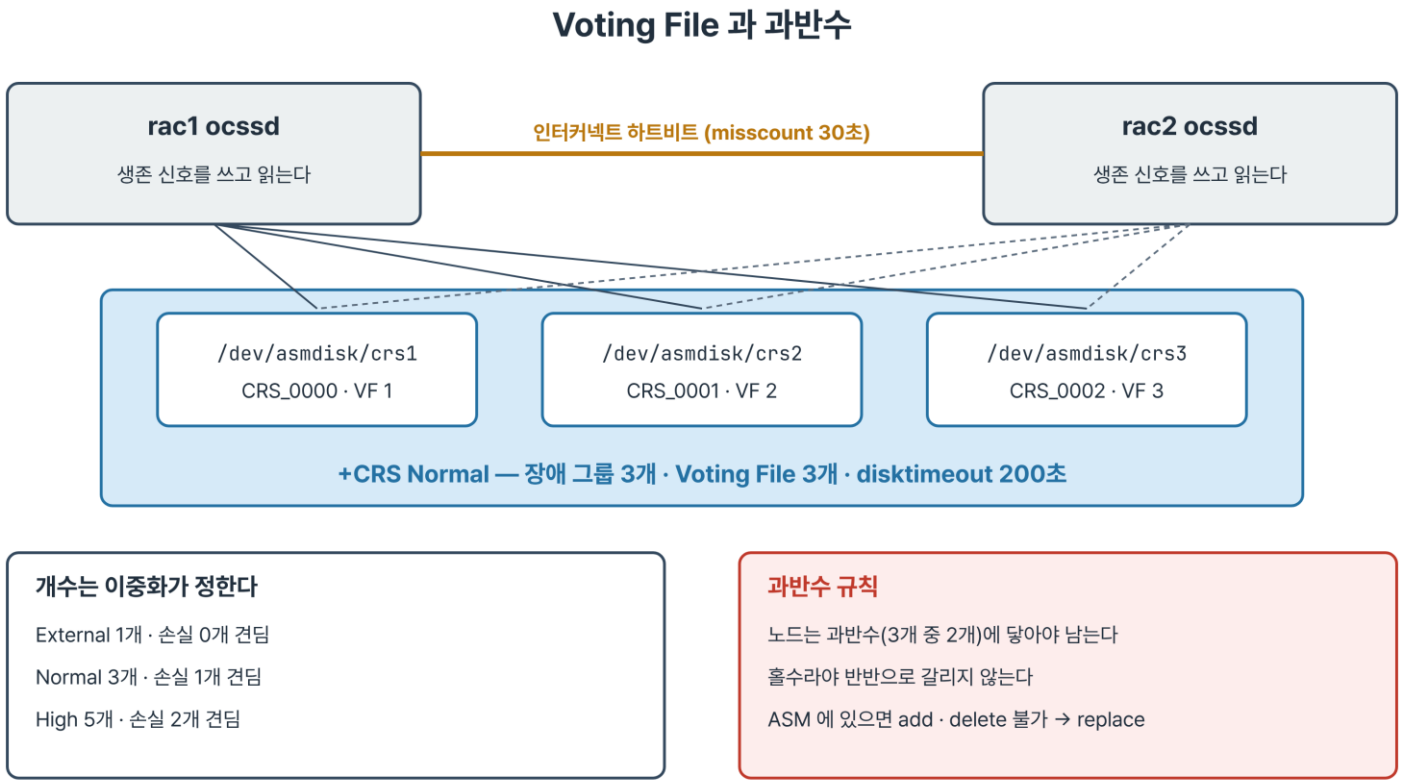
하트비트 두 경로와 축출

misscount 30초, disktimeout 200초, reboottime 3초



Voting File 과 과반수

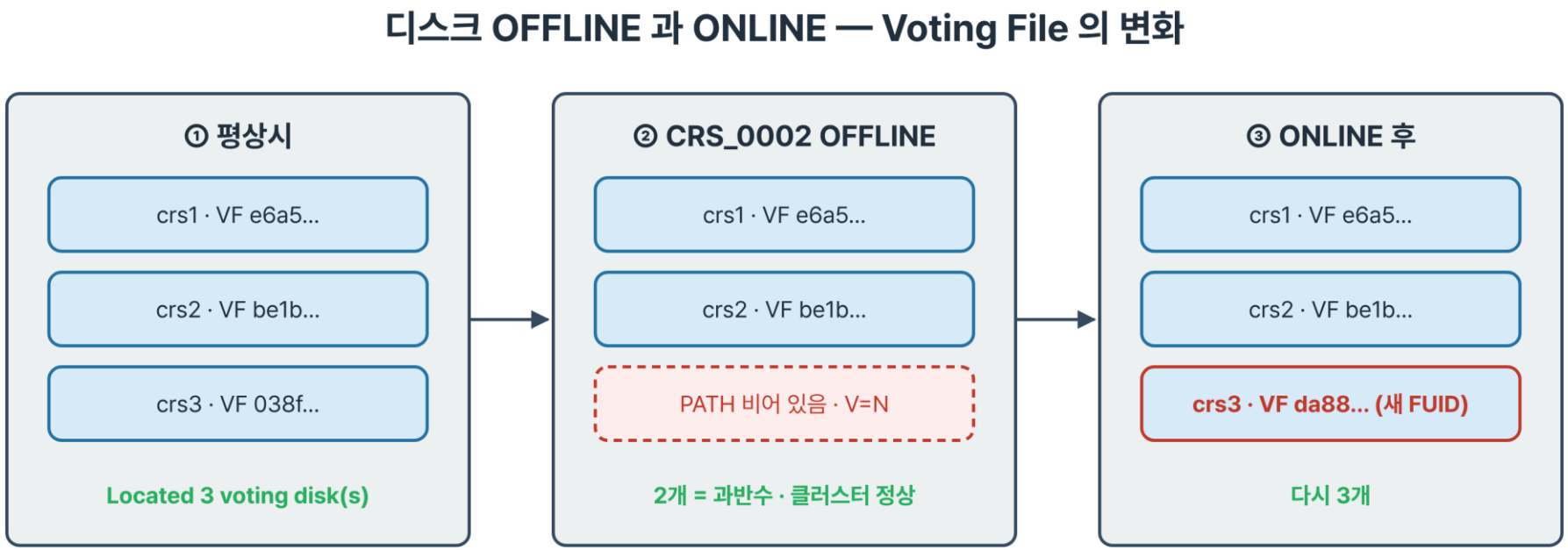
개수는 사람이 아니라 이중화가 정한다



- External 1개, Normal 3개, High 5개
- 노드는 과반수에 닿아야 남는다 — 3개 중 2개
- ASM 에 있으면 add·delete 불가(CRS-4258), replace 만

디스크 하나를 내리면

3개 → 2개(과반수 유지) → 새 FUID 로 다시 3개



ASM alert.log 의 흐름

OFFLINE: GMON 이 PST 갱신 (clear mandatory) → voting file relocation

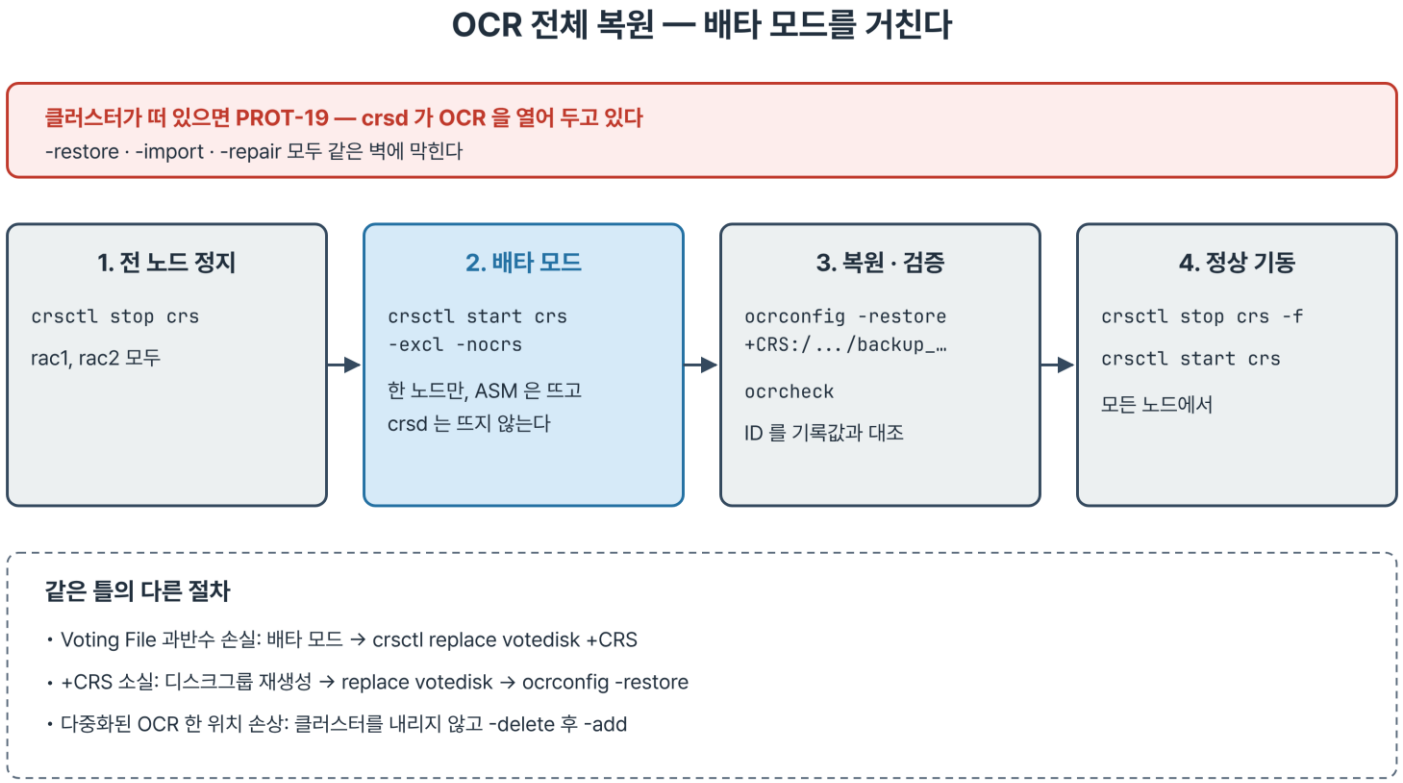
ONLINE: PST 갱신 (assign mandatory) → 리밸런스 → Found 3 voting file(s)

먼저 막히는 곳: compatible.rdbms 10.1 → ORA-15283, 11.2 로 상향 (되돌릴 수 없음)

복구 판단은 FUID 가 아니라 개수와 STATE 로 한다

OCR 전체 복원

모든 노드를 내리고 배타 모드에서



- 한 위치만 손상 — -delete 후 -add, 무중단
- Voting File 과반수 손실 — start crs -excl -nocrs → replace votedisk
- +CRS 소실 — 디스크그룹 재생성 → replace votedisk → -restore

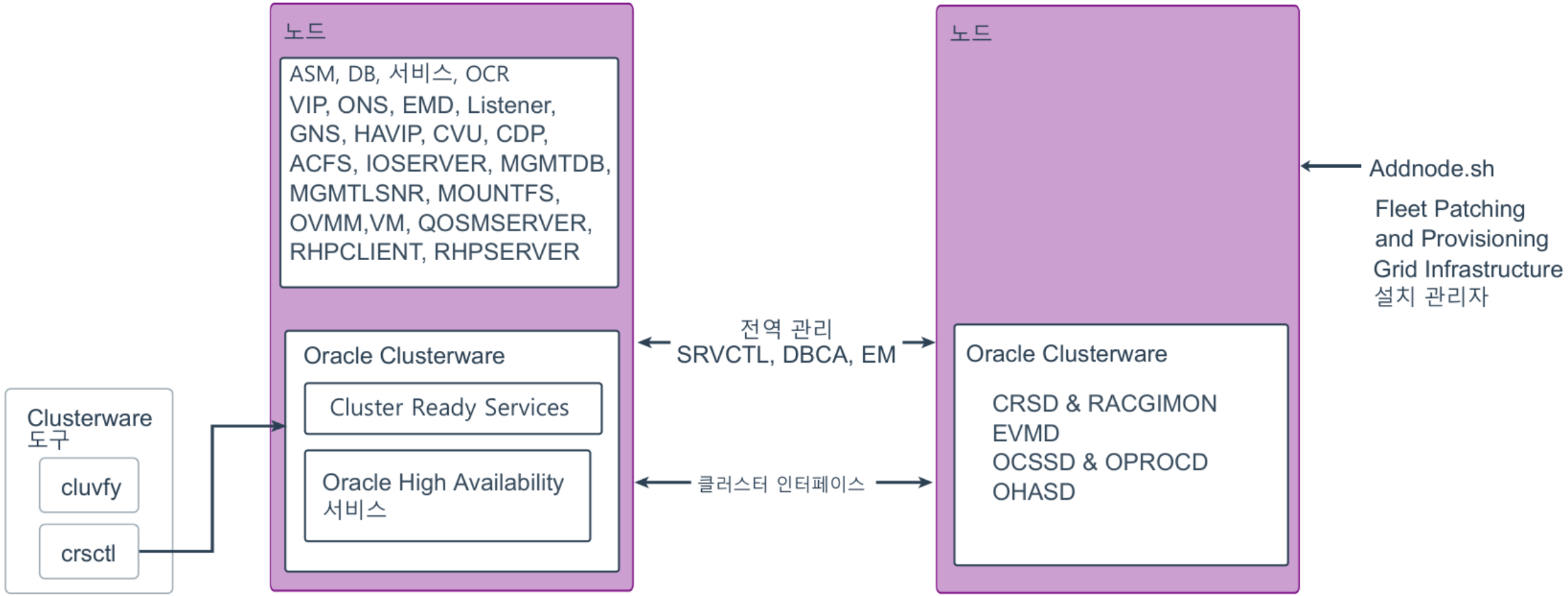
5

Clusterware 프로세스와 도구

- 두 개의 스택 — OHAS 와 CRS
- 프로세스, 실행 계정, 에이전트
- 기동 순서와 자원 의존 관계
- 장애 계층 판별, 두 ORACLE_HOME
- ONS · GNS · 서버 풀

Clusterware 프로세스와 서비스

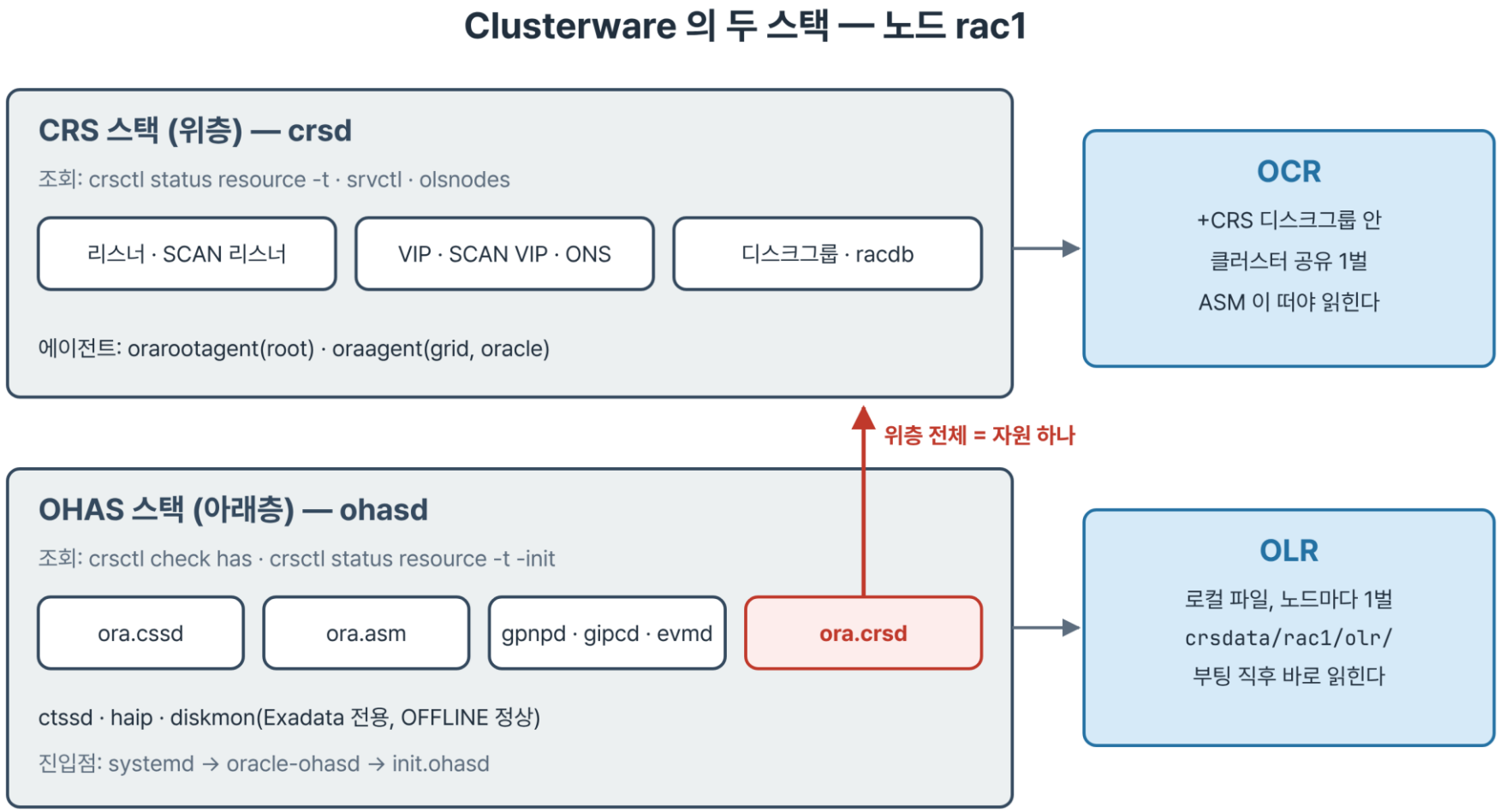
위 스택은 CRSD, 아래 스택은 OHASD



- 노드 추가 — addnode.sh, gridSetup.sh, rhpctl
- Clusterware 가 관리하는 자원 유형 — ASM, DB, 서비스, VIP, ONS, 리스너 ...

두 개의 스택

위층 전체가 아래층에게는 자원 하나



아래층 명령이 되고 위층 명령이 안 되면 crsd 이하가, 아래층도 안 되면 ohasd 가 문제다

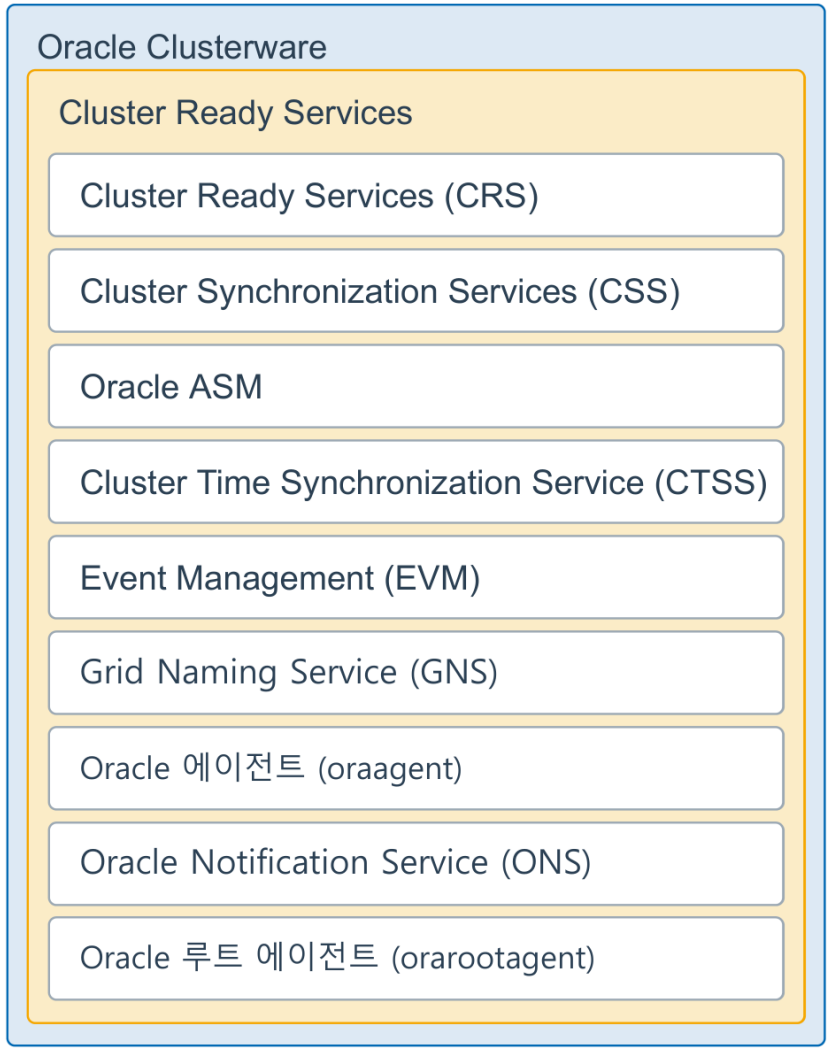
stop cluster 와 stop crs

내리는 범위가 다르다

- stop cluster — CRS 계층과 그 아래 자원, ohasd 는 남는다
- stop crs — OHAS 를 포함한 노드 스택 전체
- 정지 후 check has — CRS-4638 online / CRS-4639
- 마지막 메시지 — CRS-2792 / CRS-2792 → 2793 → 4133
- 노드를 잠시 뺄 때 stop cluster, 노드 정지·OS 패치는 stop crs

CRS 스택

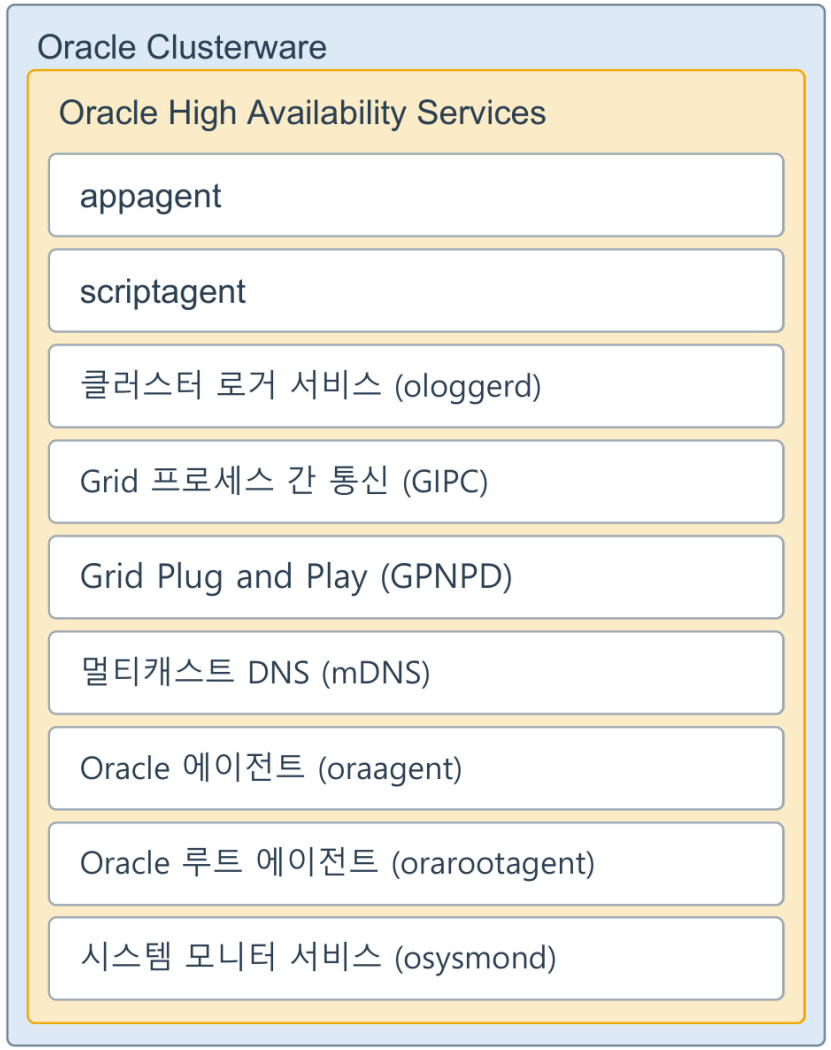
CRSD 가 OCR 의 정의대로 자원을 기동·감시·페일오버



- 자원 상태가 바뀌면 이벤트를 만든다
- 인스턴스·리스너가 죽으면 자동 재기동
- oraagent·orarootagent — OHAS 쪽과 이름은 같아도 다른 프로세스

OHAS 스택

노드 부팅 직후 OLR 만으로 서는 아래층



- GPNPD — 프로필(인터커넥트, ASM 발견 문자열)
- GIPC — 노드 간 통신 계층
- mDNS — 멀티캐스트 이름 해석
- osysmond · ologgerd — 시스템 자원 수집

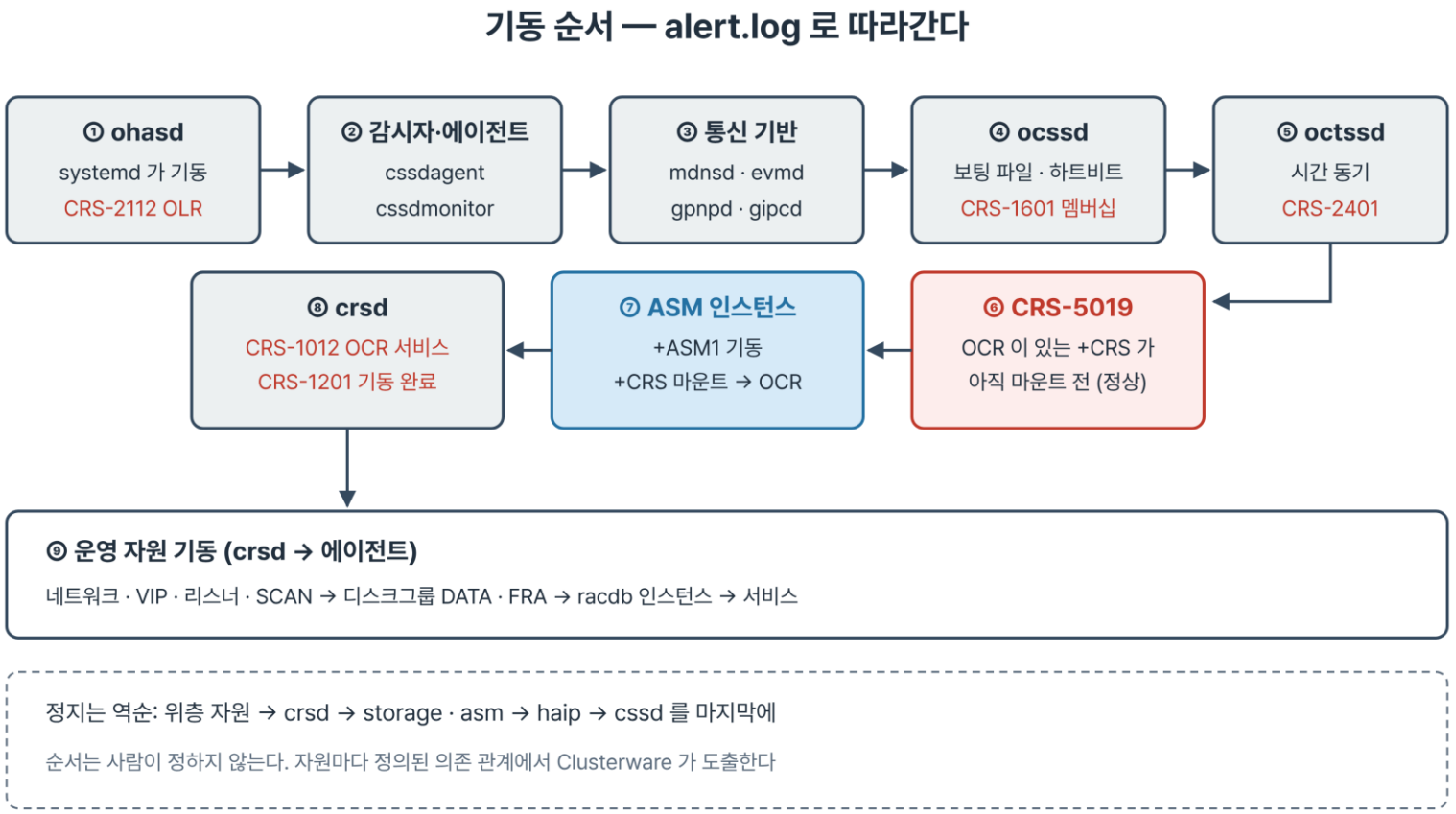
주요 데몬과 실행 계정

노드를 내릴 권한이 필요한 것은 root

- ohasd · crsd · octssd · osysmond — root
- ocssd · evmd · gpnpd · gipcd · mdnsd — grid
- cssdagent — ocssd 감시, 응답 없으면 노드 재부팅
- cssdmonitor — cssdagent 와 시스템 행 감시
- 에이전트가 죽어도 자원은 산다 — 감시만 잠시 끊긴다
- crsctl config 계열은 조회도 root — grid 면 CRS-4563

기동 순서

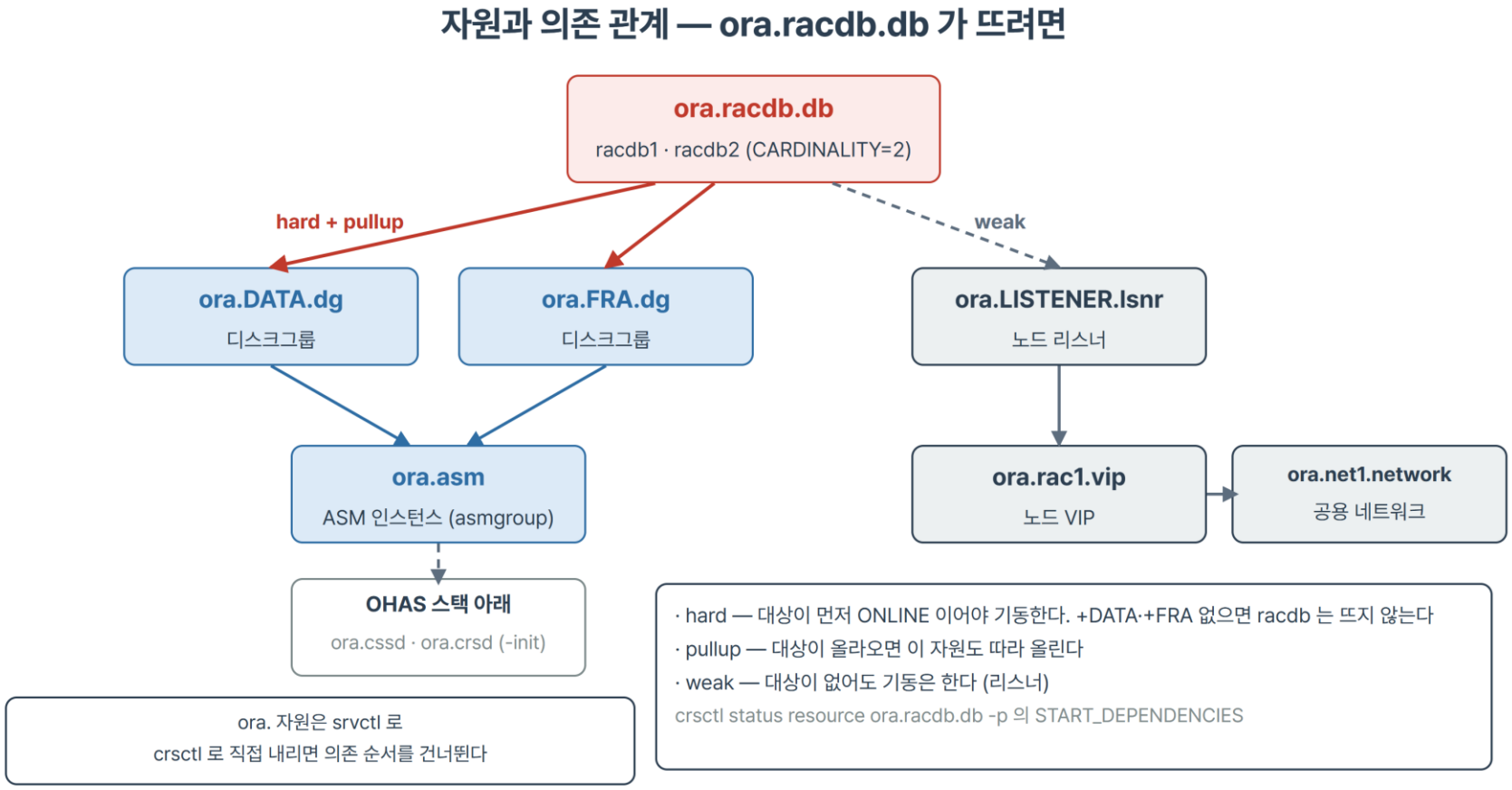
alert.log 로 따라간다



- CRS-1601 — 멤버십 확정, 축출 조사 때 먼저 찾는다
- CRS-5019 — 부팅 중 정상 과도기, OCR 복구 대상 아님

자원과 의존 관계

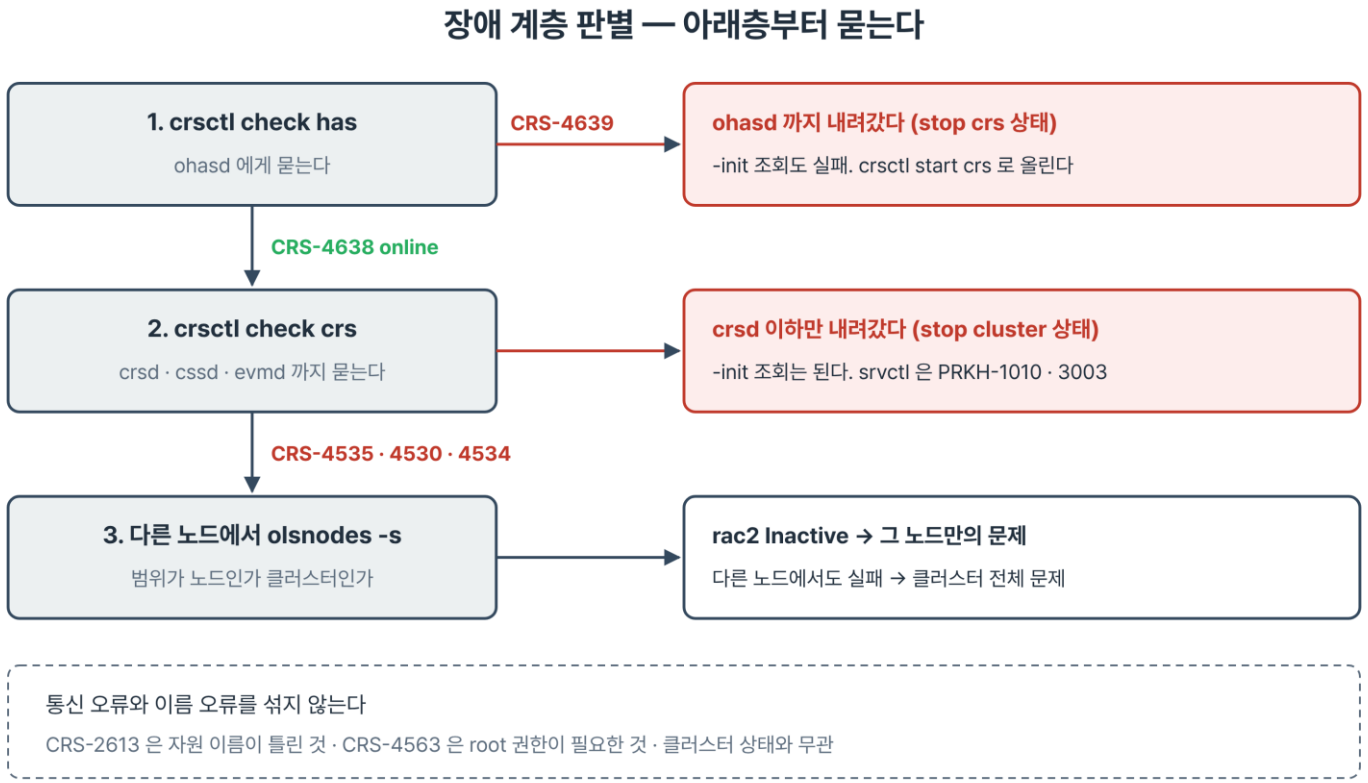
hard · pullup · weak



- 이름 규칙 — ora.racdb.db, ora.rac1.vip, ora.DATA.dg
- local 자원 — 노드마다 / cluster 자원 — 클러스터 전체에서 개수

장애 계층 판별

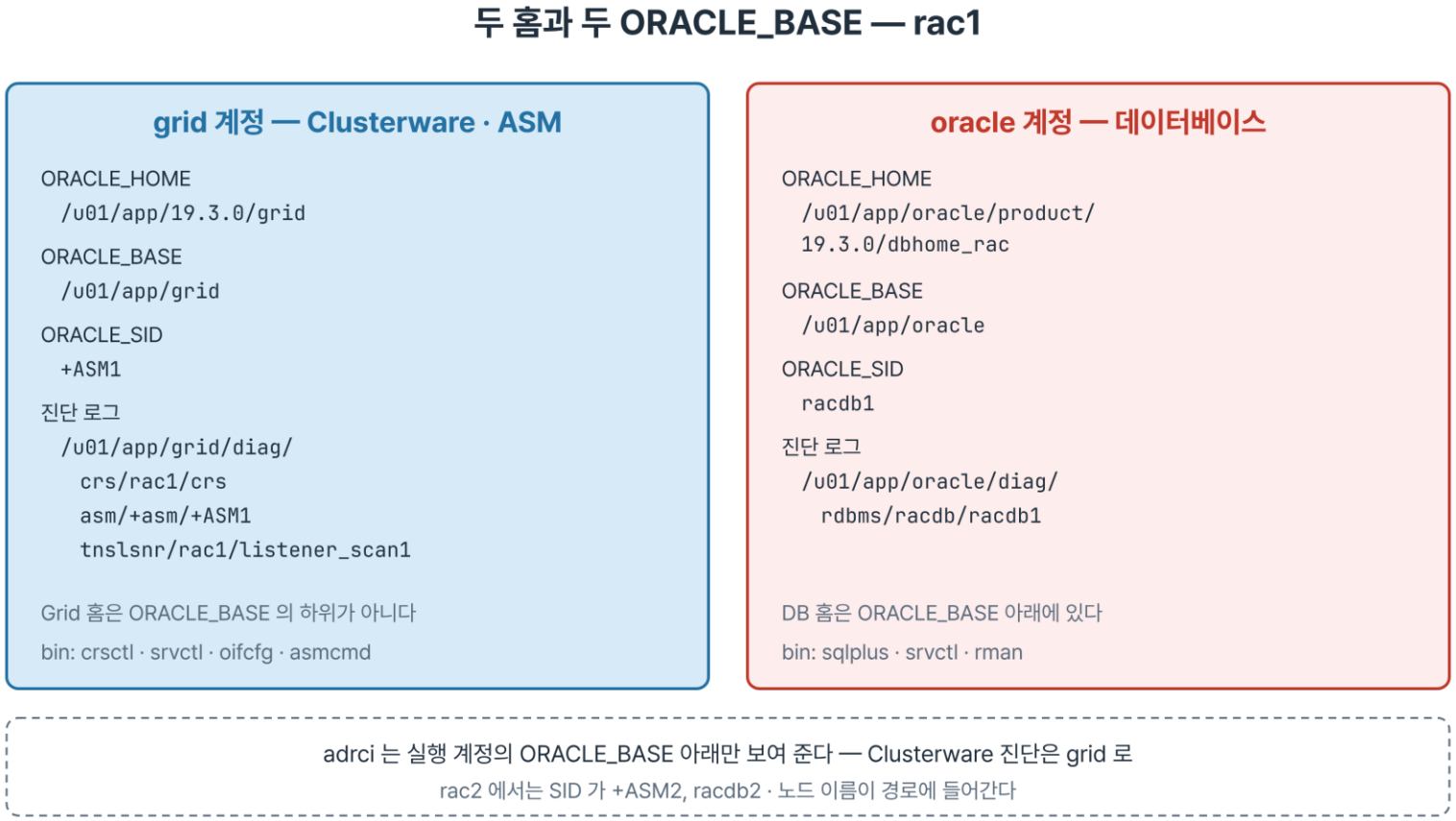
아래층부터 묻고, 다른 노드에서 확인한다



- CRS-4639 — ohasd 에 닿지 못함
- CRS-4535 · 4530 · 4534 — crsd · ocssd · evmd
- PRKH-1010 · 3003 — srvctl 이 CRS·CSS 에 닿지 못함

두 홈과 두 ORACLE_BASE

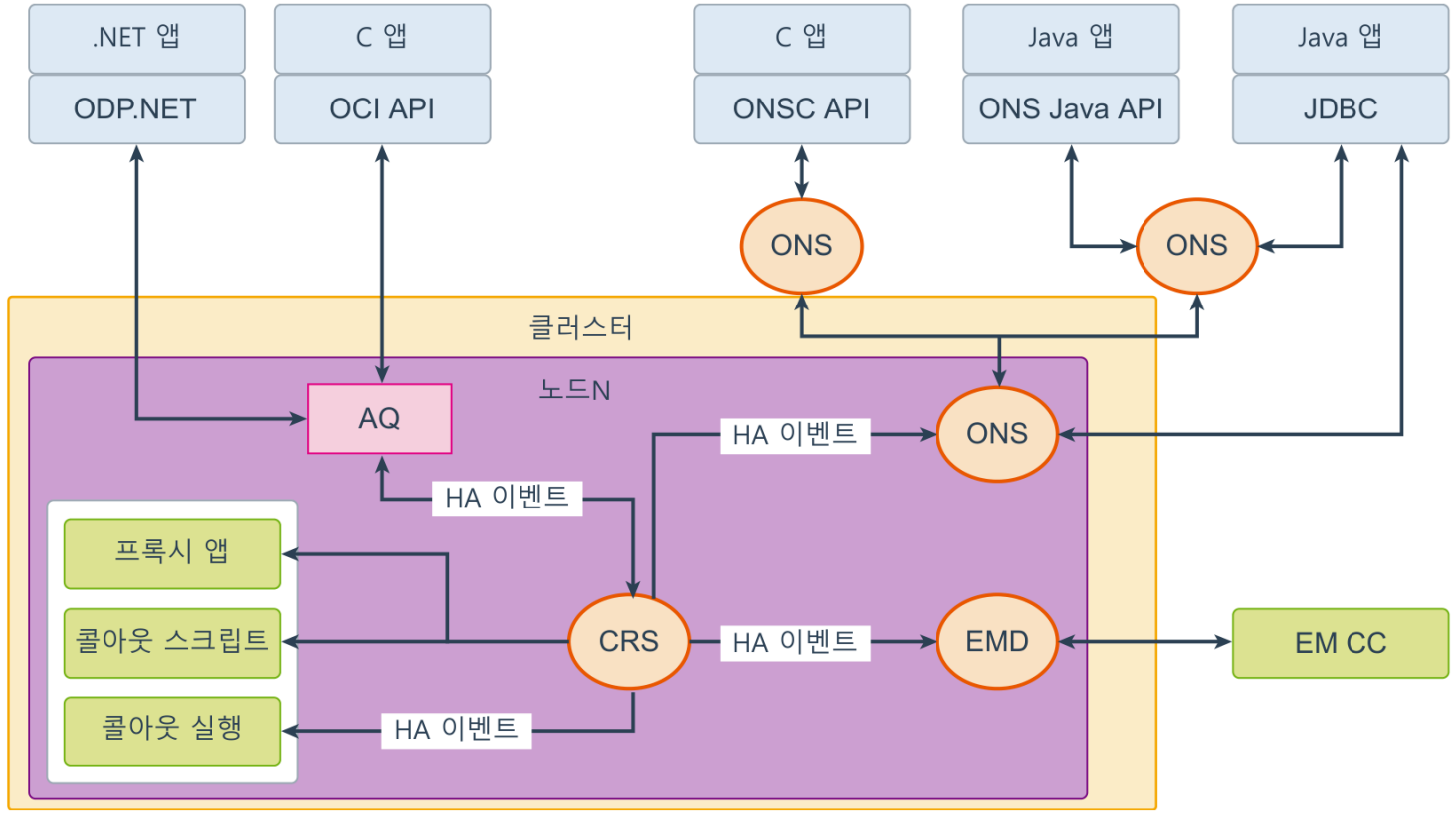
진단 로그 위치도 계정에 따라 갈린다



- 작업 시작 — whoami; echo $ORACLE_HOME; echo $ORACLE_SID
- Clusterware 진단은 grid 로

ONS 와 FAN

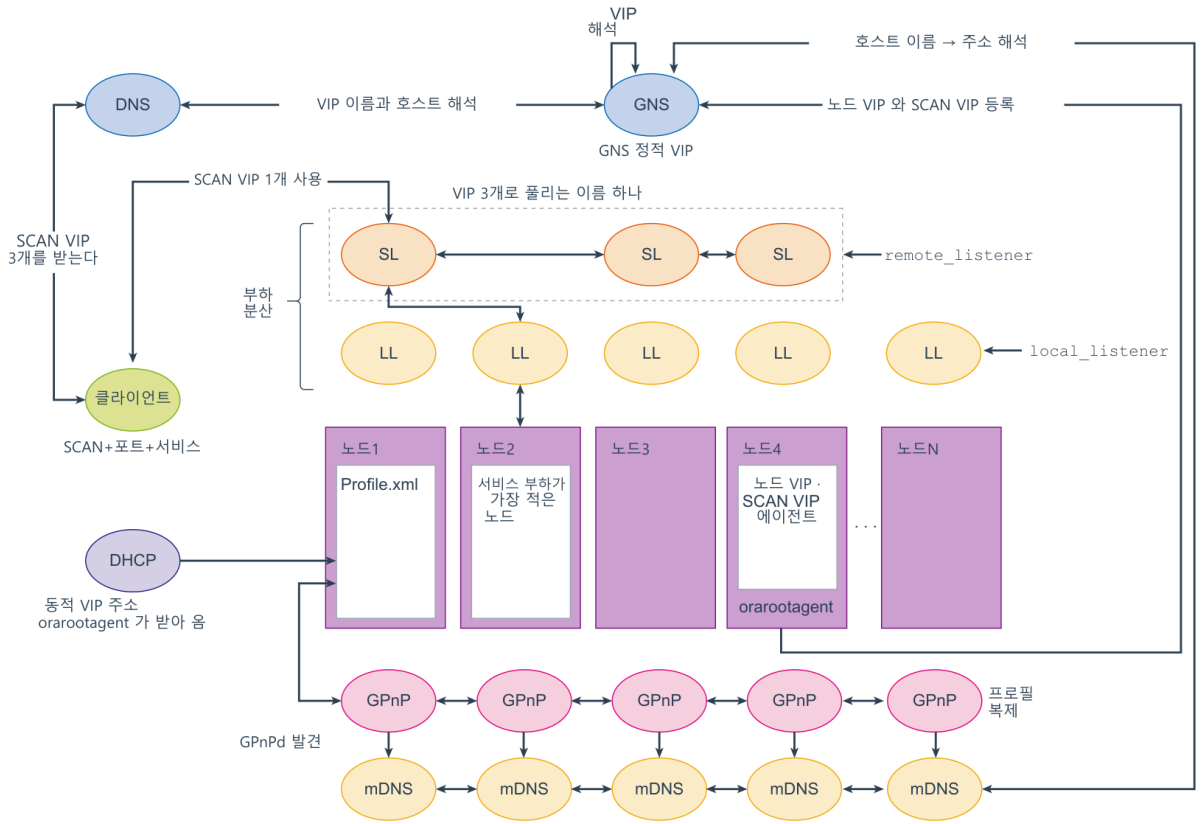
자원 상태 변화를 애플리케이션에 알린다



- HA 이벤트 → ONS → 중간 계층·클라이언트
- AQ 로 .NET 등 외부 애플리케이션에도
- FAN 통합 클라이언트 — JDBC, OCI 풀, ODP.NET, CMAN, TAF

Grid Naming Service

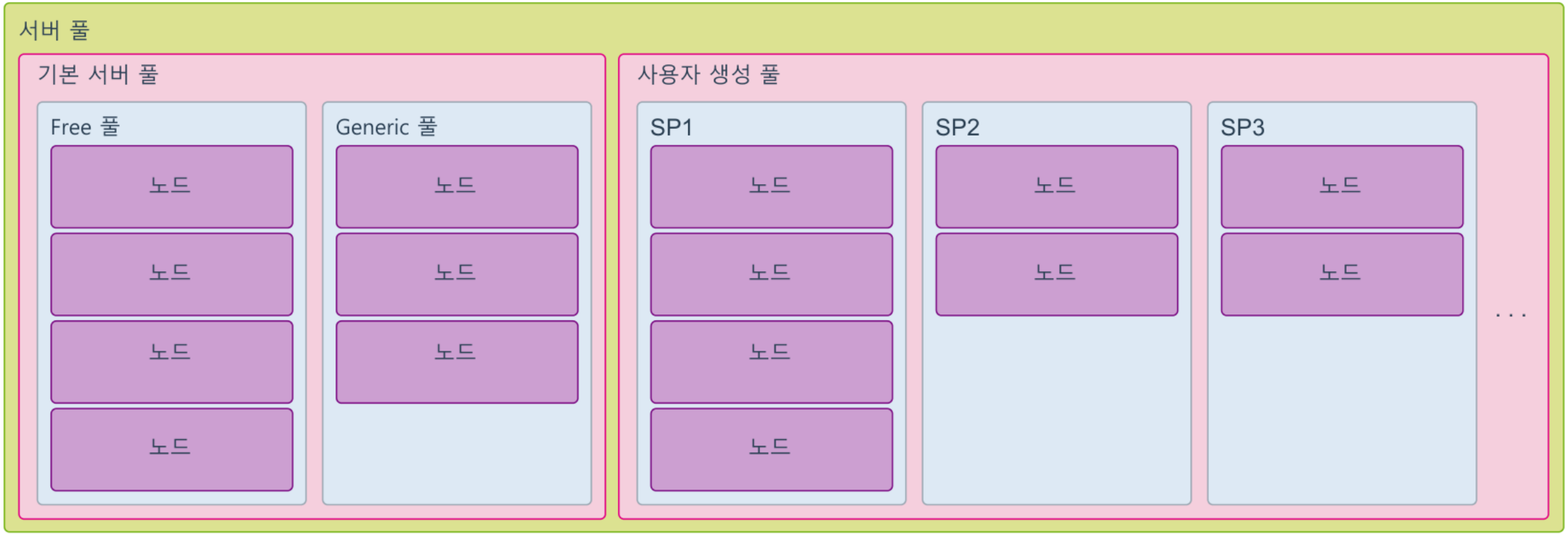
동적 주소를 쓰는 클러스터의 이름 해석



- GPnP — 마스터 없는 피어 투 피어 프로필 복제
- 도메인 안은 mDNS, 밖은 DNS 가 하위 도메인을 GNS 로 위임
- SCAN 은 노드가 아니라 클러스터로 해석된다

서버 풀

관리자 관리형은 Generic 아래에 같은 이름의 풀



- 설치하면 Free 와 Generic 이 생긴다
- 정책 기반 — 최소·최대 크기와 중요도로 서버 배정
- 관리자 관리 — 인스턴스를 노드에 고정, 서비스는 preferred·available

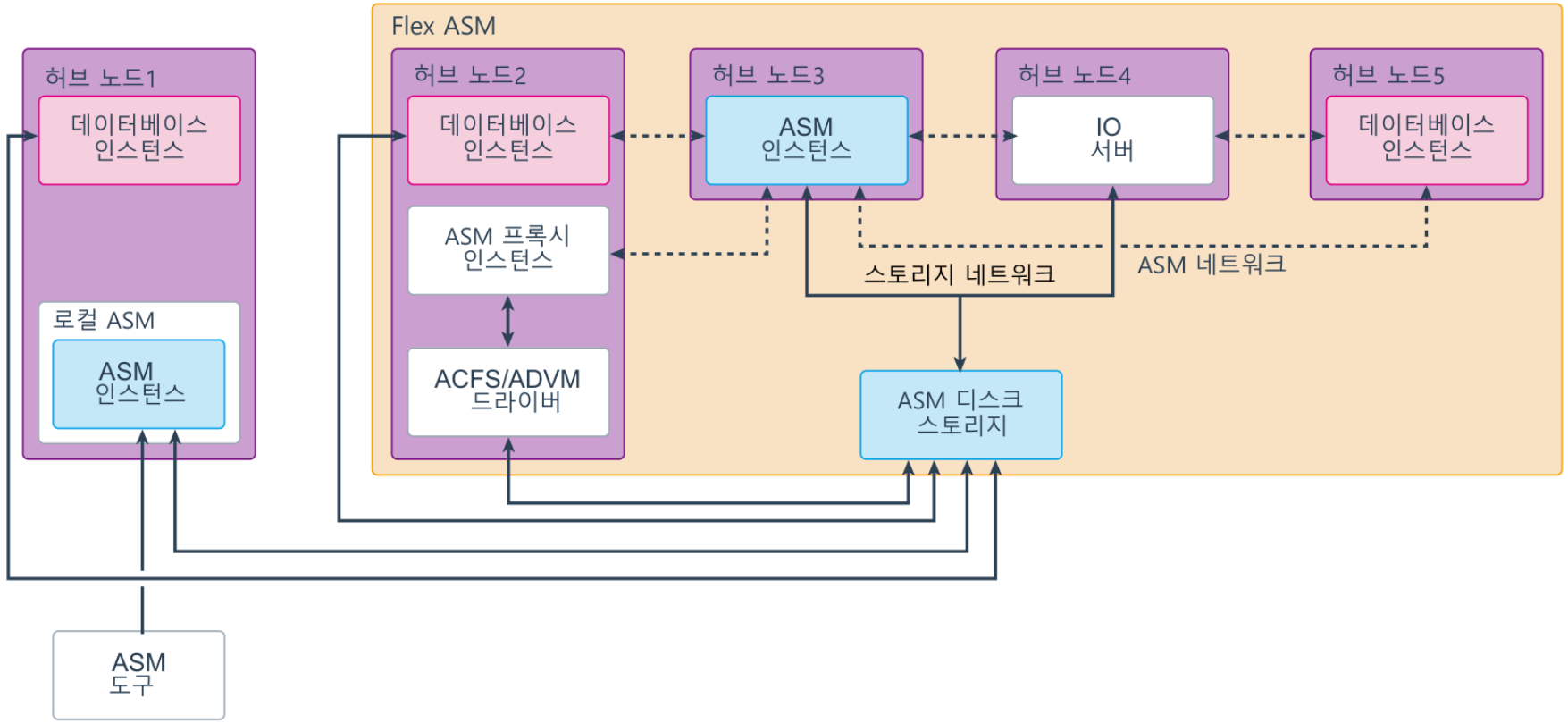
6

ASM

- Flex ASM 과 IO 서버
- ASM 인스턴스
- 디스크그룹 · 장애 그룹 · 파일 그룹
- ACFS 와 ACFS Remote

ASM 구성

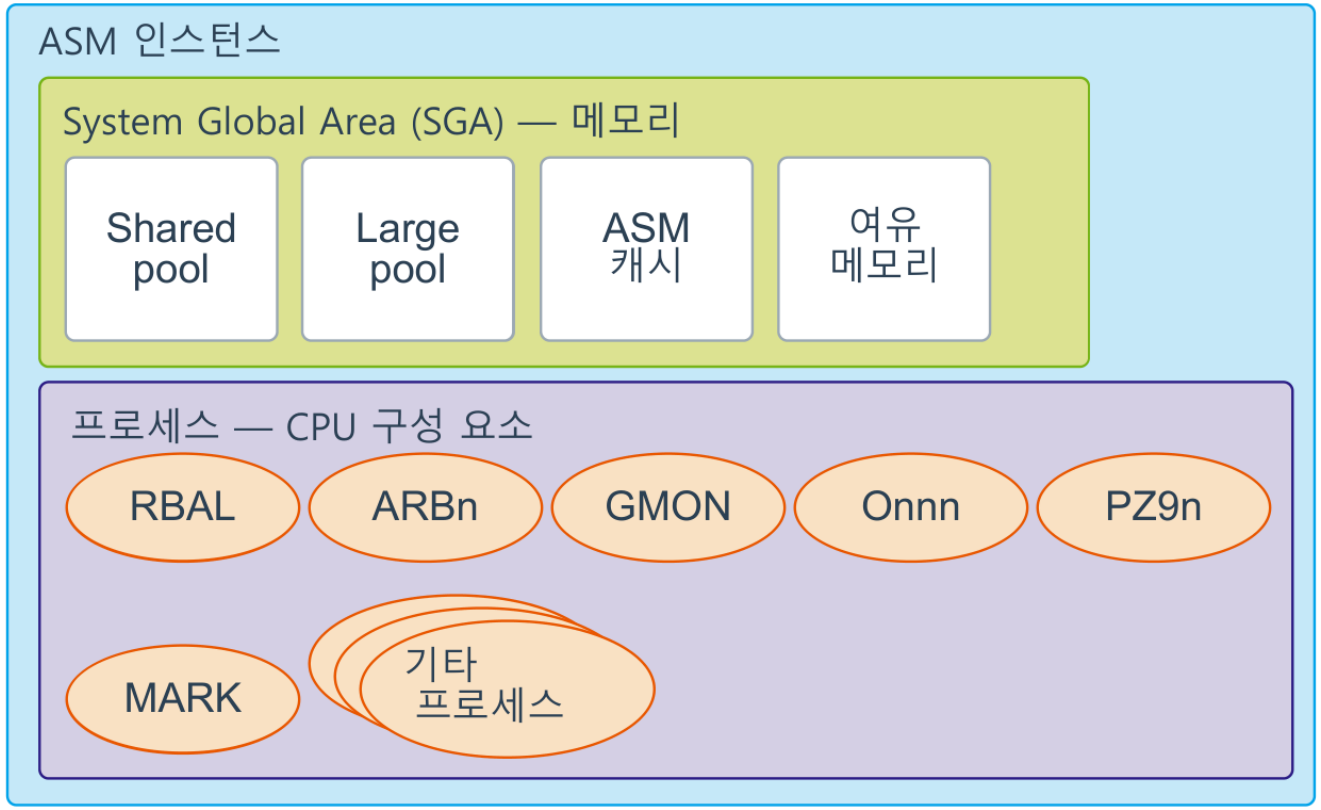
로컬 ASM · ASM 클라이언트 · IO 서버



- 허브 노드1 — 전통적인 로컬 ASM
- 허브 노드2 — 메타데이터는 남의 ASM, 블록은 디스크에 직접
- 허브 노드5 — 디스크 접근 없이 IO 서버로 (12.2~)

ASM 인스턴스

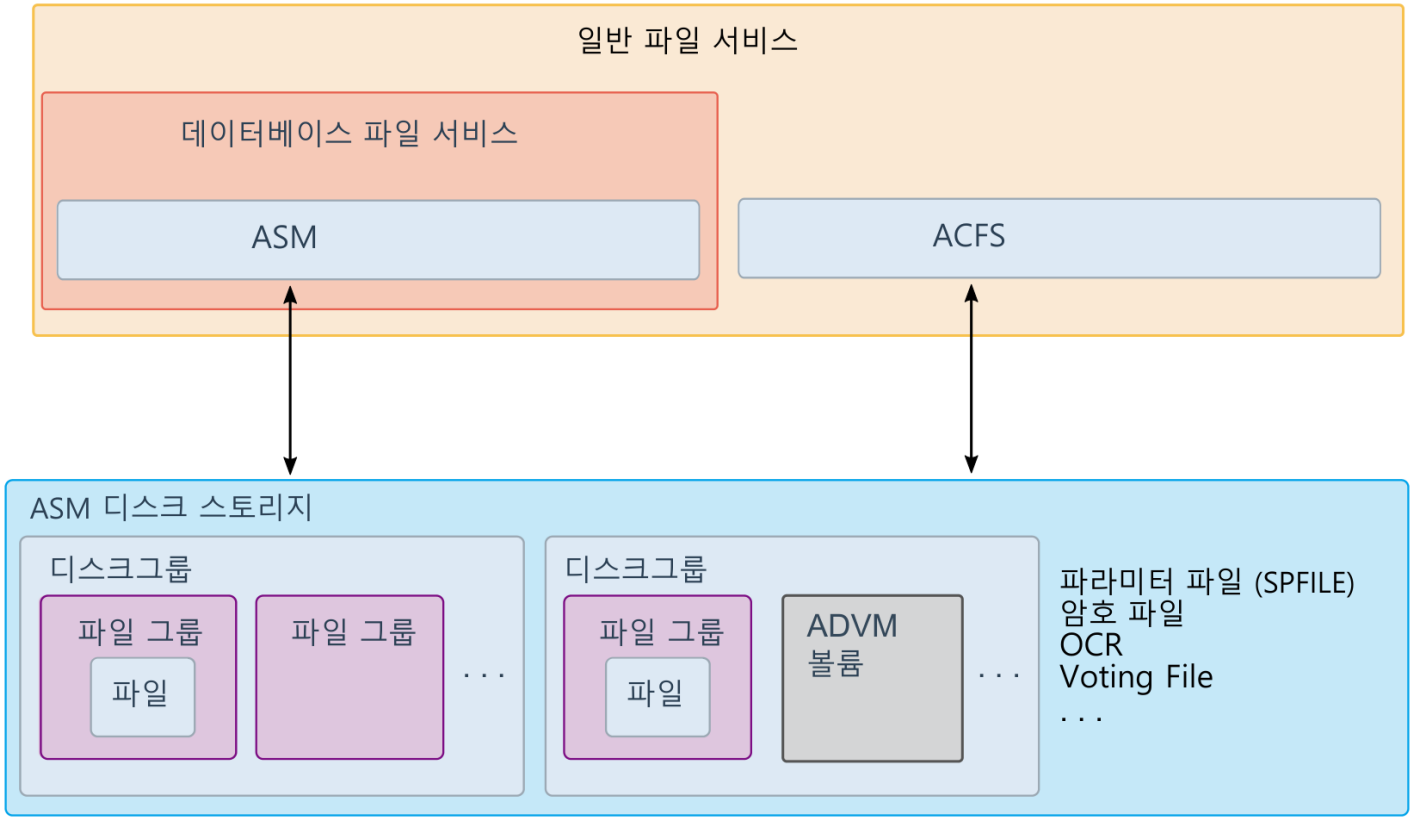
작은 SGA 와 ASM 전용 프로세스



- SGA — Shared Pool(메타데이터), Large Pool, ASM 캐시, 여유
- 최소 권장 256MB, 자동 메모리 관리
- RBAL 리밸런스 조정, ARBn 익스텐트 이동, GMON 멤버십

ASM 디스크 스토리지

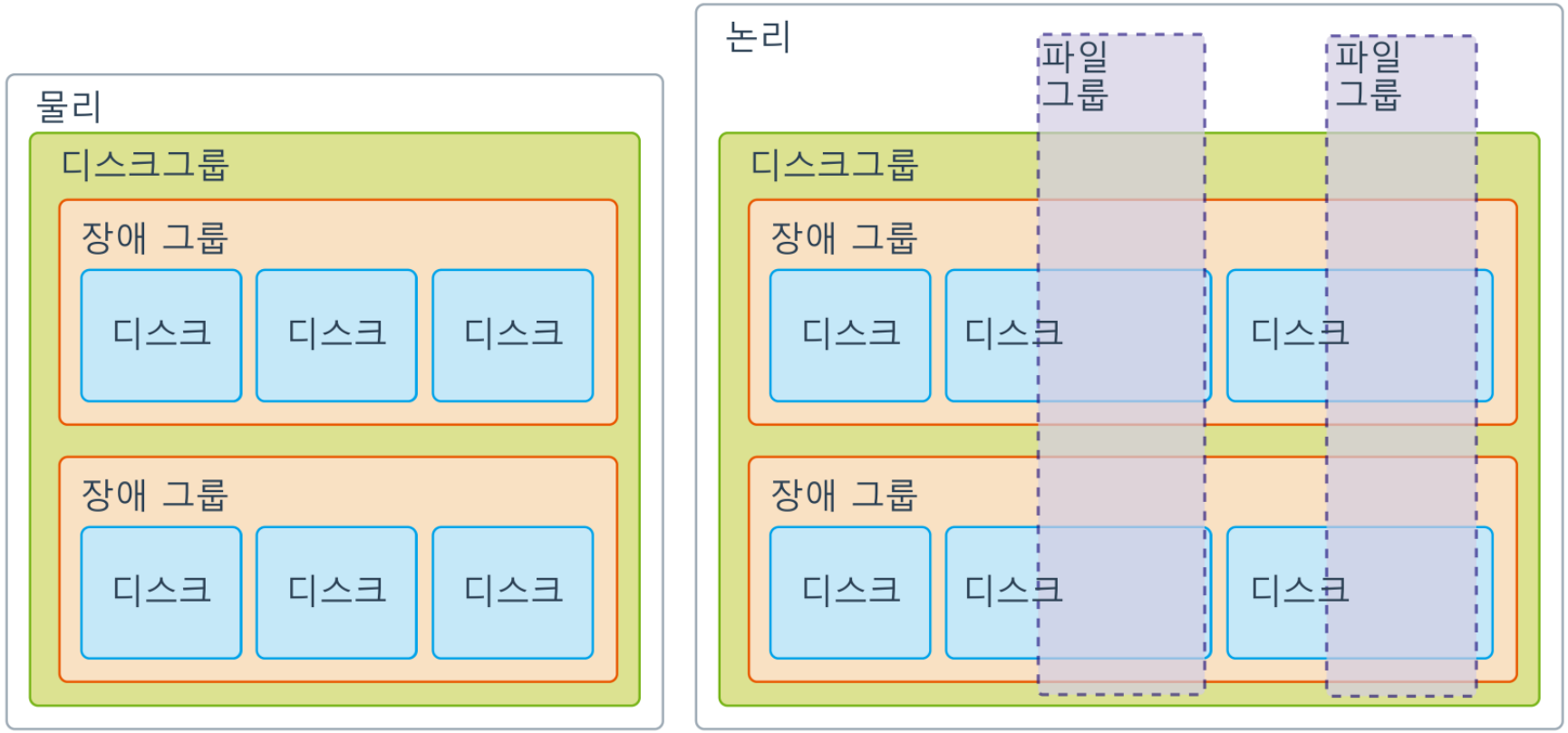
ASM 은 디스크그룹, ACFS 는 마운트 지점



- 물리 — ASM 디스크, 논리 — 디스크그룹
- 디스크 하나는 디스크그룹 하나에만
- SPFILE · 암호 파일 · OCR · Voting File 도 여기에

디스크그룹과 장애 그룹

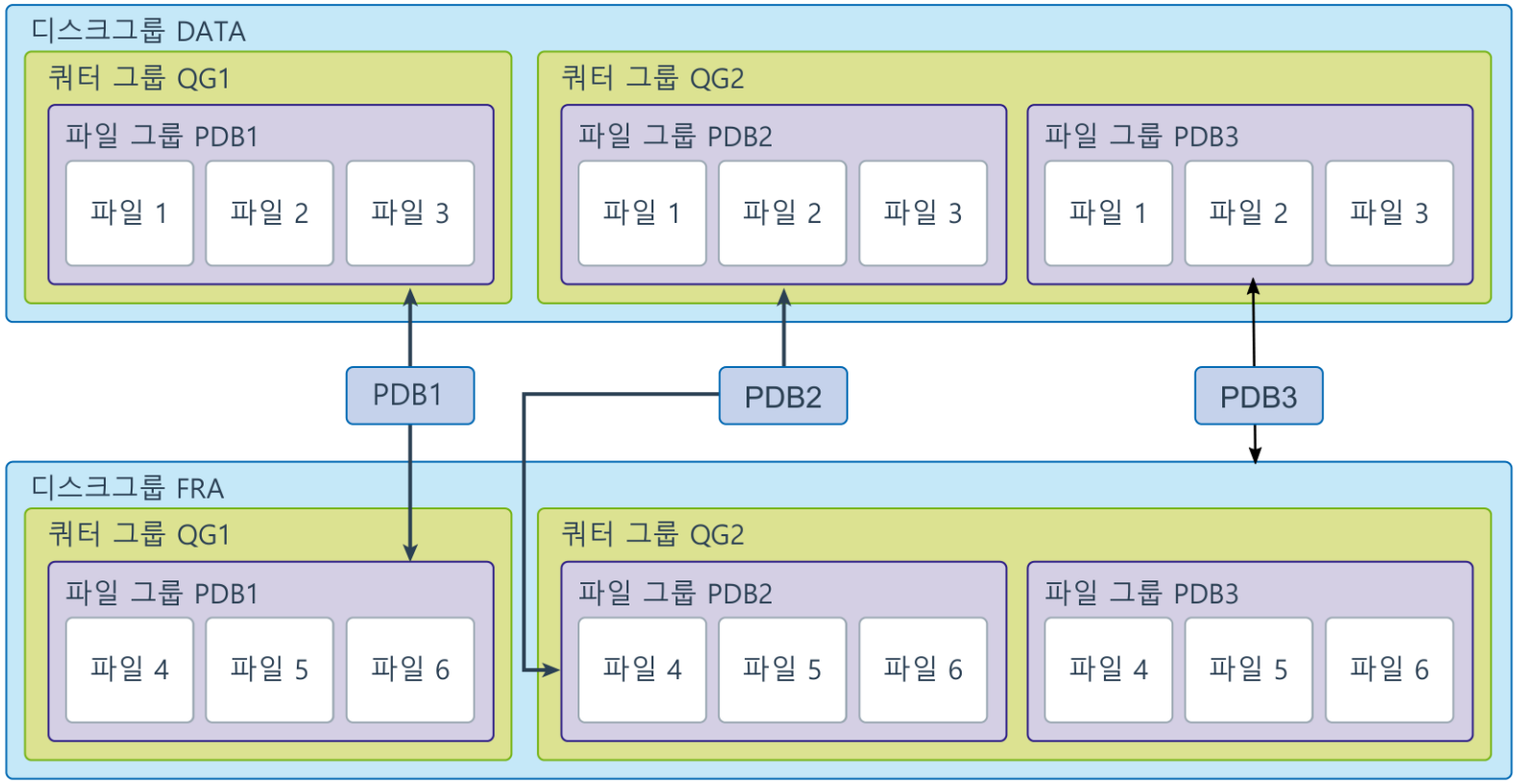
미러 사본은 다른 장애 그룹에



- 디스크마다 기본은 서로 다른 장애 그룹
- Normal 최소 2개, High 최소 3개
- 장애 그룹이 둘뿐이면 High 불가

파일 그룹과 쿼터 그룹

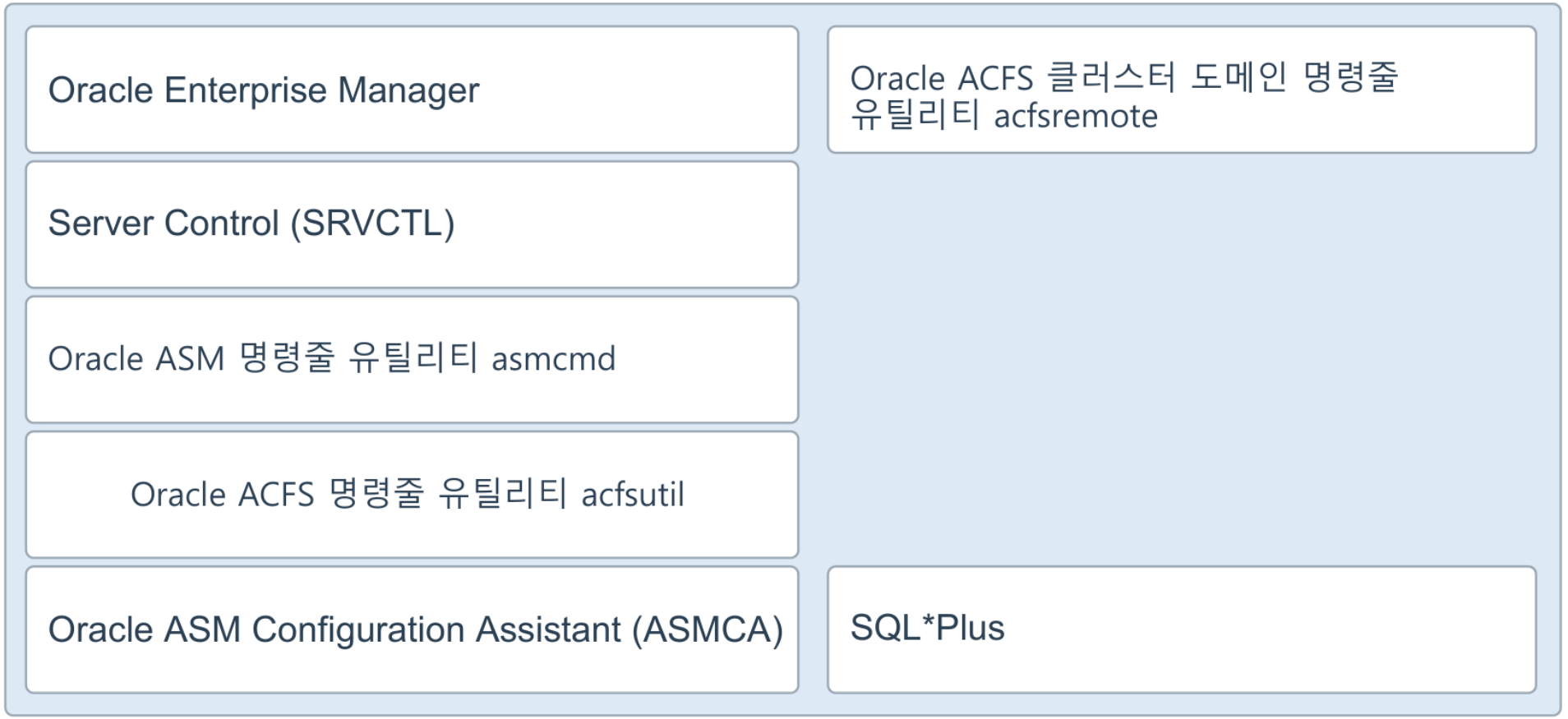
PDB 마다 파일 그룹, 이중화와 용량을 따로



- FLEX·EXTENDED 디스크그룹에서만
- PDB 는 디스크그룹마다 파일 그룹 하나
- 쿼터 그룹 하나에 파일 그룹 여럿

ASM 도구

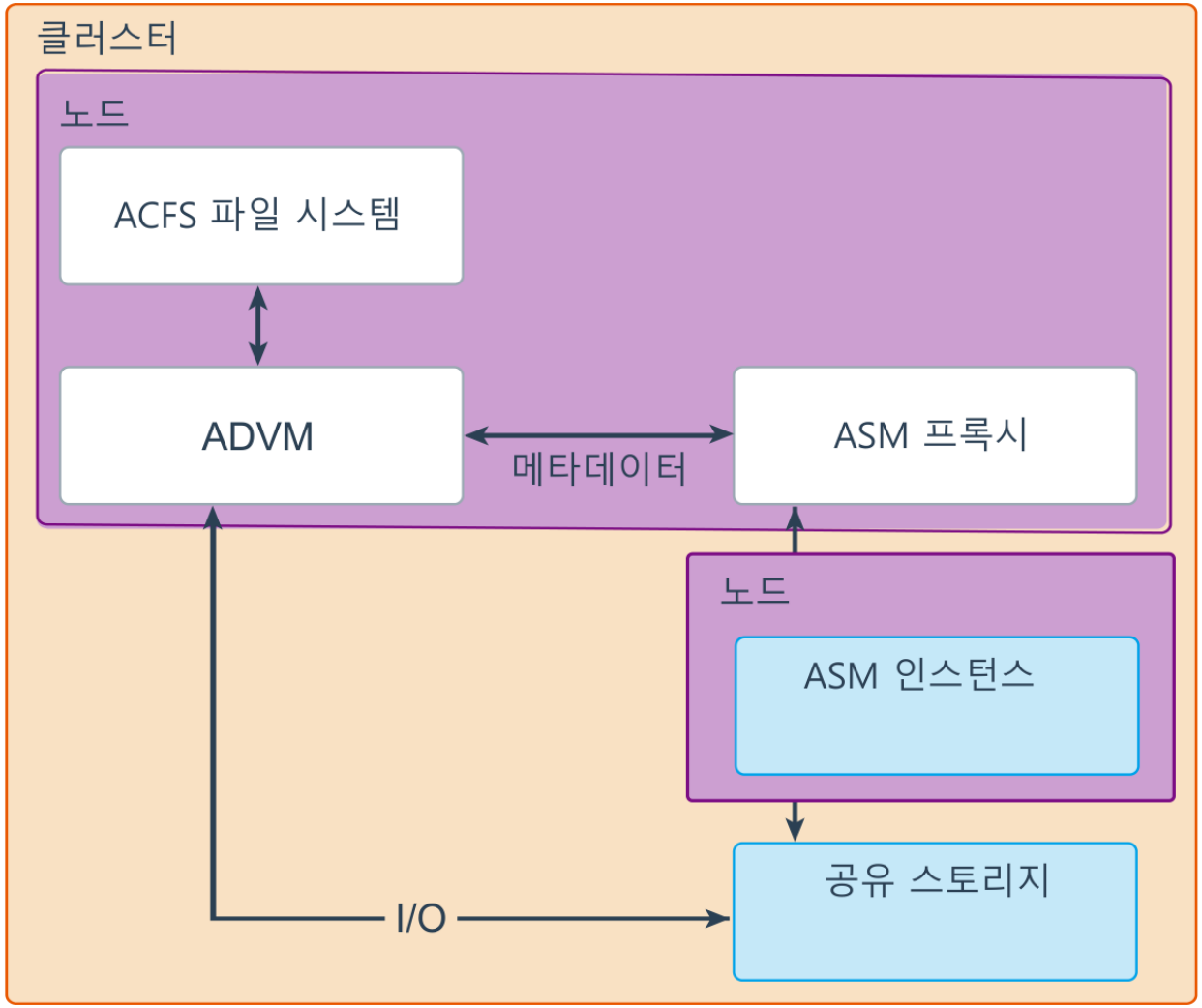
DB 도구 상당수가 ASM 에도 쓰인다



- ASMCA — GUI
- asmcmd · acfsutil · acfsremote — ASM·ACFS 전용 명령줄

ACFS

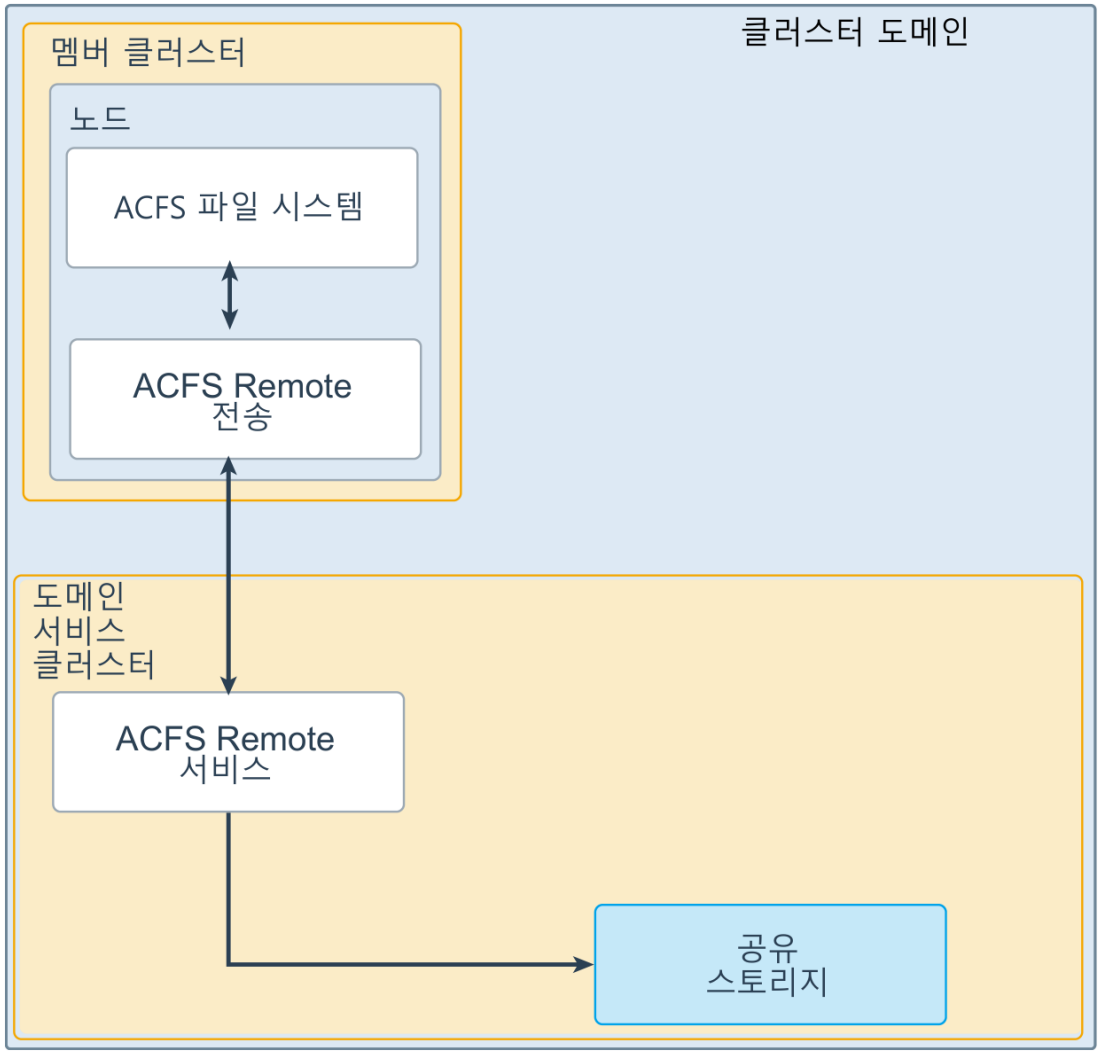
메타데이터는 ASM 프록시로, I/O 는 ADVM 이 직접



- POSIX 파일 시스템 — 실행 파일, 로그, 일반 파일
- Grid 홈과 ASM 진단 파일은 둘 수 없다
- ADVM 볼륨은 ASM 파일 — 크기를 동적으로 바꾼다

ACFS Remote

로컬 스토리지 없는 멤버 클러스터도 ACFS 를



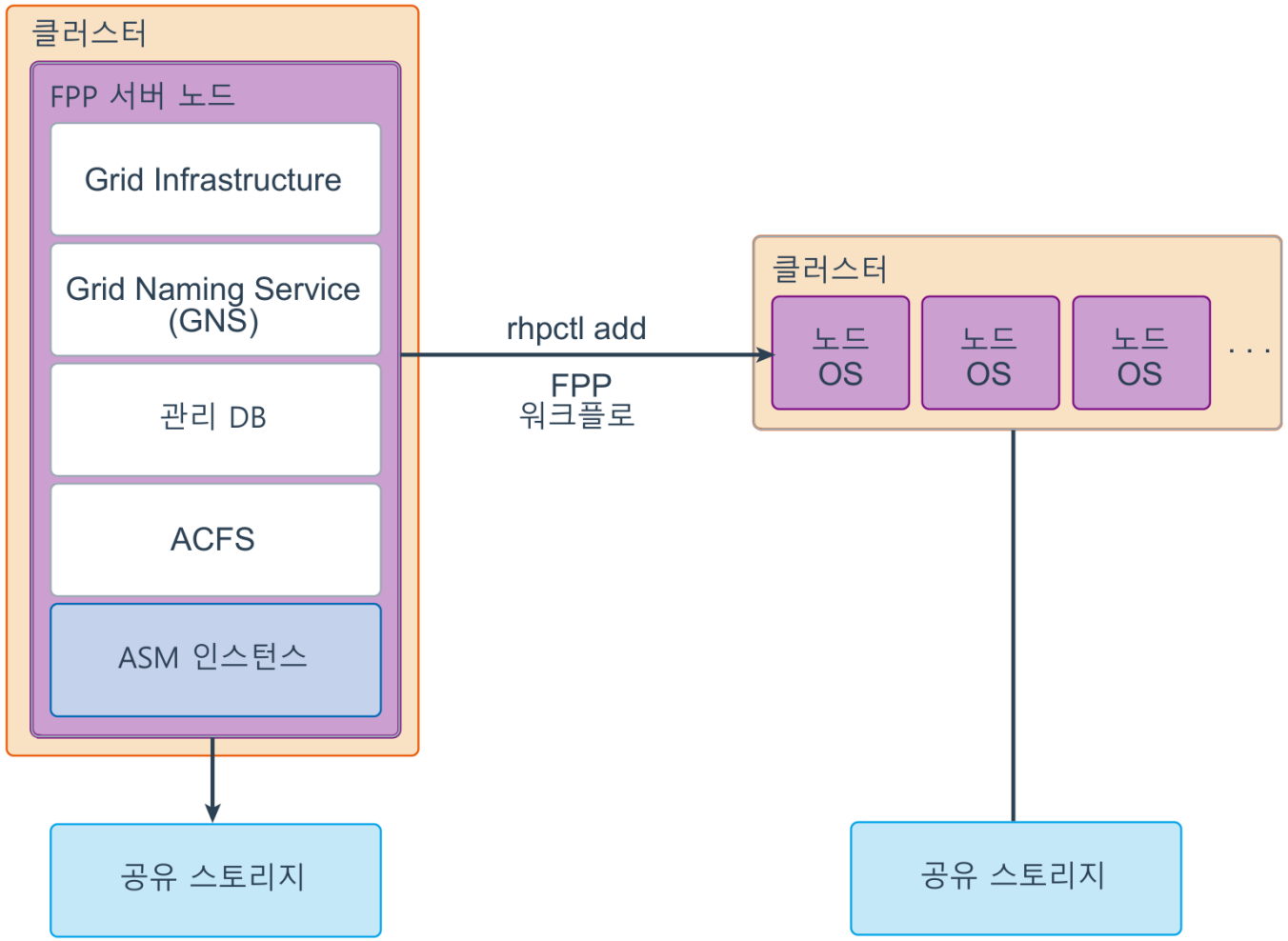
7

Fleet Patching and Provisioning

- FPP 서버와 대상
- 골드 이미지 워크플로

FPP 구성

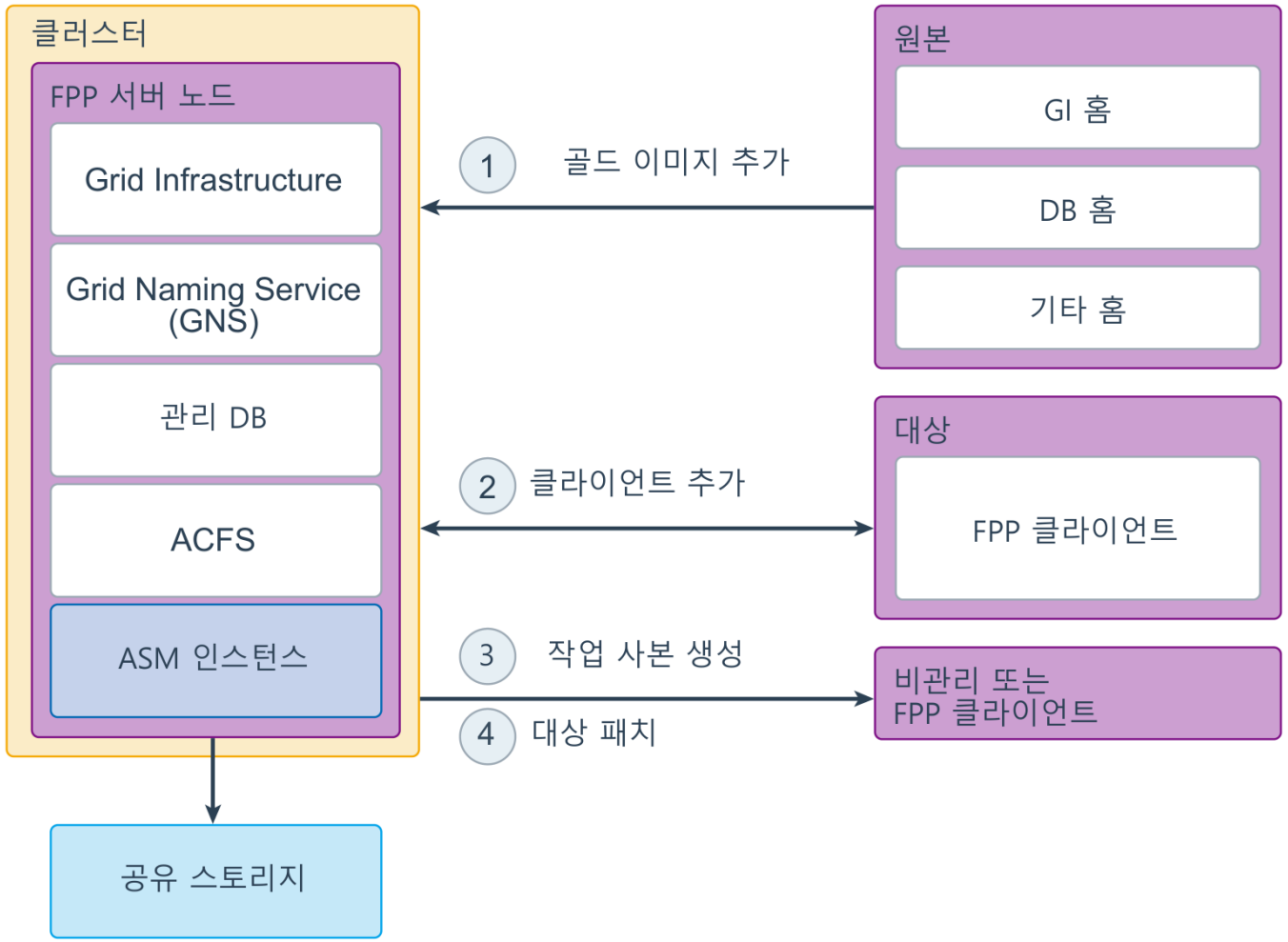
GI, GNS, 관리 DB, ACFS 를 갖춘 서버



- 예전 이름 Rapid Home Provisioning(RHP)
- 이미지 저장소는 ACFS 권장

FPP 워크플로

골드 이미지 → 클라이언트 → 작업 사본 → 패치



- 1 rhpctl import / add image — 골드 이미지
- 2 rhpctl add client
- 3 rhpctl add workingcopy — 클라이언트용 홈 생성
- 4 대상 패치도 같은 흐름

정리

- RAC — 데이터베이스 하나를 여러 인스턴스가 동시에 연다
- 공유 — 데이터 파일·컨트롤 파일·SPFILE / 인스턴스별 — 리두 스레드·UNDO (파일은 공유)
- Cache Fusion — 블록을 인터커넥트로, gc 통계와 대기로 관찰
- 접속은 SCAN, VIP 는 노드 장애 때 타임아웃을 막는다
- OCR 은 +CRS 에서 crsd 가, OLR 은 로컬에서 ohasd 가 읽는다
- Voting File 과반수에 닿는 노드만 남는다
- Clusterware 는 OHAS 와 CRS 두 스택 — 아래층부터 진단한다

확인 질문

- 1. racdb2 가 죽었을 때 racdb1 이 인스턴스 복구를 할 수 있는 이유는?
- 2. 인스턴스만 내려간 rac2 의 VIP 로 접속하면 어떤 오류가 나는가?
- 3. 2노드 클러스터에서 gc ... 3-way 대기가 거의 없는 이유는?
- 4. crsd 가 ocssd 보다 늦게 뜨는 이유를 OLR 과 OCR 의 위치로 설명하라
- 5. +CRS 디스크 하나를 OFFLINE 해도 클러스터가 도는 이유는?
- 6. crsctl check has 는 online 인데 check crs 가 CRS-4535 다. 어느 계층이 내려갔나?